Lossless Compressibility of Embeddings (20260113)

1. **Problem Statement**

   In large-scale LLM serving, the model often relies on external embedding stores for retrieval or context augmentation rather than keeping all information in its weights. These external stores behave like “memory” that the model queries during inference. At scale, managing this embedding data becomes primarily an SSD and I/O challenge rather than a model-architecture issue. Our goal is to evaluate how losslessly compressible these embeddings are and determine whether that compressibility yields tangible improvements in SSD capacity and read/write throughput.

2. **Motivation**

   RAG is used because model weights cannot contain all content. Data changes, private corpora cannot be trained in, and the long tail is too large. So systems retrieve chunks from an external corpus using embeddings.

   Embedding storage is large even at modest scale. Example: 100M vectors × 768 dims × INT8 = 100,000,000 × 768 × 1 byte ≈ 76.8 GB of raw vectors, before metadata/indexing. At FP16 that would be ~153.6 GB. Reducing raw vector bytes matters for cost, capacity planning, and I/O pressure.

   Bit-plane methods are attractive for INT and FP format. Many compressors work better when redundancy is exposed at the bit level. The hypothesis is that embeddings may have structure in higher bits (sign, exponent-like bits, or top magnitude bits) while lower bits are closer to noise, and bit-plane separation may improve locality and compressibility.

3. **What Data We Will Study (Embedding Corpora)**

   Priority is corpora with precomputed embedding. Here are some open corpora we might interested in:

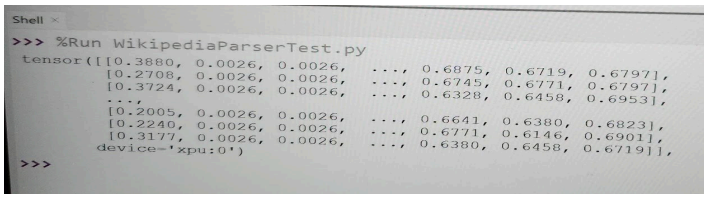
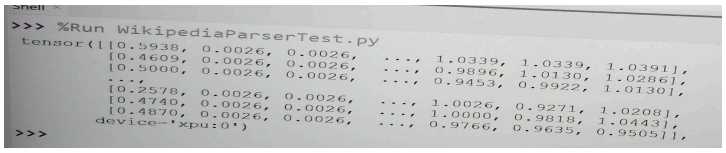
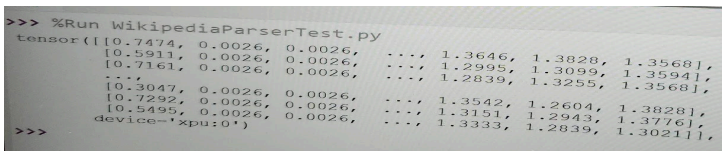
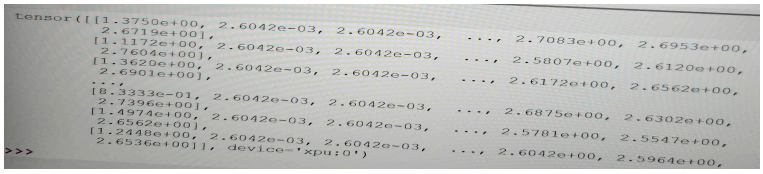
   - Wikipedia DPR passage embeddings (precomputed): large-scale passage embeddings suitable for direct vector-file compressibility studies. (https://huggingface.co/datasets/facebook/wiki_dpr)
   - TREC-RAG 2024 segmented corpus embeddings (precomputed): constructed for RAG-like retrieval at scale; closer to production-style “SSD-resident” embedding stores. (https://huggingface.co/datasets/Cohere/trec-rag-2024-index)
   - FineWeb-Edu embeddings / disk indexes (precomputed): very large-scale embeddings intended for disk-based vector search; useful for stress-testing compressibility at realistic scale. (https://huggingface.co/datasets/Cohere/fineweb-edu-emb)

01/22: Multivector Vertical Bitplane Disaggregation RLE Compression Results / Wikipedia_DPR (Tim update)

- Summary: Findings show that higher MSB bits will show higher compressibility than lower bits. Around the top 6 most significant bits will show the highest compressibility, then it drops off from there.
-

Findings:	Conclusion:	Note:
Top 6 MSB have highest compressibility, all else has low compressibility	Higher bits have high redundancy, hence can be highly compressed	Noticed that the very most MSB has lower compressibility than the next few MSB, maybe the sign bit is less compressible than the exponent bits
Compressibility is overall higher in smaller row counts/#vector embeddings. Compressibility drastically worsens after 1024 rows/vector embeddings	Blocks larger than 1024 embeddings may be too different to compress efficiently. Smaller groupings of embeddings have higher compressibility.	This smaller grouping compressibility may have to do with how similarity is calculated between vector embeddings. Vector embeddings that are next to each other/closer to eachother have higher similarity scores, hence will have more similar data, hence higher compressibility. Smaller groups will have higher similarity, hence higher compressibility. Larger groups have data that is further away from eachother on the ends, so they become less compressible. We may want to find the sweetspot for this between compressibility and block size.

- Data acquisition details:
    - File used: train-00000-of-00157.parquet
    - Part of dataset: multiset.exact
    - Starting row: 0
- Data:
-

Number of Rows:	Data
512	
768	
1024	
2048	

- How to interpret results:
    - Each column represents a bitplane, where leftmost column is MSB, and rightmost column is LSB
    - < 1 means compressed size is less than original size (closer to 0 means very high compressibility)
    - > 1 means compressed size is greater than original size (no compressibility, compression increases memory footprint)
- For next time:
    - Add analysis of horizontal bit plane single vector
    - Add analysis of averages to make it easier to compare

01/22: Single Vector Horizontal Bitplane Disaggregation RLE Compression Results / Wikipedia_DPR (Tim update)

- Summary: Single vector horizontal bitplane compression yields worse compression results than multi-vector vertical bitplane compression
- Data used:
    - Data file: train-00000-of-00157.parquet
    - Part of dataset: multiset.exact

    - Starting row: 0
- Data:
-
```
tensor([0.9714, 0.0078, 0.0078, 0.0078, 0.0078, 0.0755, 0.7760, 0.8932, 1.0052,
        0.9297, 0.9792, 1.0651, 1.0078, 0.9531, 1.0312, 0.9948, 0.9479, 1.0417,
        1.0156, 1.0312, 1.0521, 1.0052, 1.0182, 1.0026, 0.9740, 0.9505, 1.0677,
        0.9271, 1.0104, 0.9896, 0.9818, 0.9245], device='xpu:0')
```
- How to interpret results:
    - Only one row, each column represents 1 compression ratio of 1 bitplane, there are 32 elements
    - Leftmost bit is MSB, rightmost bit is LSB
    - < 1 means compressible, >1 means negative compressibility (compressed size > original size)

01/29: Added Entropy and Arithmetic coding settings / Wikipedia DPR (Tim update)

- Summary: seems like the first 8 MSBs have the least entropy, thus the highest compressibility
- Conclusion: may be that the exponents and signs are the most compressible, then the mantissa is not compressible, like noise
- Features added:
    - Entropy calculations per bit plane
    - Arithmetic coding analysis
- Data info:
    - File: train-00000-of-00157.parquet
    - Row: 0
    - # of rows: 512
- Data:
-
```
THEORETICAL AC COMPRESSIBILITY:
Plane   | Marginal Ratio   | Contextual Ratio    | Savings %
-----------------------------------------------------------------
0       | 1.00           :1 | 1.00              :1 | 0.1%
1       | 69.53          :1 | -23178936.03      :1 | 100.0%
2       | 69.53          :1 | -23178936.03      :1 | 100.0%
3       | 69.53          :1 | 1937.25           :1 | 99.9%
4       | 65.25          :1 | 119.07            :1 | 99.2%
5       | 7.11           :1 | 7.21              :1 | 86.1%
6       | 1.18           :1 | 1.57              :1 | 36.3%
7       | 1.05           :1 | 1.07              :1 | 6.4%
8       | 1.00           :1 | 1.00              :1 | 0.2%
9       | 1.02           :1 | 1.02              :1 | 2.1%
10      | 1.01           :1 | 1.01              :1 | 0.6%
```
    - This trend shows in the other row counts too. Bit planes 0-7 have notable highest compression, while everything else has almost no compressibility. MSB #6 is where the dropoff starts to occur. MSB #7 is the steep end of the dropoff. I also

curiously found that the MSB #0 has low compressibility, so maybe the sign is less compressible than we thought? At least it is less compressible with arithmetic coding, but is far more compressible with run length encoding.

1/31: Added more direct compression comparison + added vector embedding and bit plane compression averaging for multivector / Wikipedia DPR (Tim update)

- Summary: with more granular views over RLE lossless compression ratios for multivector vertical bitplane compression, we can more accurately and directly compare it
- Data information:
  - File used: train-00000-of-00157.parquet
  - Part of dataset: multiset.exact
  - Starting row: 0
  - # of rows: 768
- Data:

```
AVERAGE COMPRESSION RATIOS PER BIT PLANE:
tensor([0.4264, 0.0026, 0.0026, 0.0026, 0.0030, 0.0746, 0.6281, 0.8168, 0.9517,
        0.9662, 0.9956, 0.9992, 1.0042, 1.0024, 1.0035, 1.0014, 1.0025, 1.0026,
        1.0027, 1.0021, 1.0020, 1.0038, 1.0026, 1.0028, 1.0013, 1.0010, 1.0024,
        1.0034, 1.0037, 1.0039, 1.0026, 1.0029], device='xpu:0')

explanation of results above:
 - leftmost column is MSB
 - rightmost column is LSB
 - There is a single row, gets average of all 768 dimensions' bit planes of the # of rows you selected
 - compression ratio -> 0 means perfect compressibility, >1 means negative compressibility (takes more space to compress it than original)
```

-

```
AVG COMPRESSION RATIO PER VECTOR EMBEDDING:
tensor([0.8313, 0.8224, 0.8104, 0.7966, 0.8132, 0.8127, 0.8329, 0.8262, 0.8038,
        0.8298, 0.8101, 0.8265, 0.8097, 0.8157, 0.8180, 0.7948, 0.7792, 0.8219,
        0.8062, 0.8216, 0.8040, 0.7937, 0.7484, 0.8183, 0.8080, 0.7979, 0.7896,
        0.8180, 0.7953, 0.7936, 0.8230, 0.8000, 0.8114, 0.8206, 0.8171, 0.8228,
        0.8024, 0.8088, 0.8198, 0.7779, 0.8341, 0.8086, 0.8266, 0.8003, 0.8265,
        0.8179, 0.7417, 0.8072, 0.7946, 0.8226, 0.7961, 0.8092, 0.8192, 0.7929,
        0.8302, 0.8245, 0.7916, 0.8319, 0.8028, 0.8159, 0.8289, 0.7947, 0.8130,
        0.8262, 0.8215, 0.8115, 0.8304, 0.8258, 0.8100, 0.8127, 0.8199, 0.8118,
        0.8097, 0.8226, 0.8132, 0.8015, 0.8120, 0.8241, 0.8097, 0.8088, 0.8147,
        0.8219, 0.8110, 0.8095, 0.8122, 0.8114, 0.8132, 0.7970, 0.8050, 0.8185,
        0.8149, 0.8113, 0.8117, 0.8342, 0.8340, 0.8060, 0.7542, 0.8208, 0.8048,
        0.8057, 0.8107, 0.8229, 0.8341, 0.8108, 0.8106, 0.8328, 0.8080, 0.7976,
```

-

```
AVG COMPRESSION RATIO TOTAL BLOCK:
tensor(0.8101, device='xpu:0')
```

-
- If we want to just compare the avg compression ratio for the entire block:
-

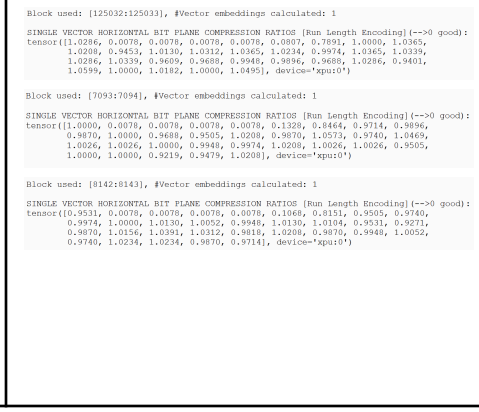
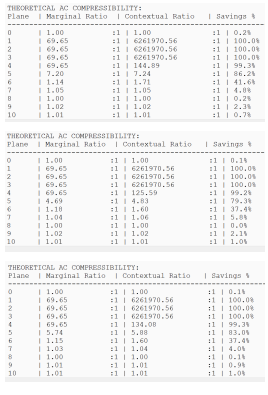
# of vector embeddings:	Avg Compression ratio:
1 (horizontal bitplane compression)	(not implemented yet)[manual math] 0.832921875
512 (vertical bitplane compression)	0.5406

768	0.8101
1024	1.0794 (negative compression)
2048	2.1534 (doubled negative compression)

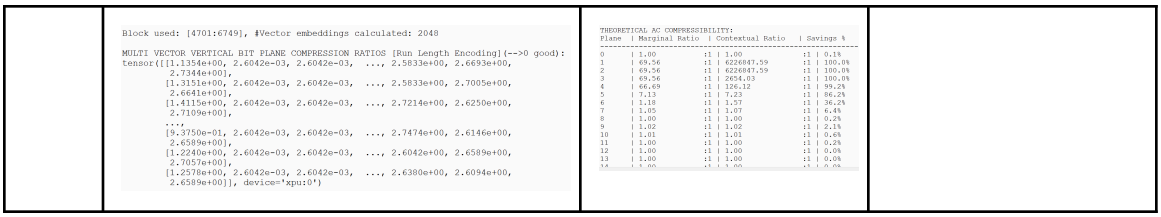
- Updated Conclusion: it seems like the sweetspot would be around 512 rows. We need to do more testing to find the actual sweetspot. Maybe file and row randomization to see how accurate this claim is.
- To do next:
    - Add file and row selection randomization (makes it less biased)
- In the future:
    - I should probably add averaging for single horizontal bitplane compression

2/1: Added better file randomization for getting more varied datapoints from a variety of .parquet files / Wikipedia DPR (Tim update)

- Summary:
- Features added:
    - Now it can randomly choose .parquet files and row locations per run, which gives more randomly chosen data. This can reduce bias.
- Now, did the trends stay the same? Yes
- Data: (performed 3 randomized runs for each)
-

# Vector embeddings/rows:	Compressibility tensor:	AC compressibility results:	Average block compressibility:
1			0.84830625 0.83838125 0.837075

512	Block used: [58937:59449], #Vector embeddings calculated: 512 MULTI VECTOR VERTICAL BIT PLANE COMPRESSION RATIOS [Run Length Encoding](-->0 good): tensor([[0.2917, 0.0026, 0.0026, ..., 0.6302, 0.6823, 0.6875], [0.3542, 0.0026, 0.0026, ..., 0.6771, 0.6745, 0.6693], [0.2630, 0.0026, 0.0026, ..., 0.6849, 0.7370, 0.7161], ..., [0.3177, 0.0026, 0.0026, ..., 0.6484, 0.6979, 0.6536], [0.3984, 0.0026, 0.0026, ..., 0.6927, 0.7005, 0.6380], [0.2656, 0.0026, 0.0026, ..., 0.7057, 0.6276, 0.6771]], device='xpu:0') Block used: [57094:57606], #Vector embeddings calculated: 512 MULTI VECTOR VERTICAL BIT PLANE COMPRESSION RATIOS [Run Length Encoding](-->0 good): tensor([[0.3203, 0.0026, 0.0026, ..., 0.6562, 0.6901, 0.6510], [0.3125, 0.0026, 0.0026, ..., 0.6198, 0.6328, 0.6458], [0.2734, 0.0026, 0.0026, ..., 0.6562, 0.7005, 0.7109], ..., [0.2969, 0.0026, 0.0026, ..., 0.7422, 0.6641, 0.6354], [0.4167, 0.0026, 0.0026, ..., 0.6875, 0.6641, 0.7188], [0.2578, 0.0026, 0.0026, ..., 0.6927, 0.6953, 0.6797]], device='xpu:0') Block used: [672:1184], #Vector embeddings calculated: 512 MULTI VECTOR VERTICAL BIT PLANE COMPRESSION RATIOS [Run Length Encoding](-->0 good): tensor([[0.4062, 0.0026, 0.0026, ..., 0.6693, 0.6953, 0.6953], [0.2161, 0.0026, 0.0026, ..., 0.6328, 0.6510, 0.6823], [0.4505, 0.0026, 0.0026, ..., 0.6849, 0.6719, 0.6693], ..., [0.0833, 0.0026, 0.0026, ..., 0.6953, 0.6589, 0.7240], [0.4714, 0.0026, 0.0026, ..., 0.6250, 0.6458, 0.6849], [0.2240, 0.0026, 0.0026, ..., 0.6823, 0.6432, 0.6875]], device='xpu:0')	THEORETICAL AC COMPRESSIBILITY: Plane | Marginal Ratio | Contextual Ratio | Savings % 0 | 1.00 :1 | 1.00 :1 | 0.1% 1 | 69.41 :1 | 2447836.96 :1 | 100.0% 2 | 69.42 :1 | 2447836.96 :1 | 100.0% 3 | 69.42 :1 | 2967.95 :1 | 100.0% 4 | 66.93 :1 | 128.61 :1 | 99.2% 5 | 7.25 :1 | 7.35 :1 | 86.4% 6 | 1.19 :1 | 1.56 :1 | 35.9% 7 | 1.05 :1 | 1.07 :1 | 6.3% 8 | 1.00 :1 | 1.00 :1 | 0.1% 9 | 1.02 :1 | 1.02 :1 | 2.2% 10 | 1.01 :1 | 1.01 :1 | 0.6% THEORETICAL AC COMPRESSIBILITY: Plane | Marginal Ratio | Contextual Ratio | Savings % 0 | 1.00 :1 | 1.00 :1 | 0.1% 1 | 69.65 :1 | 6261970.56 :1 | 100.0% 2 | 69.65 :1 | 6261970.56 :1 | 100.0% 3 | 69.65 :1 | 2293.21 :1 | 100.0% 4 | 66.18 :1 | 123.19 :1 | 99.2% 5 | 7.12 :1 | 7.22 :1 | 86.1% 6 | 1.19 :1 | 1.56 :1 | 36.1% 7 | 1.05 :1 | 1.07 :1 | 6.4% 8 | 1.00 :1 | 1.00 :1 | 0.1% 9 | 1.02 :1 | 1.02 :1 | 2.1% 10 | 1.01 :1 | 1.01 :1 | 0.6% THEORETICAL AC COMPRESSIBILITY: Plane | Marginal Ratio | Contextual Ratio | Savings % 0 | 1.00 :1 | 1.00 :1 | 0.1% 1 | 69.65 :1 | 6261970.56 :1 | 100.0% 2 | 69.65 :1 | 6261970.56 :1 | 100.0% 3 | 69.65 :1 | 2188.08 :1 | 100.0% 4 | 65.97 :1 | 122.04 :1 | 99.2% 5 | 7.10 :1 | 7.20 :1 | 86.1% 6 | 1.18 :1 | 1.56 :1 | 35.7% 7 | 1.05 :1 | 1.07 :1 | 6.1% 8 | 1.00 :1 | 1.00 :1 | 0.2% 9 | 1.02 :1 | 1.02 :1 | 2.1% 10 | 1.01 :1 | 1.01 :1 | 0.6%	0.54 0.5391 0.5391
768	Block used: [17972:18740], #Vector embeddings calculated: 768 MULTI VECTOR VERTICAL BIT PLANE COMPRESSION RATIOS [Run Length Encoding](-->0 good): tensor([[0.3776, 0.0026, 0.0026, ..., 1.0104, 1.0156, 1.0000], [0.6615, 0.0026, 0.0026, ..., 0.9583, 0.9740, 1.0182], [0.5521, 0.0026, 0.0026, ..., 1.0026, 1.0417, 1.0026], ..., [0.2995, 0.0026, 0.0026, ..., 0.9818, 1.0104, 0.9818], [0.4870, 0.0026, 0.0026, ..., 1.0885, 0.9896, 1.0469], [0.3490, 0.0026, 0.0026, ..., 0.9609, 1.0234, 0.9766]], device='xpu:0') Block used: [57687:58455], #Vector embeddings calculated: 768 MULTI VECTOR VERTICAL BIT PLANE COMPRESSION RATIOS [Run Length Encoding](-->0 good): tensor([[0.5182, 0.0026, 0.0026, ..., 0.9896, 1.0078, 1.0703], [0.5885, 0.0026, 0.0026, ..., 0.9115, 1.0026, 0.9349], [0.5495, 0.0026, 0.0026, ..., 0.9583, 1.0182, 1.0078], ..., [0.4609, 0.0026, 0.0026, ..., 1.0000, 1.0573, 1.0000], [0.7109, 0.0026, 0.0026, ..., 0.9583, 0.9505, 1.0052], [0.5052, 0.0026, 0.0026, ..., 0.9609, 0.9844, 0.9427]], device='xpu:0') Block used: [2893:3661], #Vector embeddings calculated: 768 MULTI VECTOR VERTICAL BIT PLANE COMPRESSION RATIOS [Run Length Encoding](-->0 good): tensor([[0.3984, 0.0026, 0.0026, ..., 1.0104, 0.9818, 1.0130], [0.5156, 0.0026, 0.0026, ..., 1.0000, 1.0964, 0.9635], [0.5938, 0.0026, 0.0026, ..., 0.9661, 1.0443, 1.0443], ..., [0.3021, 0.0026, 0.0026, ..., 1.0443, 0.9922, 0.9766], [0.4844, 0.0026, 0.0026, ..., 1.0339, 0.9818, 0.9922], [0.7188, 0.0026, 0.0026, ..., 1.0260, 0.9974, 1.0495]], device='xpu:0')	THEORETICAL AC COMPRESSIBILITY: Plane | Marginal Ratio | Contextual Ratio | Savings % 0 | 1.00 :1 | 1.00 :1 | 0.1% 1 | 69.65 :1 | 6261970.56 :1 | 100.0% 2 | 69.65 :1 | 6261970.56 :1 | 100.0% 3 | 69.65 :1 | 2841.93 :1 | 100.0% 4 | 67.00 :1 | 127.17 :1 | 99.2% 5 | 7.07 :1 | 7.18 :1 | 86.1% 6 | 1.17 :1 | 1.56 :1 | 36.1% 7 | 1.05 :1 | 1.07 :1 | 6.2% 8 | 1.00 :1 | 1.00 :1 | 0.2% 9 | 1.02 :1 | 1.02 :1 | 2.1% 10 | 1.01 :1 | 1.01 :1 | 0.6% THEORETICAL AC COMPRESSIBILITY: Plane | Marginal Ratio | Contextual Ratio | Savings % 0 | 1.00 :1 | 1.00 :1 | 0.1% 1 | 69.65 :1 | 6261970.56 :1 | 100.0% 2 | 69.65 :1 | 6261970.56 :1 | 100.0% 3 | 69.65 :1 | 2240.35 :1 | 100.0% 4 | 66.08 :1 | 123.18 :1 | 99.2% 5 | 7.22 :1 | 7.31 :1 | 86.3% 6 | 1.19 :1 | 1.56 :1 | 35.7% 7 | 1.05 :1 | 1.07 :1 | 6.3% 8 | 1.00 :1 | 1.00 :1 | 0.1% 9 | 1.02 :1 | 1.02 :1 | 2.1% 10 | 1.01 :1 | 1.01 :1 | 0.6% THEORETICAL AC COMPRESSIBILITY: Plane | Marginal Ratio | Contextual Ratio | Savings % 0 | 1.00 :1 | 1.00 :1 | 0.1% 1 | 69.65 :1 | 6261970.56 :1 | 100.0% 2 | 69.65 :1 | 6261970.56 :1 | 100.0% 3 | 69.65 :1 | 2618.45 :1 | 100.0% 4 | 66.71 :1 | 125.81 :1 | 99.2% 5 | 7.10 :1 | 7.21 :1 | 86.1% 6 | 1.18 :1 | 1.58 :1 | 36.6% 7 | 1.05 :1 | 1.07 :1 | 6.4% 8 | 1.00 :1 | 1.00 :1 | 0.2% 9 | 1.02 :1 | 1.02 :1 | 2.1% 10 | 1.01 :1 | 1.01 :1 | 0.6%	0.8085 0.8089 0.8076
1024	Block used: [1307:2331], #Vector embeddings calculated: 1024 MULTI VECTOR VERTICAL BIT PLANE COMPRESSION RATIOS [Run Length Encoding](-->0 good): tensor([[0.7891, 0.0026, 0.0026, ..., 1.3464, 1.3776, 1.3411], [0.7552, 0.0026, 0.0026, ..., 1.2969, 1.2500, 1.3307], [0.5755, 0.0026, 0.0026, ..., 1.3359, 1.3594, 1.3646], ..., [0.6068, 0.0026, 0.0026, ..., 1.3828, 1.3281, 1.3099], [0.7604, 0.0026, 0.0026, ..., 1.3021, 1.2396, 1.3464], [0.6276, 0.0026, 0.0026, ..., 1.3307, 1.4141, 1.3073]], device='xpu:0') Block used: [116862:117886], #Vector embeddings calculated: 1024 MULTI VECTOR VERTICAL BIT PLANE COMPRESSION RATIOS [Run Length Encoding](-->0 good): tensor([[0.7161, 0.0026, 0.0026, ..., 1.3932, 1.3906, 1.3958], [0.7370, 0.0026, 0.0026, ..., 1.3385, 1.3203, 1.2943], [0.8021, 0.0026, 0.0026, ..., 1.3906, 1.3880, 1.3177], ..., [0.5417, 0.0026, 0.0026, ..., 1.3490, 1.2969, 1.3125], [0.7760, 0.0026, 0.0026, ..., 1.2630, 1.3854, 1.3411], [0.5833, 0.0026, 0.0026, ..., 1.3359, 1.3724, 1.3672]], device='xpu:0') Block used: [29817:30841], #Vector embeddings calculated: 1024 MULTI VECTOR VERTICAL BIT PLANE COMPRESSION RATIOS [Run Length Encoding](-->0 good): tensor([[0.7005, 0.0026, 0.0026, ..., 1.3646, 1.3594, 1.3281], [0.7318, 0.0026, 0.0026, ..., 1.2526, 1.3125, 1.3151], [0.7214, 0.0026, 0.0026, ..., 1.2552, 1.3776, 1.2891], ..., [0.5911, 0.0026, 0.0026, ..., 1.3542, 1.3750, 1.4062], [0.8333, 0.0026, 0.0026, ..., 1.3307, 1.3438, 1.3958], [0.5573, 0.0026, 0.0026, ..., 1.2161, 1.3229, 1.2995]], device='xpu:0')	THEORETICAL AC COMPRESSIBILITY: Plane | Marginal Ratio | Contextual Ratio | Savings % 0 | 1.00 :1 | 1.00 :1 | 0.1% 1 | 69.59 :1 | 2453087.76 :1 | 100.0% 2 | 69.59 :1 | 2453087.76 :1 | 100.0% 3 | 69.59 :1 | 2439.30 :1 | 100.0% 4 | 66.40 :1 | 124.99 :1 | 99.2% 5 | 7.22 :1 | 7.31 :1 | 86.3% 6 | 1.19 :1 | 1.56 :1 | 35.9% 7 | 1.05 :1 | 1.07 :1 | 6.4% 8 | 1.00 :1 | 1.00 :1 | 0.2% 9 | 1.02 :1 | 1.02 :1 | 2.1% 10 | 1.01 :1 | 1.01 :1 | 0.6% THEORETICAL AC COMPRESSIBILITY: Plane | Marginal Ratio | Contextual Ratio | Savings % 0 | 1.00 :1 | 1.00 :1 | 0.1% 1 | 69.59 :1 | 2453087.76 :1 | 100.0% 2 | 69.59 :1 | 2453087.76 :1 | 100.0% 3 | 69.59 :1 | 2652.62 :1 | 100.0% 4 | 66.72 :1 | 126.68 :1 | 99.2% 5 | 7.22 :1 | 7.32 :1 | 86.3% 6 | 1.19 :1 | 1.56 :1 | 36.0% 7 | 1.05 :1 | 1.07 :1 | 6.3% 8 | 1.00 :1 | 1.00 :1 | 0.2% 9 | 1.02 :1 | 1.02 :1 | 2.1% 10 | 1.01 :1 | 1.01 :1 | 0.6% THEORETICAL AC COMPRESSIBILITY: Plane | Marginal Ratio | Contextual Ratio | Savings % 0 | 1.00 :1 | 1.00 :1 | 0.1% 1 | 69.65 :1 | 6261970.56 :1 | 100.0% 2 | 69.65 :1 | 6261970.56 :1 | 100.0% 3 | 69.65 :1 | 2920.09 :1 | 100.0% 4 | 67.10 :1 | 128.40 :1 | 99.2% 5 | 7.21 :1 | 7.31 :1 | 86.3% 6 | 1.19 :1 | 1.57 :1 | 36.3% 7 | 1.05 :1 | 1.07 :1 | 6.4% 8 | 1.00 :1 | 1.00 :1 | 0.2% 9 | 1.02 :1 | 1.02 :1 | 2.2% 10 | 1.01 :1 | 1.01 :1 | 0.6%	1.0772 1.0752 1.0773
2048	Block used: [31051:33099], #Vector embeddings calculated: 2048 MULTI VECTOR VERTICAL BIT PLANE COMPRESSION RATIOS [Run Length Encoding](-->0 good): tensor([[1.4297e+00, 2.6042e-03, 2.6042e-03, ..., 2.5677e+00, 2.7318e+00, 2.6328e+00], [1.4661e+00, 2.6042e-03, 2.6042e-03, ..., 2.6771e+00, 2.6589e+00, 2.6562e+00], [1.3854e+00, 2.6042e-03, 2.6042e-03, ..., 2.7083e+00, 2.6536e+00, 2.7109e+00], ..., [9.9479e-01, 2.6042e-03, 2.6042e-03, ..., 2.6250e+00, 2.6172e+00, 2.6458e+00], [1.3021e+00, 2.6042e-03, 2.6042e-03, ..., 2.6953e+00, 2.7214e+00, 2.6250e+00], [1.1927e+00, 2.6042e-03, 2.6042e-03, ..., 2.7135e+00, 2.5990e+00, 2.6823e+00]], device='xpu:0') Block used: [20192:22240], #Vector embeddings calculated: 2048 MULTI VECTOR VERTICAL BIT PLANE COMPRESSION RATIOS [Run Length Encoding](-->0 good): tensor([[1.1797e+00, 2.6042e-03, 2.6042e-03, ..., 2.6719e+00, 2.5573e+00, 2.7109e+00], [1.2422e+00, 2.6042e-03, 2.6042e-03, ..., 2.7839e+00, 2.5755e+00, 2.7083e+00], [1.0182e+00, 2.6042e-03, 2.6042e-03, ..., 2.6094e+00, 2.6068e+00, 2.6302e+00], ..., [1.0495e+00, 2.6042e-03, 2.6042e-03, ..., 2.6797e+00, 2.6198e+00, 2.6406e+00], [1.3125e+00, 2.6042e-03, 2.6042e-03, ..., 2.7318e+00, 2.6510e+00, 2.6510e+00], [1.2109e+00, 2.6042e-03, 2.6042e-03, ..., 2.6771e+00, 2.6562e+00, 2.6042e+00]], device='xpu:0')	THEORETICAL AC COMPRESSIBILITY: Plane | Marginal Ratio | Contextual Ratio | Savings % 0 | 1.00 :1 | 1.00 :1 | 0.1% 1 | 69.59 :1 | 2453087.76 :1 | 100.0% 2 | 69.59 :1 | 2453087.76 :1 | 100.0% 3 | 69.59 :1 | 2392.23 :1 | 100.0% 4 | 66.32 :1 | 124.44 :1 | 99.2% 5 | 7.19 :1 | 7.30 :1 | 86.3% 6 | 1.19 :1 | 1.56 :1 | 36.1% 7 | 1.05 :1 | 1.07 :1 | 6.4% 8 | 1.00 :1 | 1.00 :1 | 0.2% 9 | 1.02 :1 | 1.02 :1 | 2.1% 10 | 1.01 :1 | 1.01 :1 | 0.6% 11 | 1.00 :1 | 1.00 :1 | 0.1% 12 | 1.00 :1 | 1.00 :1 | 0.0% 13 | 1.00 :1 | 1.00 :1 | 0.0% 14 | 1.00 :1 | 1.00 :1 | 0.0% THEORETICAL AC COMPRESSIBILITY: Plane | Marginal Ratio | Contextual Ratio | Savings % 0 | 1.00 :1 | 1.00 :1 | 0.1% 1 | 69.65 :1 | 6261970.56 :1 | 100.0% 2 | 69.65 :1 | 6261970.56 :1 | 100.0% 3 | 69.65 :1 | 2176.61 :1 | 100.0% 4 | 65.94 :1 | 121.92 :1 | 99.2% 5 | 7.10 :1 | 7.20 :1 | 86.1% 6 | 1.18 :1 | 1.57 :1 | 36.3% 7 | 1.05 :1 | 1.07 :1 | 6.5% 8 | 1.00 :1 | 1.00 :1 | 0.2% 9 | 1.02 :1 | 1.02 :1 | 2.1% 10 | 1.01 :1 | 1.01 :1 | 0.6% 11 | 1.00 :1 | 1.00 :1 | 0.1% 12 | 1.00 :1 | 1.00 :1 | 0.0% 13 | 1.00 :1 | 1.00 :1 | 0.0% 14 | 1.00 :1 | 1.00 :1 | 0.0%	2.1524 2.1495 2.1516



- Overall, the same general pattern is observed even across multiple random runs
- For next time:
    - Maybe fix randomization bugs?

Please update your finding/results/insights/idea using format like

MM/DD: [Update Title / Dataset Name]

02/02: (Rui updated) Meeting – INT8 / Layout optimization

1. Before running the bit-plane analysis, please quantize the raw embedding into INT8 (symmetric scalar quantization). For beginning, use a single embedding vector (dim=768) as the granularity for calculating the scale.
    a. Table: quantization error (mean squared error) between the original FP32 and INT8 vectors to ensure the scale is applied correctly
    b. Histogram: A distribution plot of the INT8 value (from -128 to 127) to see if the range is fully utilized
2. Fix block-size compression. Apply bit-plane lossless compression to the INT8 data, test LZ4 and ZSTD using specific block sizes (4KB/8KB/16KB). This helps us simulate real-world SSD I/O behavior and page level compression.
    a. Bar chart: compression ratios for both LZ4 and ZSTD across different block sizes
    b. Bar chart: average compression ratio for each bit-plane
3. Explore the compression ratios of the columnar layout (group the values from the same index across many different embedding vectors and then perform the bit-plane split on that grouped data). This is expected to be effective because in most embedding models, certain dimensions carry specific semantic “meaning” and follow similar statistical distributions across the entire corpus. Grouping them this way maximizes the redundancy for the compressor.
    a. Document findings and any insights on which dimensions (MSBs/LSMs) show the most gain in this vertical arrangement.
    b. Bar chart: average compression ratio for each bit-plane

4. Just some personal note: Maintaining a document usually is a critical part of the research process. It is for your own professional benefit. It transforms the code into a structured study that your future self can actually use and reference.

02/06: Initial Symmetric Scalar Quantization results for Row-wise quantization and dequantization / Wikipedia DPR: (Tim update)

- Summary: It seems that symmetric scalar quantization results in a lower accuracy drop than expected looking at base numbers.
- Progress: Still in early stages of symmetric scalar quantization
    - What is currently done:
        - Row-wise symmetric scalar quantization
        - Row-wise symmetric scalar dequantization
        - Row-wise, Element-wise, overall, and worst case Mean Squared Error for dequantized results vs original results (although it isn't in a neat print formatting yet)
    - What needs further work:
        - Column-wise symmetric scalar quantization + dequantization + error analytics
- Data info:
    - Dataset used: train-00000-of-00157.parquet
    - Starting row: 0
    - # of rows: 2048
- Here are some results:
    - Original values:

```
0       [-0.07233894, 0.4803533, 0.18650995, -0.528708...
1       [-0.3405247, 0.5054066, -0.021264639, -0.36572...
2       [-0.21922281, 0.30289492, -0.07657898, -0.0742...
3       [-0.008607165, 0.49791786, -0.012508, -0.32356...
4       [0.12540339, 0.6188803, -0.036911376, -0.35676...
                                ...
2043    [0.16996974, 0.13034898, 0.3170804, 0.10499755...
2044    [0.5259842, 0.11507118, 0.018592665, 0.3359378...
2045    [0.5963158, 0.09202226, -0.0077379416, 0.35206...
2046    [0.81674725, 0.2372343, 0.016581243, 0.4861677...
2047    [0.2865936, 0.3444998, 0.22202535, 0.29200166,...
Name: embeddings, Length: 2048, dtype: object
```

    - Quantized values:

```
tensor([[ -1,  10,   4,  ...,  -7,  13, -14],
        [ -7,  10,   0,  ...,  -5,   8,  -9],
        [ -4,   6,  -2,  ...,  -3,   6, -14],
        ...,
        [ 12,   2,   0,  ...,  -4,  -2,  -1],
        [ 17,   5,   0,  ...,  -5,  -6,   5],
        [  6,   7,   4,  ...,   2,   8,   4]], device='xpu:0',
       dtype=torch.int8)
```

- Scalars:

```
tensor([[0.0498],
        [0.0498],
        [0.0508],
        ...,
        [0.0492],
        [0.0488],
        [0.0497]], device='xpu:0')
```

- Dequantized Values:

```
tensor([[-0.0498,  0.4982,  0.1993,  ..., -0.3487,  0.6476, -0.6974],
        [-0.3489,  0.4984,  0.0000,  ..., -0.2492,  0.3987, -0.4486],
        [-0.2030,  0.3045, -0.1015,  ..., -0.1523,  0.3045, -0.7105],
        ...,
        [ 0.5904,  0.0984,  0.0000,  ..., -0.1968, -0.0984, -0.0492],
        [ 0.8296,  0.2440,  0.0000,  ..., -0.2440, -0.2928,  0.2440],
        [ 0.2980,  0.3477,  0.1987,  ...,  0.0993,  0.3974,  0.1987]],
       device='xpu:0')
```

- Average Mean Square Error:

```
tensor(0.0002, device='xpu:0')
```

- Element-wise Mean Square Error:

```
tensor([[5.0730e-04, 3.1696e-04, 1.6263e-04,  ..., 1.4448e-04, 2.6555e-04,
         2.6398e-05],
        [6.9908e-05, 4.8976e-05, 4.5218e-04,  ..., 3.3676e-05, 4.3377e-04,
         3.4938e-04],
        [2.6302e-04, 2.6004e-06, 6.2118e-04,  ..., 3.2780e-04, 9.2595e-06,
         1.8348e-04],
        ...,
        [3.5218e-05, 4.0636e-05, 5.9876e-05,  ..., 5.4913e-06, 1.1256e-04,
         2.6704e-04],
        [1.6393e-04, 4.5579e-05, 2.7494e-04,  ..., 1.0395e-04, 2.0425e-04,
         5.3723e-07],
        [1.3074e-04, 1.0234e-05, 5.4477e-04,  ..., 1.6015e-05, 6.2925e-05,
         5.5500e-05]], device='xpu:0')
```

- Row-wise Mean Square Error:

```
tensor([0.0002, 0.0002, 0.0002,  ..., 0.0002, 0.0002, 0.0002], device='xpu:0')
```

- Row with worst Mean Square Error:

```
tensor(1499, device='xpu:0')
```

- Findings:
  - I noticed that while comparing the original values vs the quantized values, there were some notable gaps between certain elements:

Original:	Dequantized:	Observations:
0 [-0.07233894, 0.4803533, 0.18650995, -0.528708... 1 [-0.3405247, 0.5054066, -0.021264639, -0.36572... 2 [-0.21922281, 0.30289492, -0.07657898, -0.0742... 3 [-0.008607165, 0.49791786, -0.012508, -0.32356... 4 [0.12540339, 0.6188803, -0.036911376, -0.35676... ... 2043 [0.16996974, 0.13034898, 0.3170804, 0.10499755... 2044 [0.5259842, 0.11507118, 0.018592665, 0.3359378... 2045 [0.5963158, 0.09202226, -0.0077379416, 0.35206... 2046 [0.81674725, 0.2372343, 0.016581243, 0.4861677... 2047 [0.2865936, 0.3444998, 0.22202535, 0.29200166,... Name: embeddings, Length: 2048, dtype: object	tensor([[-0.0498, 0.4982, 0.1993, ..., [-0.3489, 0.4984, 0.0000, ..., [-0.2030, 0.3045, -0.1015, ..., ..., [0.5904, 0.0984, 0.0000, ..., [0.8296, 0.2440, 0.0000, ..., [0.2980, 0.3477, 0.1987, ..., device='xpu:0')	The top left elements on each, -0.0723 on the original vs -0.0498 on the dequantized value. That is a ~31% error (not mse). Outliers like these suggest that they might be “spikes” that get clamped the most aggressively.

- But if we look at the element-wise mean square error between the dequantized and original values, it has very small amounts of error:

```
tensor([[5.0730e-04, 3.1696e-04, 1.6263e-04,  ..., 1.4448e-04, 2.6555e-04,
         2.6398e-05],
        [6.9908e-05, 4.8976e-05, 4.5218e-04,  ..., 3.3676e-05, 4.3377e-04,
         3.4938e-04],
        [2.6302e-04, 2.6004e-06, 6.2118e-04,  ..., 3.2780e-04, 9.2595e-06,
         1.8348e-04],
        ...,
        [3.5218e-05, 4.0636e-05, 5.9876e-05,  ..., 5.4913e-06, 1.1256e-04,
         2.6704e-04],
        [1.6393e-04, 4.5579e-05, 2.7494e-04,  ..., 1.0395e-04, 2.0425e-04,
         5.3723e-07],
        [1.3074e-04, 1.0234e-05, 5.4477e-04,  ..., 1.6015e-05, 6.2925e-05,
         5.5500e-05]], device='xpu:0')
```

  - They are at most around a 0.05% MSE

    - This is supported even when we look at the row-wise error:
    - `tensor([0.0002, 0.0002, 0.0002, ..., 0.0002, 0.0002, 0.0002], device='xpu:0')`
        - The average row-wise MSE is ~0.02% per row, which is quite low, and it is consistent across the rows visible
- I also noticed something that might warrant some further analysis:
    - Scalars seem to be similar in value, and might also be highly compressible.
    -
```
tensor([[0.0498],
        [0.0498],
        [0.0508],
        ...,
        [0.0492],
        [0.0488],
        [0.0497]], device='xpu:0')
```
    - Many elements in the quantized format seem to be similar vertically, suggesting that vertical bitplane compression may result in higher compressibility than horizontal bitplane compression
    -
```
tensor([[ -1,  10,   4,  ...,  -7,  13, -14],
        [ -7,  10,   0,  ...,  -5,   8,  -9],
        [ -4,   6,  -2,  ...,  -3,   6, -14],
        ...,
        [ 12,   2,   0,  ...,  -4,  -2,  -1],
        [ 17,   5,   0,  ...,  -5,  -6,   5],
        [  6,   7,   4,  ...,   2,   8,   4]], device='xpu
       dtype=torch.int8)
```
        - It also seems like the signs are somewhat grouped together, so the sign bits might be locally compressible
- Next steps:
    - Work on the vertical/columnar symmetric scalar quantization + dequantization + error analysis

02/06: Update: Initial Symmetric Scalar Quantization Results for Columnar/Vertical scalar quantization (multivector) / Wikipedia DPR: (Tim Update)

- Summary: It seems that performing Symmetric Scalar Quantization vertically improves the accuracy and precision of dequantization when compared to Symmetric Scalar Quantization per vector embedding.
- Data used:
    - File used: train-00000-of-00157.parquet
    - Starting row: 0
    - Number of rows: 2048
- Findings + Data:

- Dequantized data seems even more similar to its original vs the quantization per vector embedding:

Original:	Dequantized:	Observations:
0 [-0.07233894, 0.4803533, 0.18650995, -0.528708... 1 [-0.3405247, 0.5054066, -0.021264639, -0.36572... 2 [-0.21922281, 0.30289492, -0.07657898, -0.0742... 3 [-0.008607165, 0.49791786, -0.012508, -0.32356... 4 [0.12540339, 0.6188803, -0.036911376, -0.35676... ... 2043 [0.16996974, 0.13034898, 0.3170804, 0.10499755... 2044 [0.5259842, 0.11507118, 0.018592665, 0.3359378... 2045 [0.5963158, 0.09202226, -0.0077379416, 0.35206... 2046 [0.81674725, 0.2372343, 0.016581243, 0.4861677... 2047 [0.2865936, 0.3444998, 0.22202535, 0.29200166,...	tensor([[-0.0729, 0.4785, 0.1840, ..., -0.3389, 0.6331, -0.7011], [-0.3402, 0.5055, -0.0251, ..., -0.2562, 0.4171, -0.4308], [-0.2187, 0.3033, -0.0753, ..., -0.1736, 0.2979, -0.7265], ..., [0.5994, 0.0944, -0.0084, ..., -0.1984, -0.1117, -0.0338], [0.8181, 0.2359, 0.0167, ..., -0.2314, -0.3054, 0.2450], [0.2835, 0.3437, 0.2258, ..., 0.0992, 0.4022, 0.2027]], device='xpu:0')	Most of the differences are now in the 0.0x or 0.00x point instead of at the 0.0x and 0.x point.

- Lets see if there are any differences in the Mean Square Errors (MSE):
- This confirms my initial observations, the elementwise MSE and row-wise MSE are considerably lower.
- Elementwise results:

Horizontal Symmetric Scalar MSE:	Vertical Symmetric Scalar MSE: (new)	Observations
tensor([[5.0730e-04, 3.1696e-04, 1.6263e-04, ..., 1.4448e-04, 2.6555e-04, 2.6398e-05], [6.9908e-05, 4.8976e-05, 4.5218e-04, ..., 3.3676e-05, 4.3377e-04, 3.4938e-04], [2.6302e-04, 2.6004e-06, 6.2118e-04, ..., 3.2780e-04, 9.2595e-06, 1.8348e-04], ..., [3.5218e-05, 4.0636e-05, 5.9876e-05, ..., 5.4913e-06, 1.1256e-04, 2.6704e-04], [1.6393e-04, 4.5579e-05, 2.7494e-04, ..., 1.0395e-04, 2.0425e-04, 5.3723e-07], [1.3074e-04, 1.0234e-05, 5.4477e-04, ..., 1.6015e-05, 6.2925e-05, 5.5500e-05]], device='xpu:0')	tensor([[3.1791e-07, 3.3391e-06, 6.3689e-06, ..., 4.7071e-06, 3.0790e-06, 2.0119e-06], [9.7196e-08, 6.1715e-09, 1.4626e-05, ..., 1.4485e-06, 6.1353e-06, 8.9230e-07], [2.6469e-07, 1.5697e-07, 1.7210e-06, ..., 1.0259e-05, 1.2621e-05, 5.8489e-06], ..., [9.6535e-06, 5.4521e-06, 3.9071e-07, ..., 6.0901e-07, 7.3484e-06, 8.6996e-07], [1.9152e-06, 1.7989e-06, 2.0962e-08, ..., 5.6336e-06, 2.9397e-06, 2.9711e-06], [9.5035e-06, 5.9272e-07, 1.4258e-05, ..., 1.4728e-05, 9.7439e-06, 1.1532e-05]], device='xpu:0')	The MSE is significantly lower, the exponents are e-5 or less (more negative), while the horizontal method has e-4. The actual MSE improvement is anywhere from 10x to 10,000x less MSE. This is a significant improvement

- Column-wise results:
- These results go off the screen, but are almost all with an exponent e-6, which suggests their column MSEs are vastly lower than the Row MSEs from horizontal symmetric quantization of 0.0002

```
tensor([5.4719e-06, 3.7626e-06, 5.5397e-06, 2.9854e-06, 5.2837e-06, 5.5600e-06,
        4.9417e-06, 4.8721e-06, 4.8736e-06, 4.8029e-06, 4.8070e-06, 4.3750e-06,
        4.5334e-06, 6.2181e-06, 2.9212e-06, 6.5169e-06, 7.5907e-06, 4.5772e-06,
        4.9509e-06, 3.5977e-06, 4.2023e-06, 7.2027e-06, 1.4859e-05, 4.7329e-06,
        7.3984e-06, 5.0224e-06, 1.1789e-05, 6.6244e-06, 6.1151e-06, 5.2590e-06,
        5.4769e-06, 5.2198e-06, 5.6919e-06, 5.0955e-06, 7.8638e-06, 3.8845e-06,
        7.6917e-06, 4.8742e-06, 4.1995e-06, 1.0679e-05, 3.5748e-06, 4.1020e-06,
        3.4559e-06, 6.5272e-06, 3.5095e-06, 4.9792e-06, 2.3381e-05, 8.2189e-06,
        5.9671e-06, 5.1101e-06, 7.6005e-06, 4.3639e-06, 4.7259e-06, 1.3434e-05,
        3.7329e-06, 3.4438e-06, 4.4141e-06, 4.3300e-06, 4.2787e-06, 4.9128e-06,
        6.8002e-06, 6.8267e-06, 4.0002e-06, 4.2518e-06, 3.5475e-06, 7.2480e-06,
        3.0968e-06, 3.6951e-06, 5.0560e-06, 5.7351e-06, 4.6104e-06, 6.2828e-06,
        5.2366e-06, 3.5465e-06, 3.2343e-06, 7.0604e-06, 5.7253e-06, 4.1136e-06,
        6.0899e-06, 9.2552e-06, 6.9822e-06, 3.8961e-06, 4.2613e-06, 5.5368e-06,
        3.6720e-06, 4.1891e-06, 3.8204e-06, 4.3969e-06, 6.6131e-06, 6.4994e-06,
        7.2210e-06, 5.1911e-06, 5.8344e-06, 3.7123e-06, 3.9967e-06, 4.4188e-06,
        1.4679e-05, 5.7489e-06, 3.6095e-06, 4.6897e-06, 6.3066e-06, 4.9337e-06,
```

- Average MSE:

```
tensor(6.0392e-06, device='xpu:0')
```

  - This is vastly lower than the horizontal symmetric scalar method with 0.0002

- I also noticed some trends about the quantized values and scalars:
  - The scalars for vertical versus horizontal shows that there are actually less scalars to store when there are many rows (when #rows > 768). This means that there are less scalars to compress.
    - Horizontal will always have 768 scalars because there are 768 dimensions per vector embedding,
    - while vertical will have as many scalars as there are rows of vector embeddings.

```
tensor([[0.0081, 0.0067, 0.0084, 0.0060, 0.0079, 0.0082, 0.0078, 0.0076, 0.0077,
         0.0077, 0.0076, 0.0073, 0.0074, 0.0087, 0.0059, 0.0089, 0.0094, 0.0075,
         0.0078, 0.0067, 0.0072, 0.0091, 0.0133, 0.0074, 0.0094, 0.0077, 0.0119,
         0.0089, 0.0086, 0.0080, 0.0081, 0.0080, 0.0083, 0.0079, 0.0097, 0.0070,
         0.0097, 0.0075, 0.0071, 0.0113, 0.0066, 0.0071, 0.0065, 0.0089, 0.0065,
         0.0077, 0.0169, 0.0102, 0.0085, 0.0080, 0.0095, 0.0072, 0.0076, 0.0125,
         0.0066, 0.0065, 0.0073, 0.0072, 0.0072, 0.0076, 0.0090, 0.0091, 0.0069,
         0.0073, 0.0066, 0.0093, 0.0060, 0.0066, 0.0077, 0.0082, 0.0076, 0.0087,
         0.0079, 0.0064, 0.0063, 0.0092, 0.0084, 0.0071, 0.0086, 0.0107, 0.0091,
         0.0068, 0.0072, 0.0081, 0.0067, 0.0070, 0.0066, 0.0072, 0.0089, 0.0089,
         0.0093, 0.0079, 0.0082, 0.0067, 0.0070, 0.0074, 0.0133, 0.0082, 0.0066,
         0.0075, 0.0087, 0.0077, 0.0072, 0.0099, 0.0109, 0.0062, 0.0076, 0.0089,
         0.0089, 0.0095, 0.0104, 0.0068, 0.0085, 0.0070, 0.0096, 0.0109, 0.0081,
         0.0092, 0.0111, 0.0059, 0.0080, 0.0103, 0.0084, 0.0080, 0.0067, 0.0083,
         0.0067, 0.0118, 0.0074, 0.0095, 0.0088, 0.0096, 0.0080, 0.0076, 0.0074,
         0.0108, 0.0088, 0.0083, 0.0125, 0.0067, 0.0068, 0.0077, 0.0099, 0.0078,
         0.0074, 0.0067, 0.0069, 0.0075, 0.0084, 0.0072, 0.0070, 0.0065, 0.0089,
```

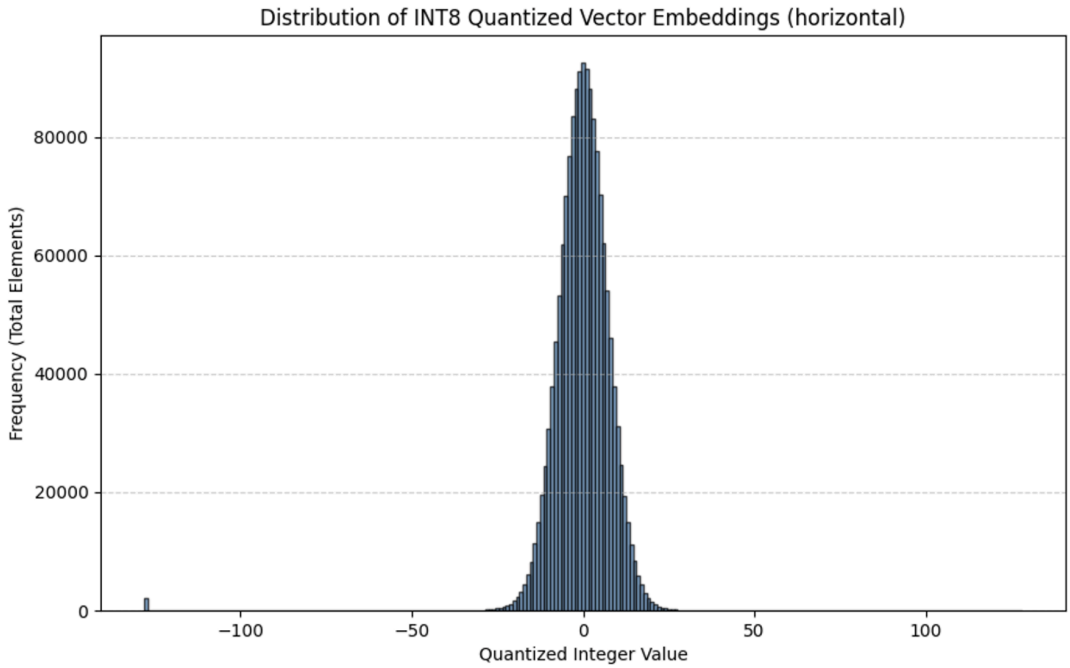
    - (it will go off the page)
  - The quantized values seem like they might be less compressible overall with this method though, so there might be a trade-off?

```
tensor([[ -9,  71,  22,  ..., -41,  85, -83],
        [-42,  75,  -3,  ..., -31,  56, -51],
        [-27,  45,  -9,  ..., -21,  40, -86],
        ...,
        [ 74,  14,  -1,  ..., -24, -15,  -4],
        [101,  35,   2,  ..., -28, -41,  29],
        [ 35,  51,  27,  ...,  12,  54,  24]], device='xpu
       dtype=torch.int8)
```

    - They still seem somewhat compressible vertically, but I am not seeing the same trend as I did with the horizontal quantized values
- Conclusion:
    - Vertical Symmetric Scalar Quantization may be more effective for recovering INT8 quantized values back into their original FP32 values
- Next steps/further experimentation:
    - Are vertically quantized values truly less compressible than horizontally quantized values? (when performing vertical bitplane compression)
    - We need to start doing bitplane formatting for both vertical and horizontal (will probably reuse some code for this)
    - I will also need to get the histogram distribution for quantization range

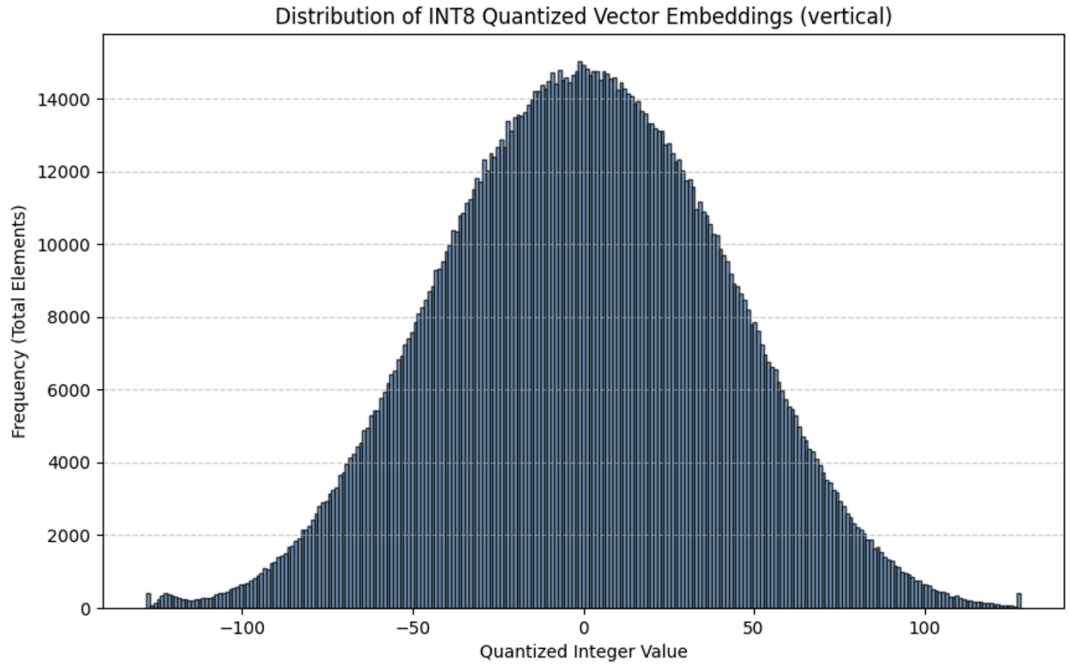
02/06: Symmetric Scalar Quantization for vertical and horizontal applications, Histograms / Wikipedia DPR: (Tim update)

- Summary: The horizontal quantization has a very narrow range (within +/- 25), while the vertical quantization has a very wide range (+/- 127).
- Data used:
    - File: train-00000-of-00157.parquet
    - Row start: 0
    - # rows: 2048
- Data:
    - Horizontal Quantization:



-

- Vertical Quantization:



-

- Next steps/further experimentation:
  - Are vertically quantized values truly less compressible than horizontally quantized values? (when performing vertical bitplane compression)

- We need to start doing bitplane formatting for both vertical and horizontal (will probably reuse some code for this)(already mostly finished)

02/07: Vertical and Horizontal Bitplane compression for both vertical and horizontal symmetric scalar quantization (RLE compression analysis) / Wikipedia DPR: (Tim update)

- Summary: Horizontal Symmetric Scalar Quantization is more highly compressible than Vertical Symmetric Scalar Quantization and no quantization.
- Data info:
  - File: train-00000-of-00157.parquet
  - Start row: 0
- Data gathered:

# of vector embeddings:	Avg Compression ratio (FP32, no quantization, from 1/31):	Avg Compression ratio (Horizontal Symmetric Quantization, INT8):	Avg Compression ratio (Horizontal Symmetric Quantization, FP32 scalars):	Avg Compression ratio (Vertical Symmetric Quantization, INT8):	Avg Compression ratio (Vertical Symmetric Quantization, FP32 scalars):
1 (horizontal bitplane compression)	(not implemented yet)[manual math] 0.832921875	0.9775	N/A	N/A	N/A
512 (vertical bitplane compression)	0.5406	0.4605	0.4188	0.5924	0.8236
768	0.8101	0.6909	0.6277	0.8849	0.8280
1024	1.0794 (negative compression)	0.9228	0.8424	1.1766	0.8297
2048	2.1534 (doubled negative compres	1.8191	1.6810	2.3279	0.8280

	sion)				

- Patterns observed:
    - Vertical Symmetric Scalar Quantization is generally worse than with no quantization at all when it comes to RLE compressibility
    - Horizontal Symmetric Scalar Quantization is generally better for RLE compressibility compared to the results without quantization
    - Horizontal Symmetric Scalar Quantization scales better with higher #vector embeddings compared to both Vertical Symmetric Scalar Quantization and FP32 (no quantization).
    - Horizontal Symmetric Scalar Quantization (HSSQ) has even better efficiency gains vs no quantization as the # of vector embeddings increase, although it still hits diminishing returns.
    - Vertical Symmetric Scalar Quantization's scalar compression ratios stay very consistent regardless of the # of vector embeddings, while HSSQ's scalar compression ratios become lower at higher # of vector embeddings.
- Conclusions:
    - For RLE bitplane compression analysis, Horizontal Symmetric Scalar Quantization yields the highest redundancy, hence, the highest compressibility.
    - Vertical Symmetric Scalar Quantization is less efficient than no quantization at all for RLE bitplane compression analysis.
- Short update: Some other patterns observed:
    - It seems that the compressibility of these horizontal quantized vector embeddings (with vertical bit planes) have similar/exact compressibility ratios within groups, and will have fairely ascending compression ratios from left to right across bitplanes.
        - It might be a good idea to check similarity of these groups to further enhance compressibility
    - Here is a picture of the 512 vector embedding results (using the same data file):
        - Quantized values:

```
[horizontal quant, quant] MULTI VECTOR VERTICAL BIT PLANE COMPRESSION 1
tensor([[0.3880, 0.3880, 0.3880,  ..., 0.7292, 0.6068, 0.7161],
        [0.2656, 0.2656, 0.2656,  ..., 0.7083, 0.6797, 0.6536],
        [0.3411, 0.3411, 0.3411,  ..., 0.6745, 0.6875, 0.6328],
        ...,
        [0.2083, 0.2083, 0.2083,  ..., 0.6432, 0.7266, 0.6771],
        [0.2292, 0.2292, 0.2292,  ..., 0.6380, 0.5781, 0.6849],
        [0.3177, 0.3177, 0.3177,  ..., 0.6380, 0.7083, 0.6536]],
       device='xpu:0')
```

        -
            - Over here, we have a matrix that is 768 tall by 8 wide.
                - Left to right is bitplane MSB compression ratio to bitplane LSB compression ratio
                - Top to bottom is dim0 of all vector embeddings all the way down to dim768 of all vector embeddings
            - Notice how every row exhibits the same pattern, where the top 3 MSB have the same compression ratio and it is low

        - Not sure if this is a coincidence, will have to try more data ranges to find out
    -
```
[horizontal quant, quant] AVERAGE COMPRESSION RATIOS PER BIT PLANE:
tensor([0.2817, 0.2817, 0.2819, 0.3163, 0.5299, 0.6589, 0.6669, 0.6667],
       device='xpu:0')
```
        - Notice here, there are 8 values, each corresponding to the average compression ratio of each bitplane. (leftmost is MSB, rightmost is LSB).
            - The compression on the top 3 MSB is the highest, and they start to trail off afterwards, albeit almost in an ‘S’ curve
            - This is speculation, needs more data to confirm
    - Scalar values:
    -
```
[horizontal quant, scale] MULTI VECTOR VERTICAL BIT PLANE COMPRESSION RATIOS [Run Length Encoding](-->
tensor([[0.0026, 0.0026, 0.0026, 0.0026, 0.0026, 0.0026, 0.0026, 0.0026, 0.0026,
         0.0807, 0.0807, 0.1901, 0.5104, 0.6042, 0.6354, 0.6641, 0.6458, 0.6849,
         0.6615, 0.6667, 0.6771, 0.7266, 0.6302, 0.7161, 0.6172, 0.6354, 0.6328,
         0.6510, 0.6198, 0.6901, 0.6536, 0.7031]], device='xpu:0')
```
        - This shows 32 values, each corresponding to the compressibility of each bit plane in the FP32 scalars. Leftmost is MSB, rightmost is LSB.
            - The first 9 MSB have the same very low compression ratios (this suggests very high compressibility). This aligns with the sign bit, and the exponent bits.
            - The rest of the bits in this example are still fairely compressible, although definitely less compressible than the top 9 bits.
- Speculative Observations that may need more investigation:
    - [Horizontal Symmetric Scalar Quantization]
        - [INT8 quantized values] Leftmost 3 MSB bitplanes have the same compression ratio (and are lowest)
        - [FP32 scalar values] Top 9 MSB have the same lowest compression ratio and correspond to the sign bit and exponent bit
        - Both need to have their similarity values calculated to see if we can further compress them by grouping. Maybe the similarity observed here might show up as very compressible patterns for LZ4 and ZSTD
- For next time:
    - Implement LZ4 and ZSTD compression analysis per bitplane and per vector embedding using 4KB blocks
    - Maybe add instructions in print for how to interpret results, it is getting confusing

02/09: Initial LZ4 and ZSTD Results for single and multivector compression (quantized, and scaled) / Wikipedia DPR:

- Summary: The quantized values seem to be very weakly compressible, often leading to negative compression. The first 3 bits of INT8 quantized values seem reasonably

compressible, although not as strongly as Run-Length Encoding compression. The scales, are all highly compressible in comparison.

- Data used:
    - File: train-00000-of-00157.parquet
    - Starting Row: 0
    - # of Rows: 512
- Data:
    - Multi Row:
        - Row-wise quantization:
        - Quantized values:

Bitplane	Global(ZSTD)	Global(LZ4)	Temporal(ZSTD)	Temporal(LZ4)	Spatial(ZSTD)	Spatial(LZ4)
7	1.000	1.000	1.139	1.358	1.013	1.030
6	1.000	1.000	1.139	1.358	1.013	1.030
5	1.000	1.000	1.139	1.358	1.013	1.030
4	0.959	1.000	1.139	1.358	1.003	1.030
3	0.749	0.909	1.117	1.335	0.829	1.027
2	0.687	0.833	1.046	1.273	0.793	1.020
1	0.686	0.833	1.046	1.273	0.793	1.020
0	0.686	0.833	1.046	1.273	0.792	1.020

        - Scales:

Bitplane	Scale Vertical (ZSTD)	Scale Vertical (LZ4)
31	0.036	0.042
30	0.036	0.042
29	0.036	0.042
28	0.036	0.042
27	0.036	0.042
26	0.036	0.042
25	0.036	0.042
24	0.036	0.042
23	0.036	0.042
22	0.036	0.042
21	0.036	0.042
20	0.036	0.042
19	0.036	0.042
18	0.036	0.042
17	0.036	0.042
16	0.036	0.042
15	0.036	0.042
14	0.036	0.042
13	0.036	0.042
12	0.036	0.042
11	0.031	0.041
10	0.017	0.026
9	0.017	0.026
8	0.008	0.017
7	0.008	0.017
6	0.008	0.017
5	0.008	0.017
4	0.008	0.017
3	0.008	0.017
2	0.008	0.017
1	0.008	0.017
0	0.008	0.017

- Column-wise Quantization:
- Quantized values:

Bitplane	Global(ZSTD)	Global(LZ4)	Temporal(ZSTD)	Temporal(LZ4)	Spatial(ZSTD)	Spatial(LZ4)
7	1.000	1.000	1.141	1.359	1.013	1.03
6	1.000	1.000	1.141	1.359	1.013	1.03
5	1.000	1.000	1.141	1.359	1.013	1.03
4	1.000	1.000	1.141	1.359	1.013	1.03
3	1.000	1.000	1.140	1.359	1.013	1.03
2	1.000	1.000	1.139	1.358	1.013	1.03
1	0.919	0.999	1.139	1.358	0.981	1.03
0	0.688	0.837	1.048	1.275	0.791	1.02

- Scales:

Bitplane	Scale Vertical (ZSTD)	Scale Vertical (LZ4)
31	0.034	0.039
30	0.034	0.039
29	0.034	0.039
28	0.034	0.039
27	0.034	0.039
26	0.034	0.039
25	0.034	0.039
24	0.034	0.039
23	0.034	0.039
22	0.034	0.039
21	0.034	0.039
20	0.034	0.039
19	0.034	0.039
18	0.034	0.039
17	0.034	0.039
16	0.034	0.039
15	0.034	0.039
14	0.034	0.039
13	0.034	0.039
12	0.034	0.039
11	0.034	0.039
10	0.034	0.039
9	0.034	0.039
8	0.034	0.039
7	0.034	0.039
6	0.034	0.039
5	0.034	0.039
4	0.006	0.011
3	0.006	0.011
2	0.006	0.011
1	0.006	0.011
0	0.006	0.011

- Single Row (Row-Wise only):
    - Quantized values:

Bitplane	Global(ZSTD)	Global(LZ4)
7	0.137	0.155
6	0.137	0.155
5	0.137	0.155
4	0.137	0.155
3	0.137	0.155
2	0.137	0.155
1	0.137	0.155
0	0.137	0.155

-
- Scale:

```
Single Scale Compression:
Original Size: 4 bytes
ZSTD Ratio: 3.25
LZ4 Ratio: 4.75
```

-
- Now, lets compare the 512 row results and single row from last time with our current ones: (these numbers are hand calculated for now due to the code not having averages for these yet)

-

# of vector embeddings:	Avg Compression ratio (Horizontal Symmetric Quantization, INT8) [RLE]:	Avg Compression ratio (Horizontal Symmetric Quantization, FP32 scalars) [RLE]:	Avg Compression ratio (Vertical Symmetric Quantization, INT8) [RLE]:	Avg Compression ratio (Vertical Symmetric Quantization, FP32 scalars) [RLE]:	Average Compression ratio (Horizontal Symmetric Quantization INT8)[LZ4 / ZSTD]	Average Compression ratio (Horizontal Symmetric Quantization FP32 scalars) [LZ4 / ZSTD]	Average Compression ratio (Vertical Symmetric Quantization INT8) [LZ4 / ZSTD]	Average Compression ratio (Vertical Symmetric Quantization FP32 scalars) [LZ4 / ZSTD]
1 (horizontal bitplane compression)	0.9775	N/A	N/A	N/A	0.155 / 0.137	4.75 / 3.25	N/A	N/A

512 (vertical bitplane compression)	0.4605	0.4188	0.5924	0.8236	0.926 / 0.845875 (Global)	0.0339375/ 0.02678125	0.9795 / 0.950875 (Global)	0.034625 / 0.029625

- These are still early numbers. Global is the closest comparison to the rest in terms of how it organizes data for compression.
    - Global: Takes a “slice” of the entire tensor, which is 1 bitplane across all dimensions and vector embeddings. Calculates the LZ4 / ZSTD compression per slice (granularity: 768 x #rows)
    - Temporal: Takes a “needle” out of a tensor. This is a single bitplane of a single vector embedding dimension across all vector embedding rows. (granularity: 1 x #rows)
    - Spatial: For a single vector embedding, for each bitplane, it calculates LZ4/ZSTD compression of all vector dimensions. (granularity: 1 x 768)
- Conclusion: There may be a chance that LZ4/ZSTD compression isn’t very effective on these quantized bitplanes. I do need to look deeper in to this to say for sure
- To do next time:
    - Verify fully that the compression is happening at the correct granularity
    - Compare at multiple row sizes
    - Check for storage overhead
    - Add Block Size support for LZ4 and ZSTD

02/10: Mini update: LZ4 and ZSTD Compression for higher values (Quantized and scaled) / Wikipedia DPR: (Tim update)

- Hypothesis: Higher #rows/vector embeddings might yield higher compression ratios for LZ4 and ZSTD due to it having a longer window for pattern recognition.
- Summary: Compared to at 512 vector embeddings, 4096 vector embeddings yields slightly better compressibility in LZ4 and ZSTD in Global mode, but yields considerably better compression in Temporal and Spatial modes.
- Data info:
    - File: train-00000-of-00157.parquet
    - Row start: 0
    - # Rows: 4096
- Data gained:
    - Horizontal Symmetric Scalar Quantization:
    - Quantized Values (INT8):

Bitplane	Global(ZSTD)	Global(LZ4)	Temporal(ZSTD)	Temporal(LZ4)	Spatial(ZSTD)	Spatial(LZ4)
7	1.000	1.000	1.018	1.044	1.013	1.030
6	1.000	1.000	1.018	1.044	1.013	1.030
5	1.000	1.000	1.018	1.044	1.013	1.030
4	0.956	1.000	1.018	1.044	1.006	1.030
3	0.737	0.904	0.873	0.983	0.825	1.027
2	0.673	0.822	0.805	0.905	0.789	1.021
1	0.672	0.822	0.805	0.904	0.789	1.021
0	0.672	0.822	0.805	0.904	0.789	1.021

- Scalar Values (FP32):

Bitplane	Scale Vertical (ZSTD)	Scale Vertical (LZ4)
31	0.032	0.033
30	0.032	0.033
29	0.032	0.033
28	0.032	0.033
27	0.032	0.033
26	0.032	0.033
25	0.032	0.033
24	0.032	0.033
23	0.032	0.033
22	0.032	0.033
21	0.032	0.033
20	0.032	0.033
19	0.032	0.033
18	0.032	0.033
17	0.032	0.033
16	0.032	0.033
15	0.032	0.033
14	0.032	0.033
13	0.032	0.033
12	0.032	0.033
11	0.020	0.027
10	0.009	0.013
9	0.009	0.013
8	0.001	0.002
7	0.001	0.002
6	0.001	0.002
5	0.001	0.002
4	0.001	0.002
3	0.001	0.002
2	0.001	0.002
1	0.001	0.002
0	0.001	0.002

- Vertical Symmetric Scalar Quantization:
- Quantized Values (INT8):

Bitplane	Global(ZSTD)	Global(LZ4)	Temporal(ZSTD)	Temporal(LZ4)	Spatial(ZSTD)	Spatial(LZ4)
7	1.000	1.000	1.020	1.045	1.013	1.030
6	1.000	1.000	1.020	1.045	1.013	1.030
5	1.000	1.000	1.020	1.045	1.013	1.030
4	1.000	1.000	1.020	1.045	1.013	1.030
3	1.000	1.000	1.019	1.044	1.013	1.030
2	1.000	1.000	1.018	1.044	1.013	1.030
1	0.854	0.987	0.969	1.036	0.927	1.030
0	0.675	0.824	0.807	0.906	0.790	1.021

- Scalar Values (FP32):

Bitplane	Scale Vertical (ZSTD)	Scale Vertical (LZ4)
31	0.034	0.039
30	0.034	0.039
29	0.034	0.039
28	0.034	0.039
27	0.034	0.039
26	0.034	0.039
25	0.034	0.039
24	0.034	0.039
23	0.034	0.039
22	0.034	0.039
21	0.034	0.039
20	0.034	0.039
19	0.034	0.039
18	0.034	0.039
17	0.034	0.039
16	0.034	0.039
15	0.034	0.039
14	0.034	0.039
13	0.034	0.039
12	0.034	0.039
11	0.034	0.039
10	0.034	0.039
9	0.034	0.039
8	0.034	0.039
7	0.034	0.039
6	0.034	0.039
5	0.034	0.039
4	0.006	0.011
3	0.006	0.011
2	0.006	0.011
1	0.006	0.011
0	0.006	0.011

- Now, lets compare it with 512 vector embeddings from last time:

Metric:	512 Rows:	4096 Rows:

Horizontal Quantization INT8 Quantized Values (Vertical Bitplaning)		
Horizontal Quantization FP32 Scalar Values (Vertical Bitplaning)		
Vertical Quantization INT8 Quantized Values (Vertical Bitplaning)		
Vertical Quantization FP32 Scalar Values (Vertical Bitplaning)		

Bitplane	Global(ZSTD)	Global(LZ4)	Temporal(ZSTD)	Temporal(LZ4)	Spatial(ZSTD)	Spatial(LZ4)
7	1.000	1.000	1.139	1.358	1.013	1.030
6	1.000	1.000	1.139	1.358	1.013	1.030
5	1.000	1.000	1.139	1.358	1.013	1.030
4	0.959	1.000	1.139	1.358	1.003	1.030
3	0.749	0.909	1.117	1.335	0.829	1.027
2	0.687	0.833	1.046	1.273	0.793	1.020
1	0.686	0.833	1.046	1.273	0.793	1.020
0	0.686	0.833	1.046	1.273	0.792	1.020

Bitplane	Global(ZSTD)	Global(LZ4)	Temporal(ZSTD)	Temporal(LZ4)	Spatial(ZSTD)	Spatial(LZ4)
7	1.000	1.000	1.018	1.044	1.013	1.030
6	1.000	1.000	1.018	1.044	1.013	1.030
5	1.000	1.000	1.018	1.044	1.013	1.030
4	0.956	1.000	1.018	1.044	1.006	1.030
3	0.737	0.904	0.873	0.983	0.825	1.027
2	0.673	0.822	0.805	0.905	0.789	1.021
1	0.672	0.822	0.805	0.904	0.789	1.021
0	0.672	0.822	0.805	0.904	0.789	1.021

Bitplane	Scale Vertical (ZSTD)	Scale Vertical (LZ4)
31	0.036	0.042
30	0.036	0.042
29	0.036	0.042
28	0.036	0.042
27	0.036	0.042
26	0.036	0.042
25	0.036	0.042
24	0.036	0.042
23	0.036	0.042
22	0.036	0.042
21	0.036	0.042
20	0.036	0.042
19	0.036	0.042
18	0.036	0.042
17	0.036	0.042
16	0.036	0.042
15	0.036	0.042
14	0.036	0.042
13	0.036	0.042
12	0.036	0.042
11	0.031	0.041
10	0.017	0.026
9	0.017	0.026
8	0.008	0.017
7	0.008	0.017
6	0.008	0.017
5	0.008	0.017
4	0.008	0.017
3	0.008	0.017
2	0.008	0.017
1	0.008	0.017
0	0.008	0.017

Bitplane	Scale Vertical (ZSTD)	Scale Vertical (LZ4)
31	0.032	0.033
30	0.032	0.033
29	0.032	0.033
28	0.032	0.033
27	0.032	0.033
26	0.032	0.033
25	0.032	0.033
24	0.032	0.033
23	0.032	0.033
22	0.032	0.033
21	0.032	0.033
20	0.032	0.033
19	0.032	0.033
18	0.032	0.033
17	0.032	0.033
16	0.032	0.033
15	0.032	0.033
14	0.032	0.033
13	0.032	0.033
12	0.032	0.033
11	0.020	0.027
10	0.009	0.013
9	0.009	0.013
8	0.001	0.002
7	0.001	0.002
6	0.001	0.002
5	0.001	0.002
4	0.001	0.002
3	0.001	0.002
2	0.001	0.002
1	0.001	0.002
0	0.001	0.002

Bitplane	Global(ZSTD)	Global(LZ4)	Temporal(ZSTD)	Temporal(LZ4)	Spatial(ZSTD)	Spatial(LZ4)
7	1.000	1.000	1.141	1.359	1.013	1.03
6	1.000	1.000	1.141	1.359	1.013	1.03
5	1.000	1.000	1.141	1.359	1.013	1.03
4	1.000	1.000	1.141	1.359	1.013	1.03
3	1.000	1.000	1.140	1.359	1.013	1.03
2	1.000	1.000	1.139	1.358	1.013	1.03
1	0.919	0.999	1.139	1.358	0.981	1.03
0	0.688	0.837	1.048	1.275	0.791	1.02

Bitplane	Global(ZSTD)	Global(LZ4)	Temporal(ZSTD)	Temporal(LZ4)	Spatial(ZSTD)	Spatial(LZ4)
7	1.000	1.000	1.020	1.045	1.013	1.030
6	1.000	1.000	1.020	1.045	1.013	1.030
5	1.000	1.000	1.020	1.045	1.013	1.030
4	1.000	1.000	1.020	1.045	1.013	1.030
3	1.000	1.000	1.019	1.044	1.013	1.030
2	1.000	1.000	1.018	1.044	1.013	1.030
1	0.854	0.987	0.969	1.036	0.927	1.030
0	0.675	0.824	0.807	0.906	0.790	1.021

Bitplane	Scale Vertical (ZSTD)	Scale Vertical (LZ4)
31	0.034	0.039
30	0.034	0.039
29	0.034	0.039
28	0.034	0.039
27	0.034	0.039
26	0.034	0.039
25	0.034	0.039
24	0.034	0.039
23	0.034	0.039
22	0.034	0.039
21	0.034	0.039
20	0.034	0.039
19	0.034	0.039
18	0.034	0.039
17	0.034	0.039
16	0.034	0.039
15	0.034	0.039
14	0.034	0.039
13	0.034	0.039
12	0.034	0.039
11	0.034	0.039
10	0.034	0.039
9	0.034	0.039
8	0.034	0.039
7	0.034	0.039
6	0.034	0.039
5	0.034	0.039
4	0.006	0.011
3	0.006	0.011
2	0.006	0.011
1	0.006	0.011
0	0.006	0.011

Bitplane	Scale Vertical (ZSTD)	Scale Vertical (LZ4)
31	0.034	0.039
30	0.034	0.039
29	0.034	0.039
28	0.034	0.039
27	0.034	0.039
26	0.034	0.039
25	0.034	0.039
24	0.034	0.039
23	0.034	0.039
22	0.034	0.039
21	0.034	0.039
20	0.034	0.039
19	0.034	0.039
18	0.034	0.039
17	0.034	0.039
16	0.034	0.039
15	0.034	0.039
14	0.034	0.039
13	0.034	0.039
12	0.034	0.039
11	0.034	0.039
10	0.034	0.039
9	0.034	0.039
8	0.034	0.039
7	0.034	0.039
6	0.034	0.039
5	0.034	0.039
4	0.006	0.011
3	0.006	0.011
2	0.006	0.011
1	0.006	0.011
0	0.006	0.011

- For context, the headers for the results relate to the granularity of data being compressed for LZ4 and ZSTD:
    - Global: Takes a “slice” of the entire tensor, which is 1 bitplane across all dimensions and vector embeddings. Calculates the LZ4 / ZSTD compression per slice (granularity: 768 x #rows)

    - Temporal: Takes a “needle” out of a tensor. This is a single bitplane of a single vector embedding dimension across all vector embedding rows. (granularity: 1 x #rows)
    - Spatial: For a single vector embedding, for each bitplane, it calculates LZ4/ZSTD compression of all vector dimensions. (granularity: 1 x 768)
- Some observations:
    - Horizontal Quantization INT8 values: the 4096 row results show slightly better global results, but notably better temporal results and better spatial results with ZSTD
    - Horizontal Quantization FP32 scalars: the 4096 row results show modest improvements in ZSTD, but considerably better results in LZ4
    - Vertical Quantization INT8 values: The 4096 row results show slightly better global results, and notably better temporal results
    - Vertical Quantization FP32 scalars: there is no improvement in compressibility
- Conclusion: Both vertical and horizontal quantization yield better compressibility in higher # of vector embeddings, but we don’t know yet if there are diminishing returns, hence more data is needed.
    - In certain bitplane isolation methods like Temporal, the # of vector embeddings is equal to the number of bits that are being compressed by LZ4 and ZSTD at a time, hence, 4096 rows of vector embeddings is treated as 4096 bits to compress. Hence, (4096 * 8) rows could almost be treated like a 4KB block for LZ4 and ZSTD compression using the temporal method.
- Still, LZ4 and ZSTD compression seems less effective than RLE compression, but I would need to further test the bitplane arrangements in my compression analysis code to make sure
- To do next time:
    - Verify fully that the compression is happening at the correct granularity
    - Compare at multiple row sizes
    - Check for storage overhead
    - Add Block Size support for LZ4 and ZSTD

2/16: Large update: More database support, larger spread of results (row, column, vertical, horizontal, symmetric scalar quantization, and no quantization), and higher configurability / Wikipedia DPR:

- Summary: It seems that preliminary results on RLE compression is different than previous results with the same file and location. This either means that the older simulation is inaccurate or that our current new simulation is inaccurate. It requires further testing to verify this. On average, it is at least 10-15% worse in compression compared to earlier simulation results.
- Data info:
    - Dataset: Wikipedia DPR (makes comparisons more consistent with previous results)

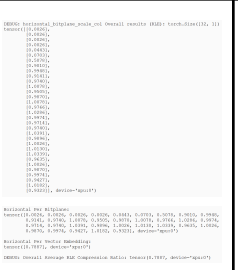
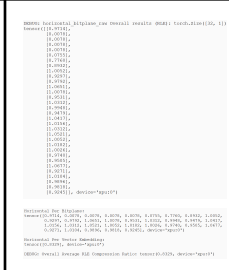
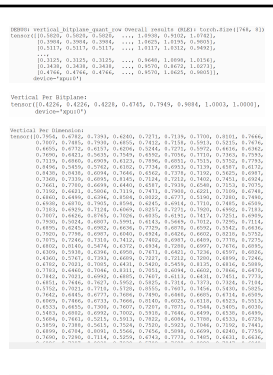
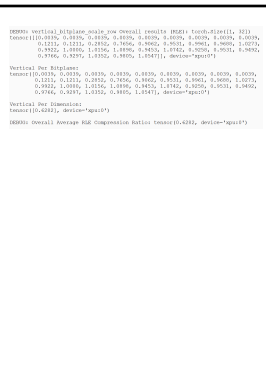
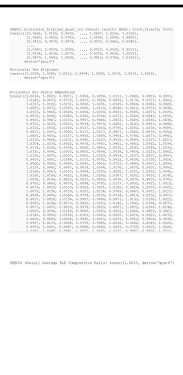
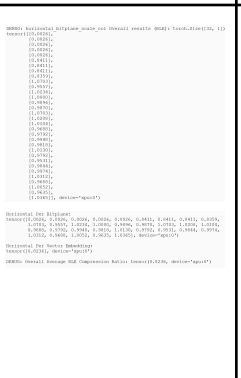
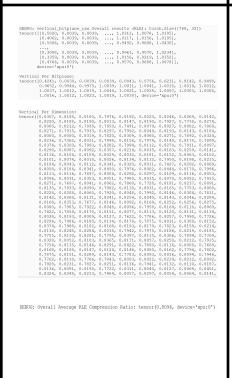
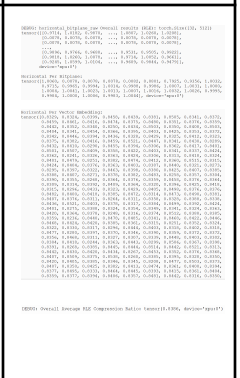
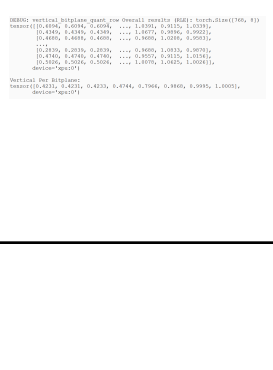
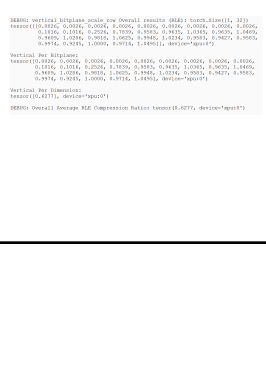
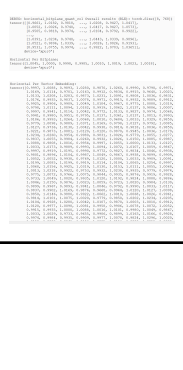
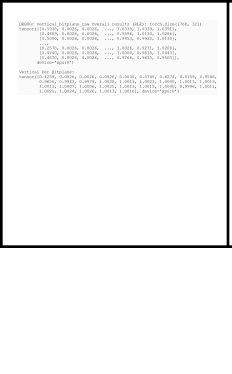
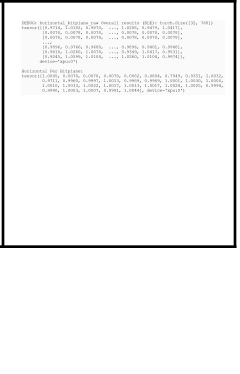
- File used: train-00000-of-00157.parquet
- Starting row: 0
- Number of rows: 512

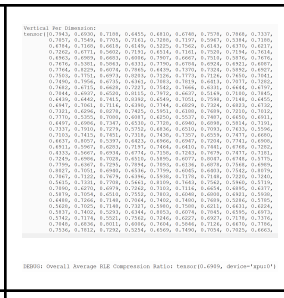
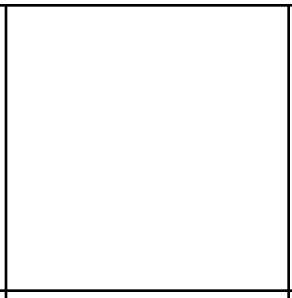
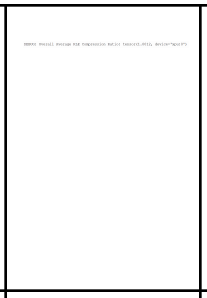
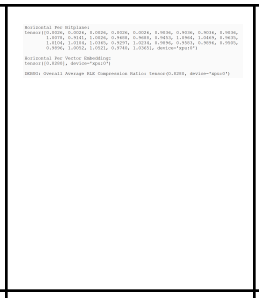
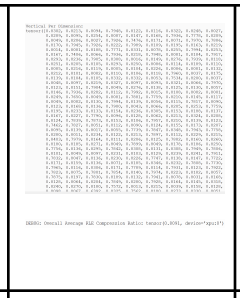
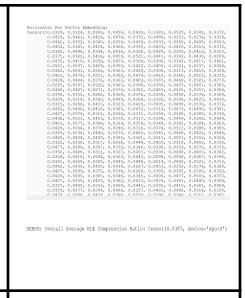
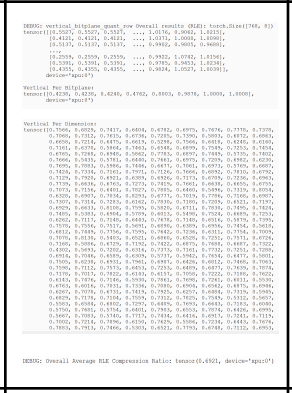
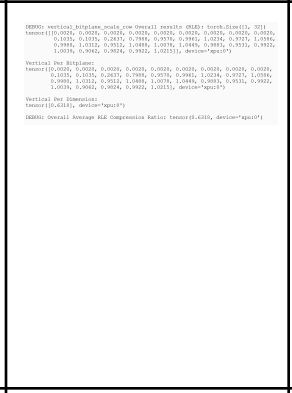
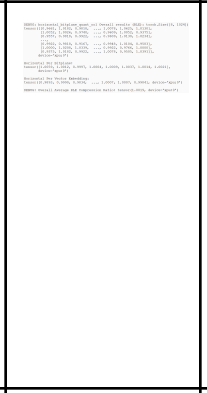
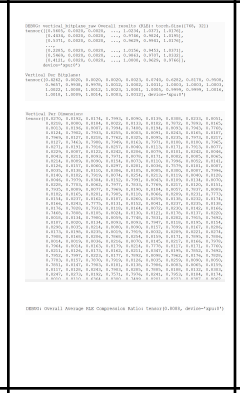
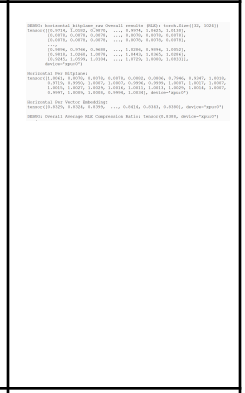
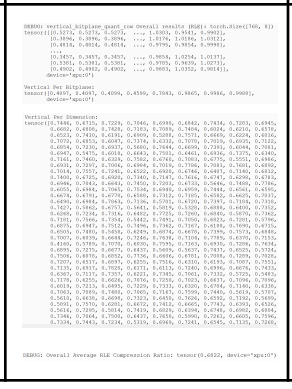
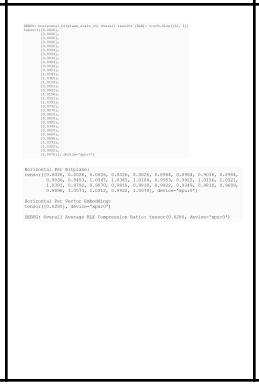
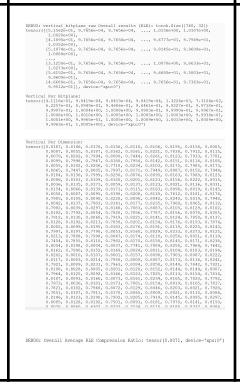
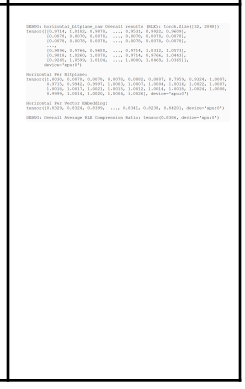
- Here is a comparison:

Metric:	Old data: (2/)	New data: (2/16)
(2/7) row-wise Quantized vertical bit plane compression + average	[horizontal quant, quant] MULTI VECTOR VERTICAL BIT PLANE COMPRESSION] tensor([[[0.3880, 0.3880, 0.3880, ..., 0.7292, 0.6068, 0.7161], [0.2656, 0.2656, 0.2656, ..., 0.7083, 0.6797, 0.6536], [0.3411, 0.3411, 0.3411, ..., 0.6745, 0.6875, 0.6328], ..., [0.2083, 0.2083, 0.2083, ..., 0.6432, 0.7266, 0.6771], [0.2292, 0.2292, 0.2292, ..., 0.6380, 0.5781, 0.6849], [0.3177, 0.3177, 0.3177, ..., 0.6380, 0.7083, 0.6536]], device='xpu:0') Avg: 0.4605	DEBUG: vertical_bitplane_quant_row Overall results (RLE): torch.Size([768, 8 tensor([[[0.5820, 0.5820, 0.5820, ..., 1.0938, 0.9102, 1.0742], [0.3984, 0.3984, 0.3984, ..., 1.0625, 1.0195, 0.9805], [0.5117, 0.5117, 0.5117, ..., 1.0117, 1.0312, 0.9492], ..., [0.3125, 0.3125, 0.3125, ..., 0.9648, 1.0898, 1.0156], [0.3438, 0.3438, 0.3438, ..., 0.9570, 0.8672, 1.0273], [0.4766, 0.4766, 0.4766, ..., 0.9570, 1.0625, 0.9805]], device='xpu:0') Avg: 0.6282

- I wanted to make a table of new results for each method:

This is an extensive table, you would need to zoom in to see the details of these

#rows	vertical_quant_row	vertical_scale_row	horizontal_quant_col	horizontal_scale_col	Vertical_raw (not quantized)	Horizontal_raw (not quantized)
1	(all 2s)	(all 2s)			(all 2s)	
512						
768						

						
1024						
2048						

- Note: certain results are omitted because I consider them extraneous. They are extraneous because their scale compression ratios are all 2. This means that they double. The reason for this for RLE is because each bitplane is 1 bit long, meaning it will automatically have double negative compression due to the 1 bit overhead of RLE.
    - This impacts:
        - Horizontal_row (quant and scale)
        - Vertical_col (quant and scale)

Now I am going to make some direct comparison tables

Overall compression:

#rows	vertical_quant_row	vertical_scale_row	horizontal_quant_col	horizontal_scale_col	Vertical_raw (not quantized)	Horizontal_raw (not quantized)

1	2	2	0.8503	0.7887	2	0.8329
512	0.6908	0.6282	1.0015	0.8236	0.8094	0.8386
768	0.6909	0.6277	1.0012	0.8280	0.8091	0.8387
1024	0.6921	0.6318	1.0019	0.8297	0.8088	0.8388
2048	0.6822	0.6304	1.0016	0.8280	0.8071	0.8386

- Some observations:
  - The compression ratios stay fairely consistent, regardless of the number of vector embeddings (better scaling with higher # of vector embeddings)
  - A single vector is not realistically compressible in certain orientations (vertical orientations, due to 1d nature in that dimension)
  - Generally vertical_row compression for RLE yields the best results, with the raw compression being in 2nd place. Horizontal_col is the worst option, not counting the extraneous options that yield negative compression

Table comparing it to previous simulation (from day 2/7):

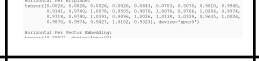
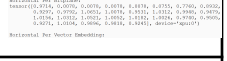
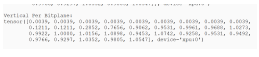
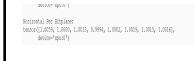
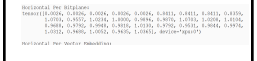
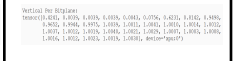
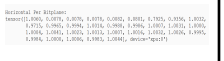
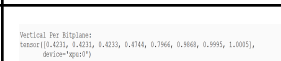
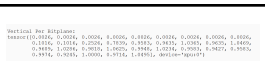
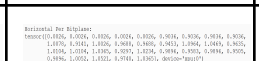
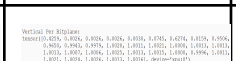
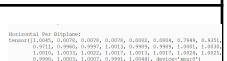
# of vector embeddings:	Avg Compression ratio (FP32, no quantization, from 1/31): old / new	Avg Compression ratio (Horizontal Symmetric Quantization, INT8): old / new	Avg Compression ratio (Horizontal Symmetric Quantization, FP32 scalars): old / new	Avg Compression ratio (Vertical Symmetric Quantization, INT8): old / new	Avg Compression ratio (Vertical Symmetric Quantization, FP32 scalars): old / new
1 (horizontal bitplane compression)	(not implemented yet)[manual math] 0.832921875 / (V: 2, H: 0.8329)	0.9775 / 2	N/A / 2	N/A / 1.0015	N/A / 0.8236
512 (vertical bitplane compression)	0.5406 / (V: 0.8094, H:	0.4605 / 0.6908	0.4188 / 0.6282	0.5924 / 1.0015	0.8236 / 0.8236

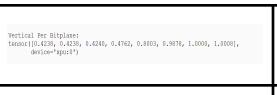
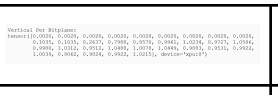
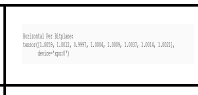
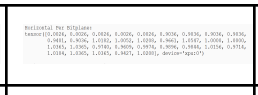
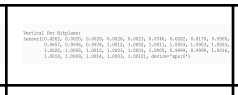
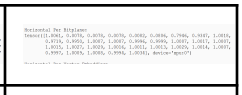
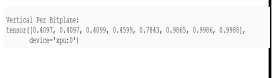
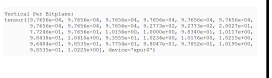
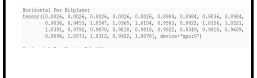
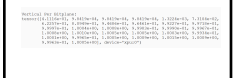
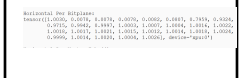
	0.8386)				
768	0.8101 / (V: 0.8091, H: 0.8387)	0.6909 / 0.6909	0.6277 / 0.6277	0.8849 / 1.0012	0.8280 / 0.8280
1024	1.0794 (negative compression) / (V: 0.8088, H: 0.8388)	0.9228 / 0.6921	0.8424 / 0.6318	1.1766 / 1.0019	0.8297 / 0.8297
2048	2.1534 (doubled negative compression) / (V: 0.8071, H: 0.8386)	1.8191 / 0.6822	1.6810 / 0.6304	2.3279 / 1.0016	0.8280 / 0.8280

- Some observations:
  - Compared with the older data, the ratios are higher at lower # vector embeddings, and lower at higher # of vector embeddings.
  - At 768 vector embeddings, 3 out of the 4 categories had the same compression ratio (might be interesting)
  - At 1024 and 2048 vector embeddings, only the vertical symmetric scalar had the same compression ratio

Compression per bitplane avg:

(you will still have to zoom in for this one)

#rows	vertical_quant_row	vertical_scale_row	horizontal_quant_col	horizontal_scale_col	Vertical_raw (not quantized)	Horizontal_raw (not quantized)
1	(all 2s)	(all 2s)			(all 2s)	
512						
768						

1024						
2048						

- Some observations:
    - Typically only the first 8-9 most significant bits have high compressibility, while the rest of the bits have almost no compressibility, or negative compression
    - Vertical_scale_row: ~12 most significant bits have high compressibility
    - Vertical_quant_row: generally a gradient from msb to lsb of compressibility (highest to lowest compressibility)
    - Horizontal_scale_col: top 5 msb are very high in compressibility, next 3-4 msb afterwards are reasonably compressible
    - Horizontal_quant_col: generally terrible compressibility
    - Vertical_raw: ~8 msb have very high compressibility, and become even more compressible as # of vector embeddings increases
    - Horizontal_raw: the 2nd msb to the 6th msb have the highest compressibility, and those ratios stay consistent regardless of the # of vector embeddings

Quantization + Scalar effective compression ratio:

#Rows	Vertical_row	horizontal_col
1	2.0	0.8010416666666667
512	0.6904379098526554	1.0000813802083333
768	0.6905734732566926	1.0003459713001943
1024	0.691759494919851	1.001240019049935
2048	0.6818955179323186	1.0012984347080085

- Observations:
    - Generally, vertical row compression is the better method compared to horizontal column.

- Today’s conclusion:
    - Vertical_row RLE compression saves around 30-31% space on average compared to only quantization
    - Horizontal_col RLE compression is not useful

        - Current results scale better with # of vector embeddings compared to previous simulations
- Accuracy/reproducibility concerns:
    - Since it is different in results compared to the previous simulation, I need to check the correctness of both simulations to see which one is correct.
        - So I may need to do more internal testing
- To do for next time:
    - Check accuracy of current simulation and previous simulation
    - Fix LZ4 + ZSTD Global, Spatial, and Temporal Analysis code (still has issues), then test accuracy of it

2/19: Mini progress + Findings update: Tested Bitpacking in both vertical and depth of bitplanes, sees redundancy / Wikipedia DPR:

- Summary: When bitpacking the bitplanes in the N and B orientations, I notice long vertical runs of the same value. This may expose redundancy in the bitplanes.
    - For reference, the tensor shape is (N, W, B)
        - N_packing results in a tensor (N//8, W, B): Used in Spatial LZ4+ZSTD Compression Analysis
        - B_packing results in a tensor (N, W, B//8): Used in Global and Temporal LZ4+ZSTD Compression Analysis
- Some other notable observations:
    - Vertical bitplanes showed much better redundancy than the horizontal bitplanes.
    - Scales often had better redundancy than their quantized values, although quantized values often showed some redundancy
    - When there isn't enough dimensions, (like for example, a scalar tensor with (1, 768, B)), it does zero-padding to make it 8 divisible
    - For vertical quant+scale rows: the first 2-3+ rows of each "block" is the same number vertically. This suggests that the first couple of MSBs of each number per dimension of vector embedding will be the same across almost all rows.
- Data info:
    - Dataset: Wikipedia DPR
    - File Used: train-00000-of-00157.parquet
    - Starting row: 0
    - Number of Rows: 512
- Some of the notable data collected:
- Vertical_bitplane_quant_row:
    - N_packed:

```
DEBUG: packed_n tensor: torch.Size([96, 8, 512]) [for Spatial]:
tensor([[[153, 152, 185,  ...,  16, 220, 221],
         [153, 152, 185,  ...,  16, 220, 221],
         [153, 152, 185,  ...,  16, 220, 221],
         ...,
         [179,   4, 251,  ...,  77, 116, 109],
         [192,  68, 122,  ...,  52,  64, 244],
         [155, 149,  20,  ..., 110, 188, 130]],

        [[232, 243, 202,  ..., 232, 142, 190],
         [232, 243, 202,  ..., 232, 142, 190],
         [232, 243, 202,  ..., 232, 142, 190],
         ...,
         [  8, 183,  79,  ..., 133,  15, 124],
         [  2,  25,  46,  ...,   9, 107, 223],
         [233,  61,  99,  ...,  60,  81, 250]],

        [[ 33,  37, 101,  ...,  45, 124,  60],
         [ 33,  37, 101,  ...,  45, 124,  60],
         [ 33,  37, 101,  ...,  45, 124,  60],
         ...,
         [178, 227, 200,  ...,  13, 255,  30],
         [ 72,   6,  97,  ..., 207,  48, 246],
         [213,  61, 118,  ..., 151,  96,   1]],

        ...,

        [[154, 158,  22,  ..., 247, 190, 238],
         [154, 158,  22,  ..., 247, 190, 238],
         [154, 158,  22,  ..., 247, 190, 238],
         ...,
         [ 56, 220, 210,  ..., 162, 161, 130],
         [ 10, 182, 242,  ..., 213, 156,  79],
         [184,  48, 118,  ...,  33, 169, 148]],
```

-

```
 [[185, 185, 184,  ..., 185,  43,  41],
  [185, 185, 184,  ..., 185,  43,  41],
  [185, 185, 184,  ..., 185,  43,  41],
  ...,
  [105,   3, 204,  ..., 246, 131, 242],
  [218,  97, 114,  ..., 155,  98,  37],
  [141,   3, 236,  ..., 172, 202, 137]],

 [[213, 149,  21,  ..., 191,  20,  18],
  [213, 149,  21,  ..., 191,  20,  18],
  [213, 149,  21,  ..., 191,  20,  18],
  ...,
  [250, 177, 190,  ..., 129, 151, 154],
  [185, 221, 147,  ..., 121, 144,  42],
  [ 86, 125,  28,  ..., 191,   4, 138]]], device='xpu:0',
dtype=torch.uint8)
```

-

- B_packed:

```
DEBUG: packed_b tensor: torch.Size([768, 8, 64]) [for Global and Temporal]:
tensor([[[225, 239, 126,  ..., 194, 132, 163],
         [225, 239, 126,  ..., 194, 132, 163],
         [225, 239, 126,  ..., 194, 132, 163],
         ...,
         [165,  43,  99,  ...,  79, 142, 176],
         [139, 178, 129,  ..., 242, 148,  65],
         [193, 168, 118,  ...,  18, 212, 227]],

        [[  0,   0,   0,  ...,  65,   4, 139],
         [  0,   0,   0,  ...,  65,   4, 139],
         [  0,   0,   0,  ...,  65,   4, 139],
         ...,
         [ 44,  71,  91,  ...,   3,  21, 199],
         [245, 175, 125,  ..., 211,   4, 187],
         [  3, 112,  53,  ...,  76, 133, 164]],

        [[ 43, 252,   0,  ...,   0,   0,   0],
         [ 43, 252,   0,  ...,   0,   0,   0],
         [ 43, 252,   0,  ...,   0,   0,   0],
         ...,
         [169, 221, 154,  ..., 119, 220, 219],
         [ 42,  57, 201,  ..., 125,  11, 205],
         [  9,  92, 107,  ..., 223, 211, 134]],

        ...,

        [[254, 254, 255,  ..., 242,  33,  14],
         [254, 254, 255,  ..., 242,  33,  14],
         [254, 254, 255,  ..., 242,  33,  14],
         ...,
         [ 58, 232, 222,  ...,  98, 161,  10],
         [ 84,  44, 167,  ..., 219,  57, 144],
         [224, 146,  65,  ...,  52, 137, 126]],

        [[  0,   0,   0,  ...,  95, 255, 253],
```

-

```
[[  0,   0,   0,  ...,  95, 255, 253],
 [  0,   0,   0,  ...,  95, 255, 253],
 [  0,   0,   0,  ...,  95, 255, 253],
 ...,
 [191, 222, 170,  ...,  98, 189,  83],
 [ 51, 112, 159,  ...,   4,   3, 193],
 [149, 192, 216,  ..., 215,  17, 101]],

[[255, 255, 255,  ..., 224,   0,   4],
 [255, 255, 255,  ..., 224,   0,   4],
 [255, 255, 255,  ..., 224,   0,   4],
 ...,
 [ 88, 139, 196,  ...,  44, 131,  78],
 [231, 215, 134,  ...,  37, 159, 188],
 [ 88,  23,  64,  ...,   8,  32, 228]]], device='xpu:0',
type=torch.uint8)
```

-

- Vertical_bitplane_scale_row:
  - N_packed:

```
DEBUG: packed_n tensor: torch.Size([1, 32, 512]) [for Spatial]:
tensor([[[  0,   0,   0,  ...,   0,   0,   0],
         [  0,   0,   0,  ...,   0,   0,   0],
         [128, 128, 128,  ..., 128, 128, 128],
         ...,
         [128, 128,   0,  ..., 128,   0, 128],
         [128, 128, 128,  ..., 128,   0, 128],
         [  0,   0, 128,  ...,   0,   0, 128]]], device='xpu:0',
       dtype=torch.uint8)
```

-

  - B_packed:

```
DEBUG: packed_b tensor: torch.Size([1, 32, 64]) [for Global and Temporal]:
tensor([[[  0,   0,   0,  ...,   0,   0,   0],
         [  0,   0,   0,  ...,   0,   0,   0],
         [255, 255, 255,  ..., 255, 255, 255],
         ...,
         [222,  73, 177,  ..., 211, 225,  21],
         [245, 165,  98,  ..., 183, 165, 117],
         [ 39,  48, 222,  ...,  55,  33, 169]]], device='xpu:0',
       dtype=torch.uint8)
```

-

- Horizontal_bitplane_quant_col:
  - N_packed:
    -
```
DEBUG: packed_n tensor: torch.Size([1, 512, 768]) [for Spatial]:
tensor([[[247,  74,  30,  ..., 213,  85, 172],
         [214,  77, 253,  ..., 224,  56, 204],
         [229,  46, 244,  ..., 234,  40, 169],
         ...,
         [255,  38,  28,  ..., 214, 212, 253],
         [217, 242,  94,  ..., 240,  25,  19],
         [226, 242, 110,  ..., 255, 249,   1]]], device='xpu:0',
       dtype=torch.uint8)
```
  - B_packed:
    -
```
DEBUG: packed_b tensor: torch.Size([8, 512, 96]) [for Global and Temporal]:
tensor([[[153, 248,  33,  ..., 155, 185, 213],
         [184, 243,  37,  ..., 158, 185, 149],
         [185, 218, 101,  ...,  22, 185,  85],
         ...,
         [144, 234,  45,  ..., 247, 185, 191],
         [221, 142, 124,  ..., 190,  47,  20],
         [221, 190,  60,  ..., 238,  41,  86]],

        [[201, 248,  65,  ..., 153,  29, 214],
         [244, 179,  37,  ..., 136,  29, 149],
         [180, 218, 101,  ...,  22,  93,  84],
         ...,
         [144, 234, 125,  ..., 213, 249, 255],
         [237,  13, 120,  ..., 191,  54,  28],
         [245,  63,  58,  ..., 239,  49, 214]],

        [[151,  28,  48,  ...,  75, 233, 233],
         [ 46, 181,  33,  ..., 221,   3, 166],
         [251,  95,  74,  ..., 193, 137, 255],
         ...,
         [204, 135, 143,  ..., 178, 242, 129],
         [ 85, 143, 222,  ..., 144, 150, 140],
         [229, 244, 156,  ..., 130, 226,  94]],

        ...,

        [[182,  90,  62,  ..., 177,  23, 111],
         [238, 254, 131,  ...,  18, 198,  49],
         [249, 255, 132,  ..., 176, 165, 232],
         ...,
         [239,  63,  93,  ..., 103,  92, 223],
         [ 37, 223, 144,  ...,  25,  21, 144],
         [ 62, 243, 112,  ...,  28, 123, 236]],

        [[240, 228,   2,  ..., 113,  80, 176],
```
    -
```
[[240, 228,   2,  ..., 113,  80, 176],
 [154, 176,  74,  ...,  34,  92,  88],
 [ 64, 139, 192,  ..., 213,  60, 204],
 ...,
 [204,  25, 173,  ..., 229,  58, 124],
 [117,  30,  18,  ...,  77, 101, 185],
 [234, 231,  28,  ..., 125,  16,   4]],

[[153, 126, 143,  ..., 124, 250,  30],
 [116,  31,  70,  ..., 139, 213, 216],
 [134, 151,  85,  ..., 208, 152, 137],
 ...,
 [132, 143,  18,  ...,  72,  88,  57],
 [138,  50,  17,  ...,   9, 238, 227],
 [  6, 167, 140,  ...,  98,  44, 231]]], device='xpu:0',
dtype=torch.uint8)
```
- Horizontal_bitplane_scale_col:

- N_packed:

```
DEBUG: packed_n tensor: torch.Size([4, 1, 768]) [for Spatial]:
tensor([[[ 60,  59,  59,  ...,  60,  59,  60]],

        [[  4, 214, 201,  ...,   0, 244,   8]],

        [[183,  37,   8,  ..., 187,  12,  78]],

        [[ 40, 220, 145,  ..., 252, 172, 247]]], device='xpu:0',
       dtype=torch.uint8)
```

- B_packed:

```
DEBUG: packed_b tensor: torch.Size([32, 1, 96]) [for Global and Temporal]:
tensor([[[  0,   0,   0,  ...,   0,   0,   0]],

        [[  0,   0,   0,  ...,   0,   0,   0]],

        [[255, 255, 255,  ..., 255, 255, 255]],

        ...,

        [[ 92,  87, 137,  ..., 161,  80,  63]],

        [[ 27, 238, 181,  ..., 227, 193,  97]],

        [[ 62,  81, 122,  ...,  74,  58, 169]]], device='xpu:0',
       dtype=torch.uint8)
```

- Vertical_bitplane_raw:
  - N_packed:

```
DEBUG: packed_n tensor: torch.Size([96, 32, 512]) [for Spatial]:
tensor([[[153, 184, 185,  ..., 144, 221, 221],
         [  0,   0,   0,  ...,   0,   0,   0],
         [255, 255, 255,  ..., 255, 255, 255],
         ...,
         [105, 126,  14,  ..., 192, 160, 231],
         [106,  51, 205,  ..., 240, 191, 128],
         [ 33, 138, 115,  ..., 129, 152,  39]],

        [[248, 243, 218,  ..., 234, 142, 190],
         [  0,   0,   0,  ...,   0,   0,   0],
         [255, 255, 255,  ..., 255, 255, 255],
         ...,
         [143, 232,  89,  ...,  45,  91,  58],
         [ 92,  34, 240,  ...,  83,  52,  46],
         [157, 154,  84,  ..., 120, 220, 250]],

        [[ 33,  37, 101,  ...,  45, 124,  60],
         [  0,   0,   0,  ...,   0,   0,   0],
         [255, 255, 255,  ..., 255, 255, 255],
         ...,
         [ 86, 125, 205,  ..., 160, 124, 229],
         [ 46, 216,  13,  ..., 139, 247, 232],
         [211,  76,  16,  ..., 157,  55, 198]],

        ...,

        [[155, 158,  22,  ..., 247, 190, 238],
         [  0,   0,   0,  ...,   0,   0,   0],
         [255, 255, 255,  ..., 255, 255, 255],
         ...,
         [ 56, 232,  27,  ...,  62, 231, 148],
         [ 46, 186, 144,  ...,  98, 194, 124],
         [ 96, 216,  66,  ..., 237,  51,   5]],

        [[185, 185, 185,  ..., 185,  47,  41],
```

```
[[185, 185, 185,  ..., 185,  47,  41],
 [  0,   0,   0,  ...,   0,   0,   0],
 [255, 255, 255,  ..., 255, 255, 255],
 ...,
 [242, 196, 121,  ...,  14, 152,  19],
 [ 76,  39, 252,  ..., 228, 140,  11],
 [227, 167, 252,  ..., 184, 246,  36]],

[[213, 149,  85,  ..., 191,  20,  86],
 [  0,   0,   0,  ...,   0,   0,   0],
 [255, 255, 255,  ..., 255, 255, 255],
 ...,
 [245, 134,  85,  ..., 206, 253,  71],
 [218,  95,  45,  ..., 159, 192, 137],
 [ 50, 223, 121,  ..., 167, 214, 207]]], device='xpu:0',
dtype=torch.uint8)
```

- B_packed:

```
DEBUG: packed_b tensor: torch.Size([768, 32, 64]) [for Global and Temporal]:
tensor([[[241, 239, 126,  ..., 194, 164, 167],
         [  0,   0,   0,  ...,   0,   0,   0],
         [255, 255, 255,  ..., 255, 255, 255],
         ...,
         [ 12, 107,  85,  ..., 204,  43,  47],
         [ 42,  49,  66,  ...,  54, 249, 119],
         [ 90, 188,  30,  ...,  44, 134, 254]],

        [[  0,   0,   0,  ...,  65,   6, 139],
         [  0,   0,   0,  ...,   0,   0,   0],
         [255, 255, 255,  ..., 255, 255, 255],
         ...,
         [210, 196, 226,  ..., 239, 195, 133],
         [182, 181, 117,  ...,  49, 216, 180],
         [ 51,  99, 132,  ..., 239, 152,  64]],

        [[123, 254,   0,  ...,   0,   0,   0],
         [  0,   0,   0,  ...,   0,   0,   0],
         [255, 255, 255,  ..., 255, 255, 255],
         ...,
         [199,  22,   1,  ..., 132,  44,  43],
         [220, 250, 140,  ...,  10,  26, 158],
         [177, 101,  51,  ..., 250, 158,  33]],

        ...,

        [[255, 255, 255,  ..., 242,  35,  15],
         [  0,   0,   0,  ...,   0,   0,   0],
         [255, 255, 255,  ..., 255, 255, 255],
         ...,
         [243, 204,  31,  ..., 204, 166, 151],
         [107, 233, 248,  ..., 187,  72,  92],
         [ 75,  99, 106,  ...,   5,  42, 255]],

        [[  0,   0,   0,  ...,  95, 255, 253],
```

```
[[  0,   0,   0,  ...,  95, 255, 253],
 [  0,   0,   0,  ...,   0,   0,   0],
 [255, 255, 255,  ..., 255, 255, 255],
 ...,
 [ 74, 211, 126,  ...,  26,  76, 101],
 [204,  22, 194,  ..., 254, 210, 172],
 [211, 203, 117,  ...,  22, 253,  15]],

[[255, 255, 255,  ..., 224,   0,   4],
 [  0,   0,   0,  ...,   0,   0,   0],
 [255, 255, 255,  ..., 255, 255, 255],
 ...,
 [161, 139,  34,  ...,  50, 156,  35],
 [121,   6,  31,  ...,   2, 218, 213],
 [ 96, 203, 133,  ..., 217, 242,  69]]], device='xpu:0',
dtype=torch.uint8)
```

- Vertical Bitplane Raw: Both methods show redundancy and duplicates in the horizontal orientation

- Horizontal_bitplane_raw:
  - N_packed:

```
DEBUG: packed_n tensor: torch.Size([4, 512, 768]) [for Spatial]:
tensor([[[189,  62,  62,  ..., 190,  63, 191],
         [190,  63, 188,  ..., 190,  62, 190],
         [190,  62, 189,  ..., 190,  62, 191],
         ...,
         [188,  62,  62,  ..., 190, 190, 188],
         [190, 189,  63,  ..., 190,  62,  62],
         [190, 189,  63,  ..., 187, 189,  60]],

        [[148, 245,  62,  ..., 172,  33,  51],
         [174,   1, 174,  ..., 130, 214, 220],
         [ 96, 155, 156,  ...,  46, 154,  57],
         ...,
         [ 43, 126,  47,  ..., 169, 166, 204],
         [163, 181,  18,  ...,   4,  61,  36],
         [119, 190,  45,  ..., 202,  82,  22]],

        [[ 38, 240, 252,  ...,  98, 157, 218],
         [ 89,  98,  51,  ..., 144, 207,  24],
         [123,  21, 213,  ..., 114,  89,  92],
         ...,
         [253, 126, 154,  ..., 230,  59,  95],
         [186,  48, 253,  ...,  67,  16, 155],
         [241,  60, 130,  ...,  79, 228,   8]],

        [[112, 222, 119,  ..., 156, 107, 204],
         [ 65,  84,  46,  ...,  79, 191, 171],
         [242,  11, 113,  ..., 158, 144,  47],
         ...,
         [103, 230, 130,  ...,   7, 207, 147],
         [183, 224,  62,  ...,  21, 153, 196],
         [254, 220, 157,  ...,  21, 253,   7]]], device='xpu:0',
       dtype=torch.uint8)
```

  - B_packed:

```
DEBUG: packed_b tensor: torch.Size([32, 512, 96]) [for Global and Temporal]:
tensor([[[153, 248,  33,  ..., 155, 185, 213],
         [184, 243,  37,  ..., 158, 185, 149],
         [185, 218, 101,  ...,  22, 185,  85],
         ...,
         [144, 234,  45,  ..., 247, 185, 191],
         [221, 142, 124,  ..., 190,  47,  20],
         [221, 190,  60,  ..., 238,  41,  86]],

        [[  0,   0,   0,  ...,   0,   0,   0],
         [  0,   0,   0,  ...,   0,   0,   0],
         [  0,   0,   0,  ...,   0,   0,   0],
         ...,
         [  0,   0,   0,  ...,   0,   0,   0],
         [  0,   0,   0,  ...,   0,   0,   0],
         [  0,   0,   0,  ...,   0,   0,   0]],

        [[255, 255, 255,  ..., 255, 255, 255],
         [255, 255, 255,  ..., 255, 255, 255],
         [255, 255, 255,  ..., 255, 255, 255],
         ...,
         [255, 255, 255,  ..., 255, 255, 255],
         [255, 255, 255,  ..., 255, 255, 255],
         [255, 255, 255,  ..., 255, 255, 255]],

        ...,

        [[105, 143,  86,  ...,  56, 242, 245],
         [126, 232, 125,  ..., 232, 196, 134],
         [ 14,  89, 205,  ...,  27, 121,  85],
         ...,
         [192,  45, 160,  ...,  62,  14, 206],
         [160,  91, 124,  ..., 231, 152, 253],
         [231,  58, 229,  ..., 148,  19,  71]],
```

- Look at the blocks above with all the same numbers, there might be some highly compressible blocks here

```
[[106,  92,  46,  ...,  46,  76, 218],
 [ 51,  34, 216,  ..., 186,  39,  95],
 [205, 240,  13,  ..., 144, 252,  45],
 ...,
 [240,  83, 139,  ...,  98, 228, 159],
 [191,  52, 247,  ..., 194, 140, 192],
 [128,  46, 232,  ..., 124,  11, 137]],

[[ 33, 157, 211,  ...,  96, 227,  50],
 [138, 154,  76,  ..., 216, 167, 223],
 [115,  84,  16,  ...,  66, 252, 121],
 ...,
 [129, 120, 157,  ..., 237, 184, 167],
 [152, 220,  55,  ...,  51, 246, 214],
 [ 39, 250, 198,  ...,   5,  36, 207]]], device='xpu:0',
dtype=torch.uint8)
```

- For these raws: There is redundancy and similar numbers in the vertical orientation.

- Now for the “Miscellaneous ones”:
  - Horizontal_bitplane_quant_row:
    - N_packed:

```
DEBUG: packed_n tensor: torch.Size([1, 512, 768]) [for Spatial]:
tensor([[[255,  10,   4,  ..., 249,  13, 242],
         [249,  10,   0,  ..., 251,   8, 247],
         [252,   6, 254,  ..., 253,   6, 242],
         ...,
         [  0,   5,   3,  ..., 249, 249, 255],
         [249, 254,  13,  ..., 253,   4,   4],
         [251, 254,  14,  ...,   0, 255,   0]]], device='xpu:0',
       dtype=torch.uint8)
```

- B_packed:

```
DEBUG: packed_b tensor: torch.Size([8, 512, 96]) [for Global and Temporal]:
tensor([[[153, 232,  33,  ..., 154, 185, 213],
         [152, 243,  37,  ..., 158, 185, 149],
         [185, 202, 101,  ...,  22, 184,  21],
         ...,
         [ 16, 232,  45,  ..., 247, 185, 191],
         [220, 142, 124,  ..., 190,  43,  20],
         [221, 190,  60,  ..., 238,  41,  18]],

        [[153, 232,  33,  ..., 154, 185, 213],
         [152, 243,  37,  ..., 158, 185, 149],
         [185, 202, 101,  ...,  22, 184,  21],
         ...,
         [ 16, 232,  45,  ..., 247, 185, 191],
         [220, 142, 124,  ..., 190,  43,  20],
         [221, 190,  60,  ..., 238,  41,  18]],

        [[153, 232,  33,  ..., 154, 185, 213],
         [152, 243,  37,  ..., 158, 185, 149],
         [185, 202, 101,  ...,  22, 184,  21],
         ...,
         [ 16, 232,  45,  ..., 247, 185, 191],
         [220, 142, 124,  ..., 190,  43,  20],
         [221, 190,  60,  ..., 238,  41,  18]],

        ...,

        [[179,   8, 178,  ...,  56, 105, 250],
         [  4, 183, 227,  ..., 220,   3, 177],
         [251,  79, 200,  ..., 210, 204, 190],
         ...,
         [ 77, 133,  13,  ..., 162, 246, 129],
         [116,  15, 255,  ..., 161, 131, 151],
         [109, 124,  30,  ..., 130, 242, 154]],
```

```
[[192,   2,  72,  ...,  10, 218, 185],
 [ 68,  25,   6,  ..., 182,  97, 221],
 [122,  46,  97,  ..., 242, 114, 147],
 ...,
 [ 52,   9, 207,  ..., 213, 155, 121],
 [ 64, 107,  48,  ..., 156,  98, 144],
 [244, 223, 246,  ...,  79,  37,  42]],

[[155, 233, 213,  ..., 184, 141,  86],
 [149,  61,  61,  ...,  48,   3, 125],
 [ 20,  99, 118,  ..., 118, 236,  28],
 ...,
 [110,  60, 151,  ...,  33, 172, 191],
 [188,  81,  96,  ..., 169, 202,   4],
 [130, 250,   1,  ..., 148, 137, 138]]], device='xpu:0',
type=torch.uint8)
```

- Horizontal_bitplane_scale_row:
  - N_packed:

```
DEBUG: packed_n tensor: torch.Size([4, 512, 1]) [for Spatial]:
tensor([[[ 61],
         [ 61],
         [ 61],
         ...,
         [ 61],
         [ 61],
         [ 61]],

        [[ 76],
         [ 76],
         [ 79],
         ...,
         [ 74],
         [ 55],
         [ 66]],

        [[ 11],
         [ 37],
         [224],
         ...,
         [ 34],
         [242],
         [177]],

        [[134],
         [230],
         [139],
         ...,
         [222],
         [184],
         [223]]], device='xpu:0', dtype=torch.uint8)
```

- B_packed:

```
DEBUG: packed_b tensor: torch.Size([32, 512, 1]) [for Global and Temporal]:
tensor([[[  0],
         [  0],
         [  0],
         ...,
         [  0],
         [  0],
         [  0]],

        [[  0],
         [  0],
         [  0],
         ...,
         [  0],
         [  0],
         [  0]],

        [[128],
         [128],
         [128],
         ...,
         [128],
         [128],
         [128]],

        ...,

        [[128],
         [128],
         [  0],
         ...,
         [128],
         [  0],
         [128]],

        [[128]
```

-

```
[[128],
 [128],
 [128],
 ...,
 [128],
 [  0],
 [128]],

[[  0],
 [  0],
 [128],
 ...,
 [  0],
 [  0],
 [128]]], device='xpu:0', dtype=torch.uint8)
```

-

- Vertical_bitplane_quant_col:
  - N_packed:

```
DEBUG: packed_n tensor: torch.Size([96, 8, 512]) [for Spatial]:
tensor([[[153, 184, 185,  ..., 144, 221, 221],
         [201, 244, 180,  ..., 144, 237, 245],
         [151,  46, 251,  ..., 204,  85, 229],
         ...,
         [182, 238, 249,  ..., 239,  37,  62],
         [240, 154,  64,  ..., 204, 117, 234],
         [153, 116, 134,  ..., 132, 138,   6]],

        [[248, 243, 218,  ..., 234, 142, 190],
         [248, 179, 218,  ..., 234,  13,  63],
         [ 28, 181,  95,  ..., 135, 143, 244],
         ...,
         [ 90, 254, 255,  ...,  63, 223, 243],
         [228, 176, 139,  ...,  25,  30, 231],
         [126,  31, 151,  ..., 143,  50, 167]],

        [[ 33,  37, 101,  ...,  45, 124,  60],
         [ 65,  37, 101,  ..., 125, 120,  58],
         [ 48,  33,  74,  ..., 143, 222, 156],
         ...,
         [ 62, 131, 132,  ...,  93, 144, 112],
         [  2,  74, 192,  ..., 173,  18,  28],
         [143,  70,  85,  ...,  18,  17, 140]],

        ...,

        [[155, 158,  22,  ..., 247, 190, 238],
         [153, 136,  22,  ..., 213, 191, 239],
         [ 75, 221, 193,  ..., 178, 144, 130],
         ...,
         [177,  18, 176,  ..., 103,  25,  28],
         [113,  34, 213,  ..., 229,  77, 125],
         [124, 139, 208,  ...,  72,   9,  98]],

        [[185, 185, 185,  ..., 185,  47,  41]
```

-

```
[[185, 185, 185,  ..., 185,  47,  41],
 [ 29,  29,  93,  ..., 249,  54,  49],
 [233,   3, 137,  ..., 242, 150, 226],
 ...,
 [ 23, 198, 165,  ...,  92,  21, 123],
 [ 80,  92,  60,  ...,  58, 101,  16],
 [250, 213, 152,  ...,  88, 238,  44]],

[[213, 149,  85,  ..., 191,  20,  86],
 [214, 149,  84,  ..., 255,  28, 214],
 [233, 166, 255,  ..., 129, 140,  94],
 ...,
 [111,  49, 232,  ..., 223, 144, 236],
 [176,  88, 204,  ..., 124, 185,   4],
 [ 30, 216, 137,  ...,  57, 227, 231]]], device='xpu:0',
dtype=torch.uint8)
```

-

- B_packed:

```
DEBUG: packed_b tensor: torch.Size([768, 8, 64]) [for Global and Temporal]:
tensor([[[241, 239, 126,  ..., 194, 164, 167],
         [241, 239, 126,  ..., 194, 164, 167],
         [177, 171,  99,  ...,  66, 164, 165],
         ...,
         [250,  54,  75,  ..., 128,  98, 196],
         [218,  56, 136,  ..., 110, 230,  45],
         [191,  62, 219,  ..., 126,  18, 190]],

        [[  0,   0,   0,  ...,  65,   6, 139],
         [223, 152, 230,  ...,  65,   6, 139],
         [ 36,  71,  89,  ...,   3,  23, 135],
         ...,
         [124,  33,   8,  ..., 135, 135, 244],
         [169, 124,  60,  ...,  19, 238, 103],
         [ 78, 114,  26,  ..., 228,   0, 160]],

        [[123, 254,   0,  ...,   0,   0,   0],
         [123, 254,   0,  ...,  27, 247, 147],
         [121, 223, 158,  ..., 119, 220, 249],
         ...,
         [243,  26, 166,  ..., 227, 174, 119],
         [158,  91,  44,  ...,  65, 193, 187],
         [ 69, 148,  72,  ...,  19, 229, 168]],

        ...,

        [[255, 255, 255,  ..., 242,  35,  15],
         [255, 191, 251,  ..., 210,  35,  15],
         [123, 233, 223,  ...,  34,  35,  11],
         ...,
         [153,  61,  86,  ...,  55, 186, 173],
         [ 61, 115, 244,  ...,  93, 182,  61],
         [142,  76, 221,  ..., 164, 217,  33]],

        [[  0,   0,   0,  ...,  95, 255, 253],
```

-

```
[[  0,   0,   0,  ...,  95, 255, 253],
 [128, 130,   0,  ...,  72,   1, 229],
 [119,  81, 202,  ...,  86, 172,  17],
 ...,
 [131,  52, 227,  ..., 110, 207,  20],
 [ 31, 179,  24,  ...,  47, 111, 232],
 [142, 123, 250,  ..., 169, 242,  43]],

[[255, 255, 255,  ..., 224,   0,   4],
 [ 64, 163, 255,  ..., 224,   0,  12],
 [191,  92,   0,  ...,  23, 247,   4],
 ...,
 [222, 212, 135,  ..., 120, 139,  92],
 [ 27,  23,  59,  ..., 116,  21,  66],
 [ 36, 246, 106,  ..., 221,  17, 223]]], device='xpu:0',
ltype=torch.uint8)
```

-

- Vertical_bitplane_scale_col:
  - N_packed:

```
DEBUG: packed_n tensor: torch.Size([96, 32, 1]) [for Spatial]:
tensor([[[  0],
         [  0],
         [255],
         ...,
         [ 92],
         [ 27],
         [ 62]],

        [[  0],
         [  0],
         [255],
         ...,
         [ 87],
         [238],
         [ 81]],

        [[  0],
         [  0],
         [255],
         ...,
         [137],
         [181],
         [122]],

        ...,

        [[  0],
         [  0],
         [255],
         ...,
         [161],
         [227],
         [ 74]],
```

-

```
[[  0],
 [  0],
 [255],
 ...,
 [ 80],
 [193],
 [ 58]],

[[  0],
 [  0],
 [255],
 ...,
 [ 63],
 [ 97],
 [169]]], device='xpu:0', dtype=torch.uint8)
```

-

- B_packed:

```
DEBUG: packed_b tensor: torch.Size([768, 32, 1]) [for Global and Temporal]:
tensor([[[  0],
         [  0],
         [128],
         ...,
         [  0],
         [  0],
         [  0]],

        [[  0],
         [  0],
         [128],
         ...,
         [128],
         [  0],
         [  0]],

        [[  0],
         [  0],
         [128],
         ...,
         [  0],
         [  0],
         [128]],

        ...,

        [[  0],
         [  0],
         [128],
         ...,
         [128],
         [  0],
         [  0]],

        [[  0]
```

-

```
[[  0],
 [  0],
 [128],
 ...,
 [128],
 [  0],
 [  0]],

[[  0],
 [  0],
 [128],
 ...,
 [128],
 [128],
 [128]]], device='xpu:0', dtype=torch.uint8)
```

-

- Conclusion:
  - Vertical_bitplane_quant+scalars_row seems to have the highest redundancy
  - We will need to confirm the actual redundancy/compressibility with LZ4 and ZSTD compression
- For next time:

- Analyze the ZSTD and LZ4 compression of these N-packed and B-packed bitplane tensors in the Global, Temporal, and Spatial granularities

2/23: Progress update: Still working on fixes / Wikipedia DPR (Tim Update):

- Summary: Found many issues with current bitplane code implementation that causes memory scrambling. Introducing noise that reduces compression. I am currently working on fixes and rigorous tests. It is very hard to tell, so I won't give any results until I am certain that they are proven correct.
- Some other findings:
  - Apparently, doing symmetric scalar quantization removes a lot of the redundancy that compression relies on.
  - Scalars so far seem more compressible than the quantized values.
- To continue doing:
  - Fix the bitplane code and verify it works
  - Continue troubleshooting the LZ4+ZSTD analysis code

2/24: Debugging progress update: Working on fixes, found issues with bitplaning functions and other functions / Custom data tensors (Tim Update):

- Summary: There were inconsistencies found in the bitplane disaggregation code:
  - ~~Vertical bitplaning seems to give the same results as horizontal bitplaning, this is partially due to the wrong permutations~~
  - Raw bitplaning seems to give the wrong binary format, causing issues like the first 8 bits being wrong
- Tested working:
  - Symmetric Scalar Quantization (row-wise and col-wise)
  - Horizontal bitplaning for quantized and scalars
  - (Fixed) Vertical bitplaning now works (returns (W,B,N) tensor)
- What still needs work:
  - Raw bitplaning
  - ~~Horizontal bitplaning~~
- What isn't tested yet:
  - Bitpacking (n_packed, b_packed, w_packed)
  - Compression analysis (RLE+LZ4+ZSTD)
- Current debugging progress:
  - Stage 0: raw tensor (finished, verified)
  - Stage 1: Symmetric Scalar Quantization (finished, verified)
  - Stage 2+3: Binary String Conversion + Bitplane Disaggregation (unfinished, nearly done)
    - quant and scale seems to be done? Will check
    - haven't finished trying out new raw_bitplane function for raw unquantized tensors
  - Stage 4: RLE Analysis (not there yet)

- Stage 5: Bitpacking [n,b,w packing] (not there yet)
- Stage 6: LZ4 + ZSTD Compression Analysis [Global, Temporal, Spatial] (not there yet)

- If you are interested, I am manually testing my own tensors, doing the calculations by hand, then comparing the results with my simulation. Even with the smallest possible tensors, this is a time consuming process, but helps ensure total correctness
- Here is my initial tensor:
  -

0.50	0.330	0.1250
0.750	0.250	0.3750

  - It is an (2,3) tensor, in the form (N,W)
    - B is the bitplane length arrangement (# of bitplanes = B)
  - My simulation got the quantized forms + scalars:

```
Initial Tensor Data: torch.Size([2, 3])
tensor([[0.5000, 0.3300, 0.1250],
        [0.7500, 0.2500, 0.3750]])

ROW:
Scales dtype: torch.float32
row

Symmetric Scalar Quantization (Quantized Values):
tensor([[127,  84,  32],
        [127,  42,  64]], dtype=torch.int8)

Symmetric Scalar Quantization (Scalar Values):
tensor([[0.0039],
        [0.0059]])

COL:
Scales dtype: torch.float32
col

Symmetric Scalar Quantization (Quantized Values):
tensor([[ 85, 127,  42],
        [127,  96, 127]], dtype=torch.int8)

Symmetric Scalar Quantization (Scalar Values):
tensor([[0.0059, 0.0026, 0.0030]])
```

  -

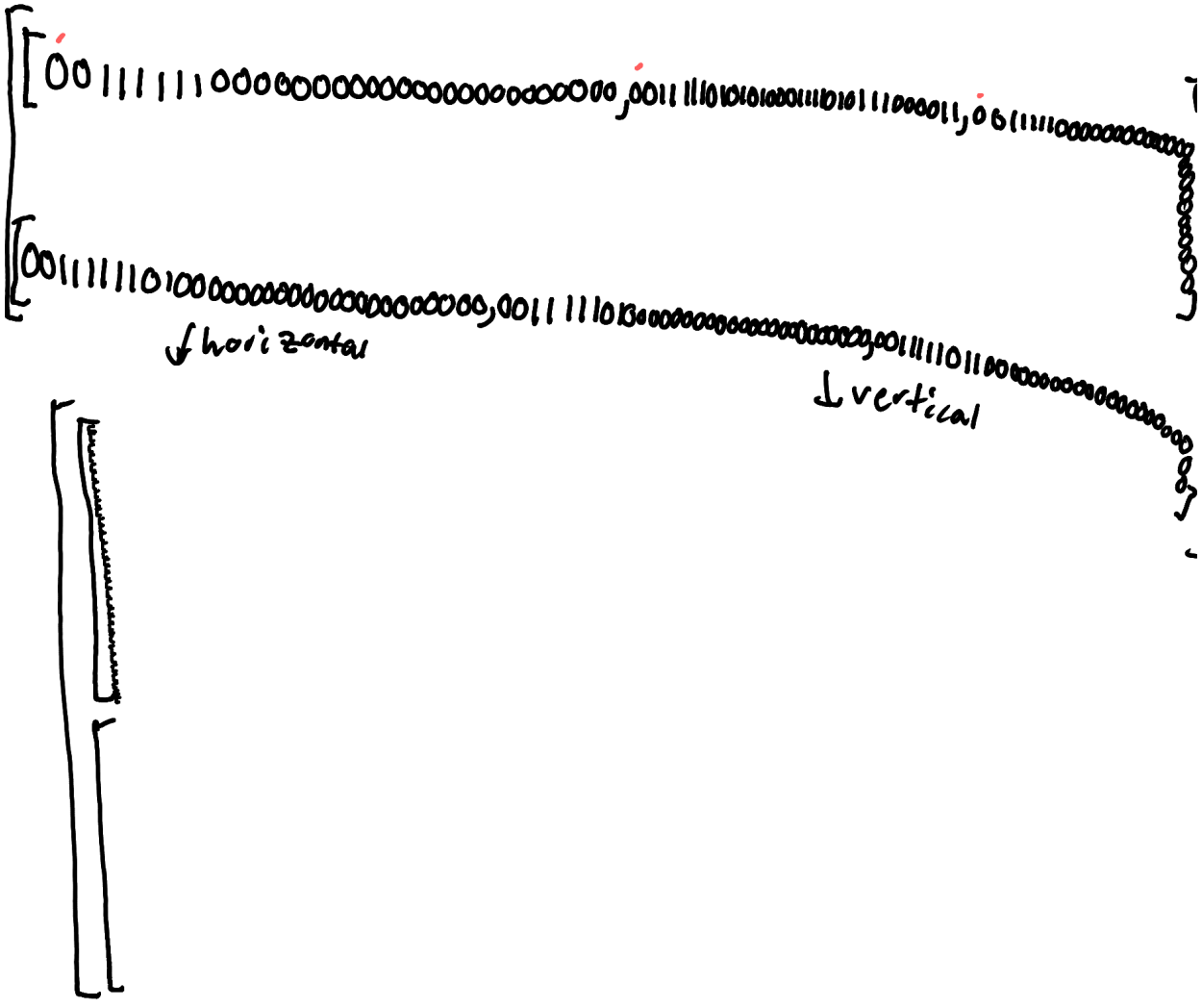
        - I checked the values when dequantizing them, it is in the correct orientation and it is very close to the original values
- Here are my notes for the bitplanes and comparing them to the simulation:
    - Note: they only show the quantized value results due to space constraints
    - Original (not done yet):

original tensor

[[0.50, 0.330, 0.1250]

[0.750, 0.250, 0.3750]]

↓ binary



```
STAGE 3: Bitplane Disaggregation (raw/direct): H: torch.Size([2, 32, 3]); V: torch.Size([3, 32, 2]):
Horizontal Raw:
tensor([[[0, 0, 0],
         [0, 0, 0],
         [1, 1, 1],
         [1, 1, 1],
         [1, 1, 1],
         [1, 1, 1],
         [1, 1, 1],
         [1, 0, 0],
         [0, 1, 0],
         [0, 0, 0],
         [0, 1, 0],
         [0, 0, 0],
         [0, 1, 0],
         [0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
         [0, 1, 0],
         [0, 1, 0],
         [0, 1, 0],
         [0, 1, 0],
         [0, 0, 0],
         [0, 1, 0],
         [0, 0, 0],
         [0, 1, 0],
         [0, 1, 0],
         [0, 1, 0],
         [0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
```

```
[[0, 0, 0],
 [0, 0, 0],
 [1, 1, 1],
 [1, 1, 1],
 [1, 1, 1],
 [1, 1, 1],
 [1, 1, 1],
 [1, 0, 0],
 [0, 1, 1],
 [1, 0, 1],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0],
 [0, 0, 0]]], dtype=torch.uint8)
```

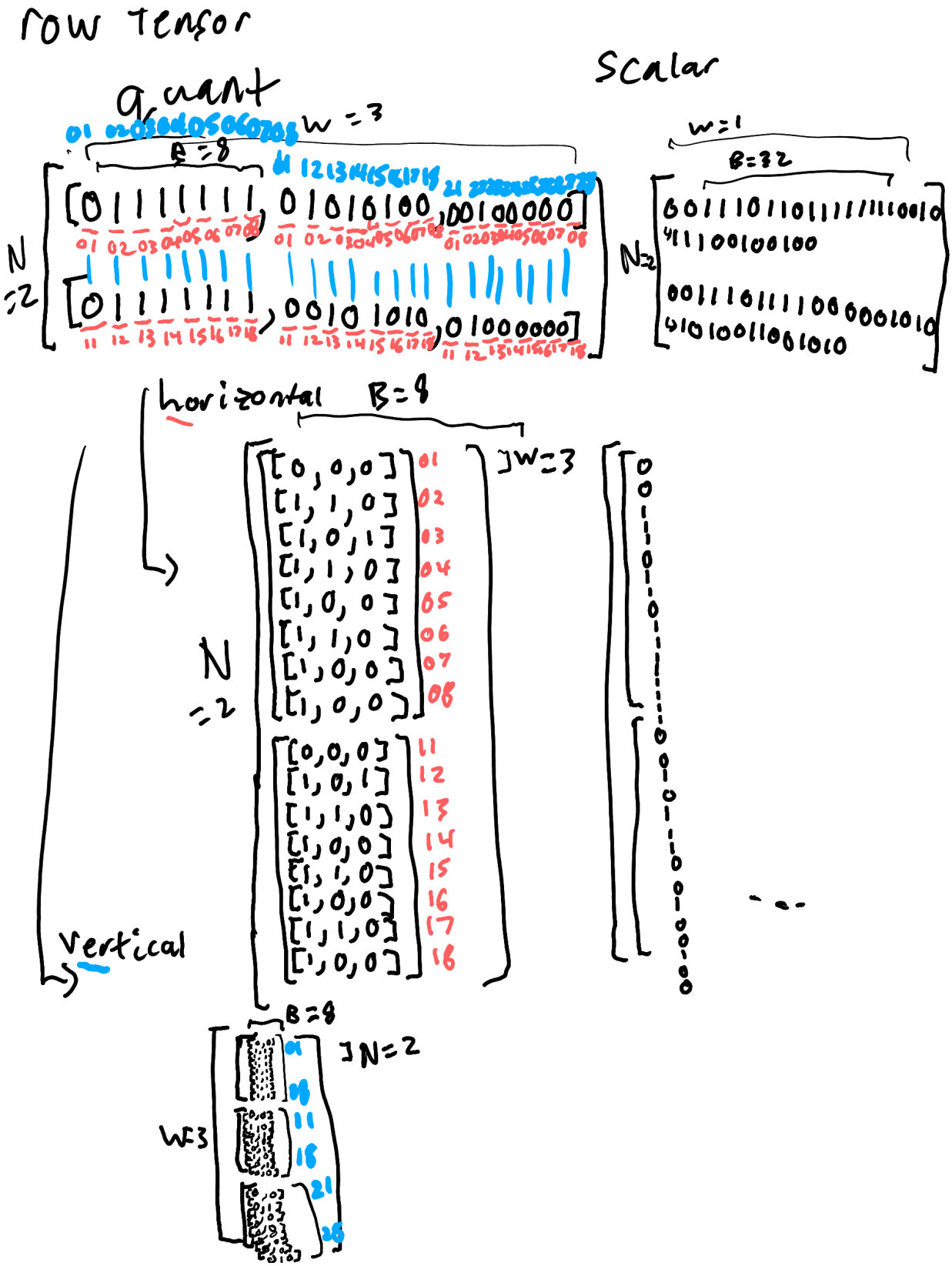
Notice how there are lots of zeros

```
Vertical Raw:
tensor([[[0, 0],
         [0, 0],
         [1, 1],
         [1, 1],
         [1, 1],
         [1, 1],
         [1, 1],
         [1, 1],
         [0, 0],
         [0, 1],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
```

```
[[0, 0],
 [0, 0],
 [1, 1],
 [1, 1],
 [1, 1],
 [1, 1],
 [1, 1],
 [0, 0],
 [1, 1],
 [0, 0],
 [1, 0],
 [0, 0],
 [1, 0],
 [0, 0],
 [0, 0],
 [0, 0],
 [1, 0],
 [1, 0],
 [1, 0],
 [1, 0],
 [0, 0],
 [1, 0],
 [0, 0],
 [1, 0],
 [1, 0],
 [1, 0],
 [0, 0],
 [0, 0],
 [0, 0],
 [0, 0],
 [1, 0],
 [1, 0]],
```

Notice how there is a little more noise here

- Row-wise Quantization + Scalar tensors:


row tensor
quant
Scalar
W=3
B=8
N=2
W=1
B=32
horizontal
vertical
...

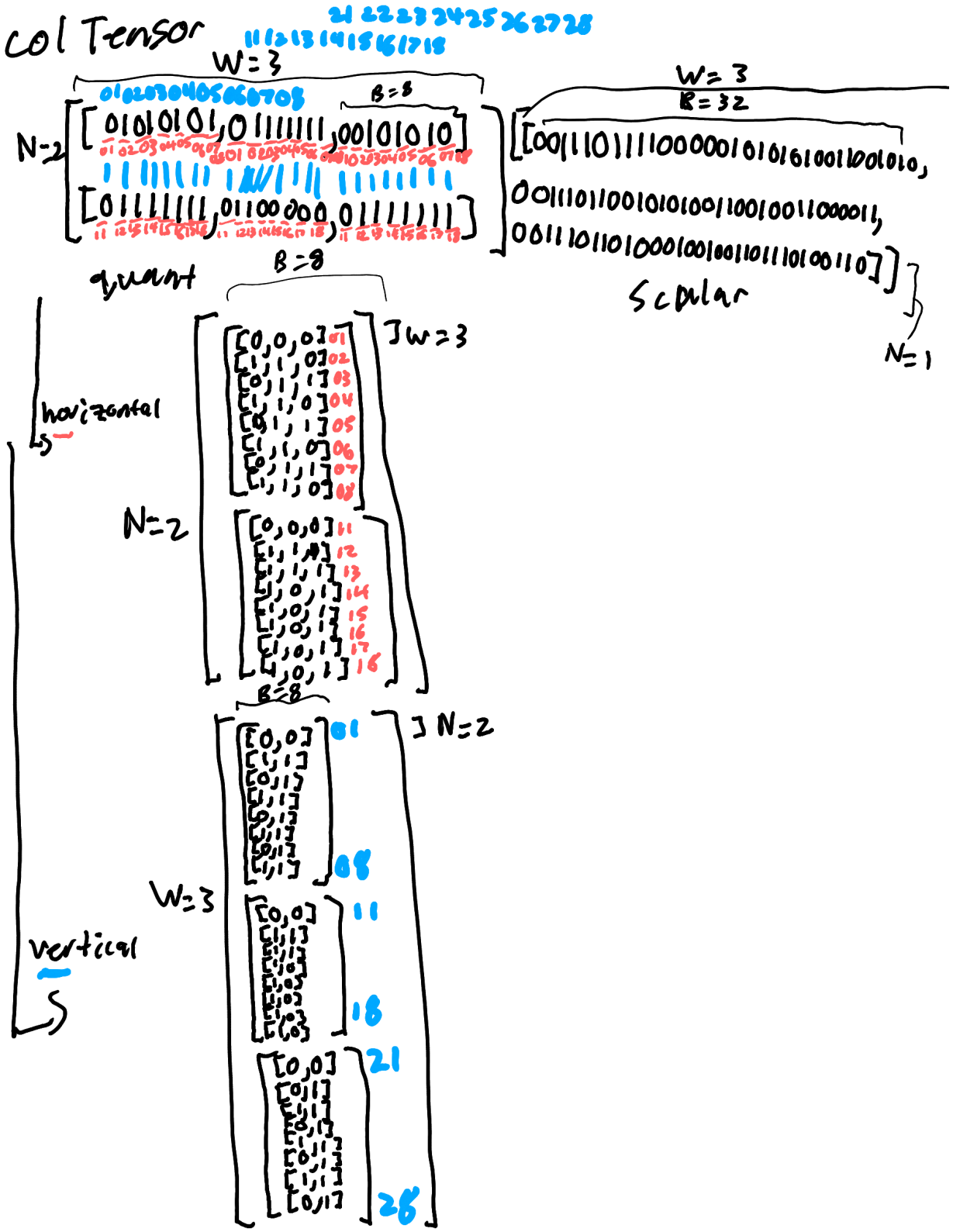
Here is the horizontal result

```
STAGE 3: Bitplane Disaggregation (Quant+Scalar): Q: torch.Size([2, 8, 3]), S: torch.Size([2, 32, 1]) :
Quant:
tensor([[[0, 0, 0],
         [1, 1, 0],
         [1, 0, 1],
         [1, 1, 0],
         [1, 0, 0],
         [1, 1, 0],
         [1, 0, 0],
         [1, 0, 0]],

        [[0, 0, 0],
         [1, 0, 1],
         [1, 1, 0],
         [1, 0, 0],
         [1, 1, 0],
         [1, 0, 0],
         [1, 1, 0],
         [1, 0, 0]]], dtype=torch.uint8)
```

Here is the vertical result

```
STAGE 3: Bitplane Disaggregation (Quant+Scalar): Q: torch.Size([3, 8, 2]), S: torch.Size([1, 32, 2]) :
Quant:
tensor([[[0, 0],
         [1, 1],
         [1, 1],
         [1, 1],
         [1, 1],
         [1, 1],
         [1, 1],
         [1, 1]],

        [[0, 0],
         [1, 0],
         [0, 1],
         [1, 0],
         [0, 1],
         [1, 0],
         [0, 1],
         [0, 0]],

        [[0, 0],
         [0, 1],
         [1, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0],
         [0, 0]]], dtype=torch.uint8)
```

- Column-wise Quantization + Scalar tensors:


col Tensor
W=3
B=8
N=2
W=3
R=32
scalar
N=1
quant
B=8
horizontal
N=2
W=3
B=8
N=2
W=3
vertical

Here is the horizontal result

```
STAGE 3: Bitplane Disaggregation (Quant+Scalar): Q: torch.Size([2, 8, 3]), S: torch.Size([1, 32, 3]) :
Quant:
tensor([[[0, 0, 0],
         [1, 1, 0],
         [0, 1, 1],
         [1, 1, 0],
         [0, 1, 1],
         [1, 1, 0],
         [0, 1, 1],
         [1, 1, 0]],

        [[0, 0, 0],
         [1, 1, 1],
         [1, 1, 1],
         [1, 0, 1],
         [1, 0, 1],
         [1, 0, 1],
         [1, 0, 1],
         [1, 0, 1]]], dtype=torch.uint8)
```

Here is the vertical result

```
STAGE 3: Bitplane Disaggregation (Quant+Scalar): Q: torch.Size([3, 8, 2]), S: torch.Size([3, 32, 1]) :
Quant:
tensor([[[0, 0],
         [1, 1],
         [0, 1],
         [1, 1],
         [0, 1],
         [1, 1],
         [0, 1],
         [1, 1]],

        [[0, 0],
         [1, 1],
         [1, 1],
         [1, 0],
         [1, 0],
         [1, 0],
         [1, 0],
         [1, 0]],

        [[0, 0],
         [0, 1],
         [1, 1],
         [0, 1],
         [1, 1],
         [0, 1],
         [1, 1],
         [0, 1]]], dtype=torch.uint8)
```

- For next time:
    - Finish testing and verifying all bitplane disaggregation (the 3 width scalar)(also the raw bitplanes, need to check them)(for wednesday)
    - Move on to stage 4 RLE analysis (for wednesday)
    - Maybe check bitpacking (if you have time, might be for thursday)
    - Maybe LZ4+ZSTD compression analysis (if you have time, might be for thursday or friday)

2/25: Debugging update / Custom Tensor Dataset: (Tim Update)

- Confirmed working this time:
    - Raw bitplane disaggregation (in horizontal and vertical orientations)
    - Scalars in all bitplane orientations
- Now we need to move on to Stage 4: RLE analysis
    - Checked it all works
- So, RLE works for all the 10 required transformations:
    - Raw_horizontal (no quantization)
    - Raw_vertical (no quantization)
    - Quant_horizontal_row
    - Scale_horizontal_row
    - Quant_vertical_row
    - Scale_vertical_row
    - Quant_horizontal_col
    - Scale_horizontal_col
    - Quant_vertical_col
    - Scale_vertical_col
- I would show the pictures and calculations if possible, but it takes forever and would crowd the pages. I did manually check hand calculations, simulation results, and simulation calculations.
- No modifications for this code was needed. I did have to replace the direct_bitplanes function with a new raw_bitplanes function in the simulations. It is more correct and efficient, using the same type of logic as the quantized+scalar version of the function.
- To do next time:
    - Work on testing and debugging Stage 5: bitpacking (n,b,w)
        - Note: this requires a new and larger test tensor of at least 8x8(x8) with int8 minimum
        - This is a larger tensor, so hand calculations and manual comparisons will take even longer. I have to do manual calculations and multidirectional comparisons on 64-512 different values.
        - To shorten the testing time for this stage, I will skip all the previous stages and just test the bitpacking function with a custom 8x8x8 tensor of 0s and 1s. This will still be quite time consuming.

2/25: Debugging update: Bitpacking in all 3 dimensions works / Custom Tensor Dataset: (Tim Update)

- Summary of progress: Verified stage 5 (bitpacking) in the N, W, and B dimensions of tensor (N,W,B).
- Explanation of N, W, B packing:
    - Lets say our tensor is in the form (N,W,B)
        - Where:
            - N = # of vector embeddings
            - W = Dim of vector embeddings

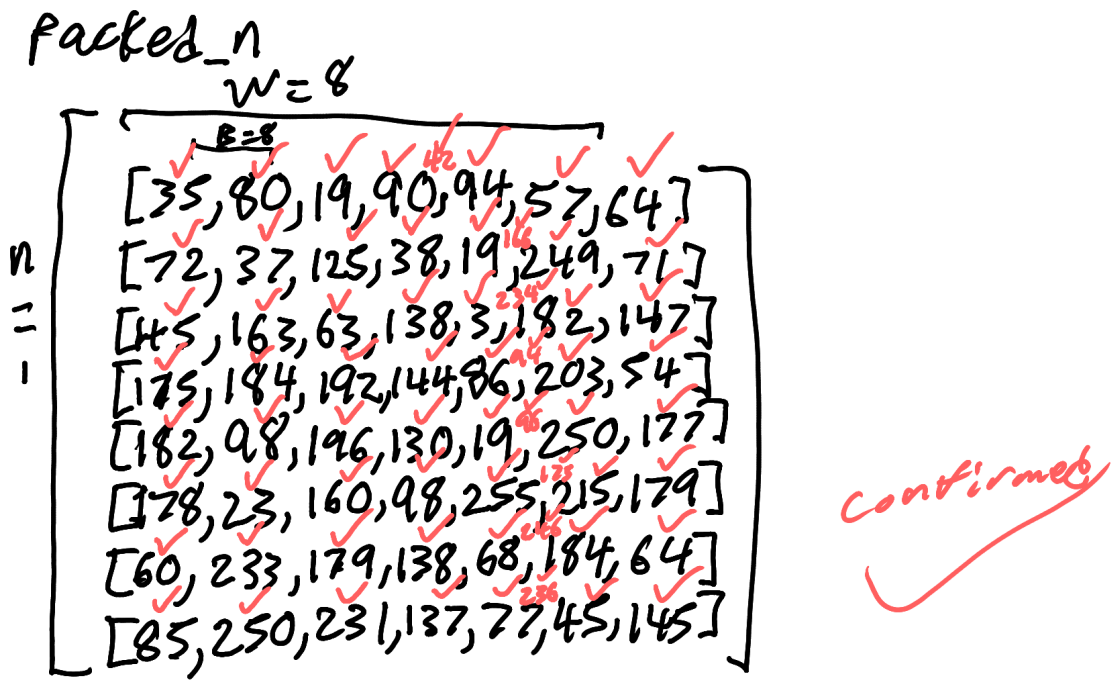
        - B = bitplane length
    - N_packing packs bits in the N dimension: used for Spatial Analysis for LZ4+ZSTD Compression
    - W_packing packs bits in the W dimension: not used for any purpose, may be useful though
    - B_packing packs bits in the B dimension: used for Global and Temporal Analysis for LZ4+ZSTD Compression
- Methods:
    - First, I used an 8x8x8 tensor (N=8,W=8,B=8) of binary 1s and 0s
    - Here it is for this randomized run:

```
Stage 5: Bitpacking: Original Tensor: torch.Size([8, 8, 8])
tensor([[[0, 0, 0, 0, 0, 0, 0, 0],
         [0, 0, 0, 0, 0, 1, 1, 0],
         [0, 1, 0, 1, 0, 1, 1, 1],
         [1, 1, 1, 1, 0, 0, 1, 0],
         [1, 0, 1, 1, 0, 0, 1, 1],
         [1, 0, 1, 0, 1, 1, 1, 1],
         [0, 1, 1, 1, 0, 1, 1, 0],
         [0, 1, 1, 1, 0, 1, 0, 1]],

        [[0, 1, 0, 1, 1, 0, 0, 1],
         [1, 0, 1, 0, 0, 0, 1, 1],
         [0, 0, 0, 0, 0, 1, 0, 0],
         [0, 0, 1, 0, 1, 1, 1, 0],
         [0, 1, 1, 0, 0, 1, 1, 0],
         [0, 0, 0, 1, 1, 0, 1, 0],
         [0, 1, 0, 0, 1, 1, 0, 1],
         [1, 1, 1, 0, 1, 1, 0, 0]],

        [[1, 0, 0, 0, 0, 1, 1, 0],
         [0, 1, 1, 1, 0, 1, 1, 0],
         [1, 1, 1, 0, 0, 1, 1, 0],
         [1, 1, 0, 0, 0, 0, 0, 1],
         [1, 1, 0, 0, 0, 1, 1, 1],
         [1, 0, 1, 1, 1, 1, 0, 1],
         [1, 1, 1, 0, 0, 1, 1, 0],
         [0, 1, 1, 0, 0, 1, 1, 0]],
```

```
[[0, 1, 1, 1, 1, 0, 1, 0],
 [0, 0, 1, 0, 1, 0, 1, 0],
 [0, 0, 1, 0, 0, 0, 1, 1],
 [0, 1, 0, 1, 1, 1, 0, 1],
 [1, 0, 0, 0, 1, 0, 1, 1],
 [1, 1, 0, 0, 1, 0, 1, 1],
 [1, 0, 1, 0, 0, 1, 1, 0],
 [1, 1, 0, 0, 0, 0, 0, 1]],

[[0, 0, 0, 1, 1, 1, 1, 0],
 [1, 0, 1, 0, 0, 0, 1, 0],
 [1, 0, 1, 1, 0, 1, 0, 0],
 [1, 1, 0, 0, 0, 1, 1, 0],
 [0, 0, 0, 0, 0, 0, 1, 0],
 [0, 0, 0, 0, 1, 1, 0, 0],
 [1, 1, 0, 1, 0, 0, 1, 0],
 [0, 1, 0, 1, 1, 1, 1, 0]],

[[0, 0, 0, 0, 1, 0, 0, 0],
 [0, 1, 1, 1, 0, 1, 0, 1],
 [1, 0, 1, 0, 0, 0, 1, 0],
 [1, 0, 0, 0, 1, 1, 0, 1],
 [1, 0, 1, 0, 0, 0, 0, 0],
 [0, 1, 0, 0, 1, 1, 1, 0],
 [1, 0, 0, 0, 1, 1, 0, 0],
 [1, 0, 1, 0, 1, 1, 1, 0]],
```

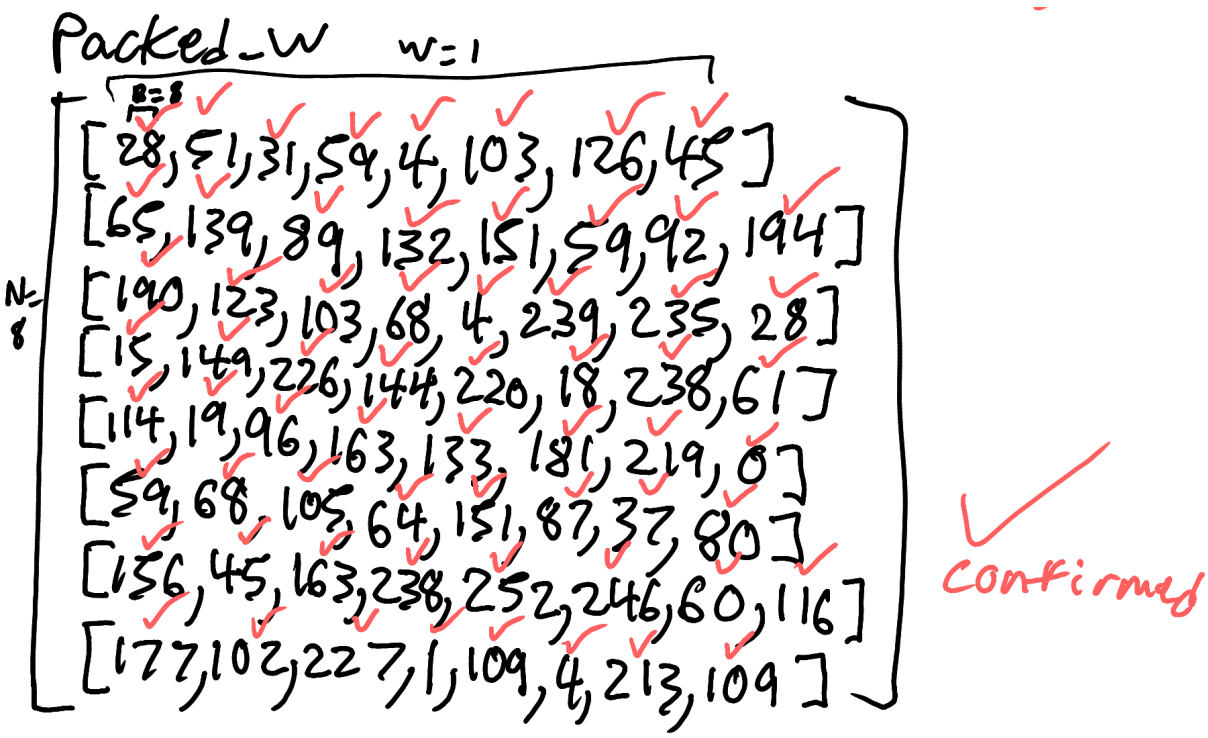
```
[[1, 0, 1, 1, 1, 1, 0, 0],
 [0, 0, 0, 1, 1, 1, 0, 1],
 [0, 1, 1, 1, 1, 1, 1, 1],
 [1, 0, 0, 0, 1, 1, 1, 1],
 [1, 1, 0, 1, 1, 0, 1, 0],
 [1, 1, 0, 1, 1, 1, 1, 1],
 [0, 0, 1, 1, 0, 1, 0, 0],
 [0, 1, 1, 0, 0, 0, 0, 0]],

[[1, 0, 1, 0, 0, 0, 1, 0],
 [0, 1, 1, 0, 1, 0, 1, 1],
 [1, 1, 1, 0, 1, 0, 0, 1],
 [1, 0, 0, 0, 0, 0, 1, 0],
 [0, 0, 0, 0, 1, 0, 0, 1],
 [0, 1, 0, 0, 1, 1, 1, 1],
 [0, 1, 1, 0, 0, 0, 0, 0],
 [1, 0, 1, 1, 1, 0, 1, 1]]], dtype=torch.uint8)
```

- Now here are my calculated values + verification from simulation:
  - Packed_n:



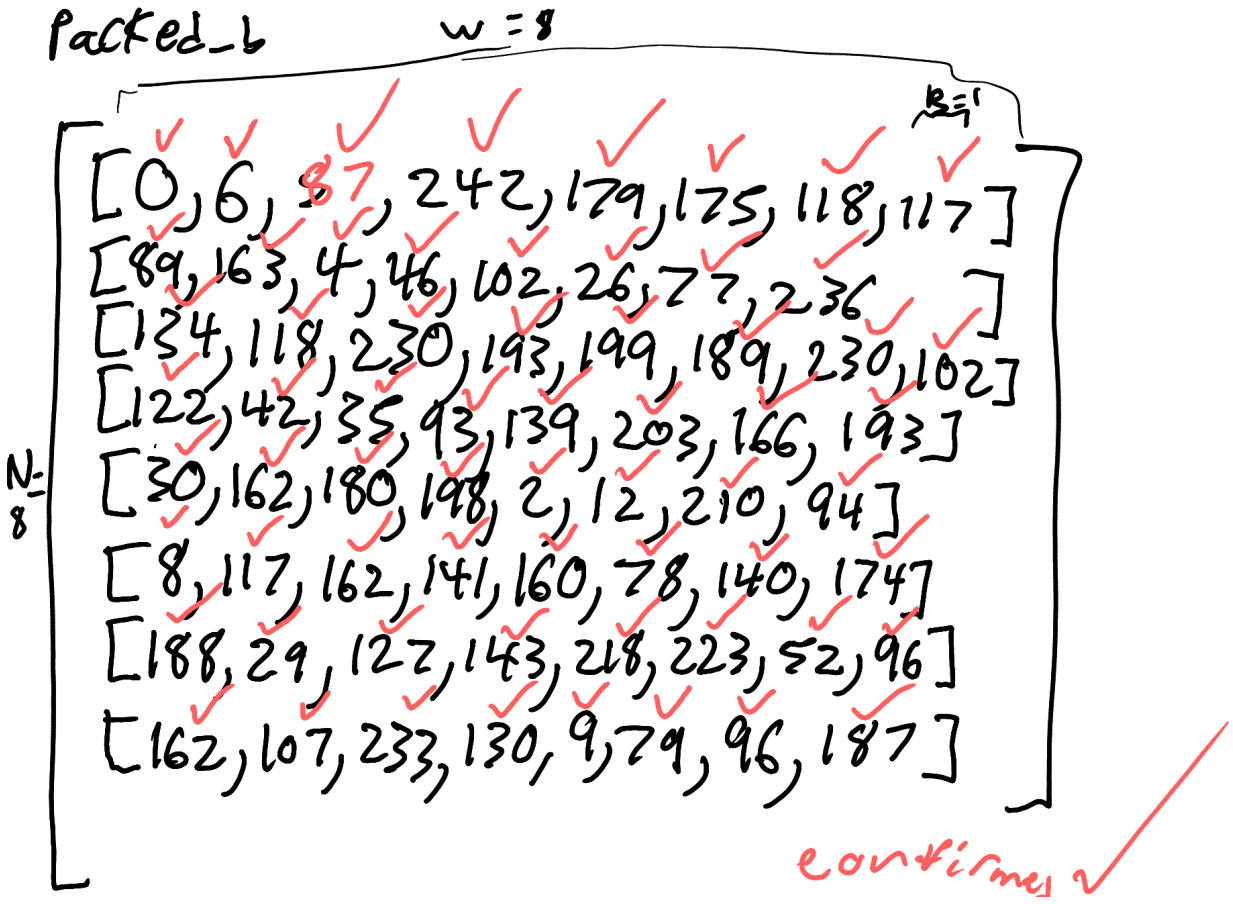
```
pack_n: torch.Size([1, 8, 8])
tensor([[[ 35,  80,  19,  90,  94,  42,  57,  64],
         [ 72,  37, 125,  38,  19, 166, 249,  71],
         [ 45, 163,  63, 138,   3, 234, 182, 147],
         [175, 184, 192, 144,  86,  94, 203,  54],
         [182,  98, 196, 130,  19,  96, 250, 177],
         [178,  23, 160,  98, 255, 175, 215, 179],
         [ 60, 233, 179, 138,  68, 246, 184,  64],
         [ 85, 250, 231, 137,  77, 236,  45, 145]]], device='xpu:0',
       dtype=torch.uint8)
```

- Packed_w:



```
pack_w: torch.Size([8, 1, 8])
tensor([[[ 28,  51,  31,  59,   4, 103, 126,  45]],

        [[ 65, 139,  89, 132, 151,  59,  92, 194]],

        [[190, 123, 103,  68,   4, 239, 235,  28]],

        [[ 15, 149, 226, 144, 220,  18, 238,  61]],

        [[114,  19,  96, 163, 133, 181, 219,   0]],

        [[ 59,  68, 105,  64, 151,  87,  37,  80]],

        [[156,  45, 163, 238, 252, 246,  60, 116]],

        [[177, 102, 227,   1, 109,   4, 213, 109]]], device='xpu:0',
       dtype=torch.uint8)
```

- Packed_b:



```
pack_b: torch.Size([8, 8, 1])
tensor([[[  0],
         [  6],
         [ 87],
         [242],
         [179],
         [175],
         [118],
         [117]],

        [[ 89],
         [163],
         [  4],
         [ 46],
         [102],
         [ 26],
         [ 77],
         [236]],

        [[134],
         [118],
         [230],
         [193],
         [199],
         [189],
         [230],
         [102]],
```

```
       [[122],
        [ 42],
        [ 35],
        [ 93],
        [139],
        [203],
        [166],
        [193]],

       [[ 30],
        [162],
        [180],
        [198],
        [  2],
        [ 12],
        [210],
        [ 94]],

       [[  8],
        [117],
        [162],
        [141],
        [160],
        [ 78],
        [140],
        [174]],
```

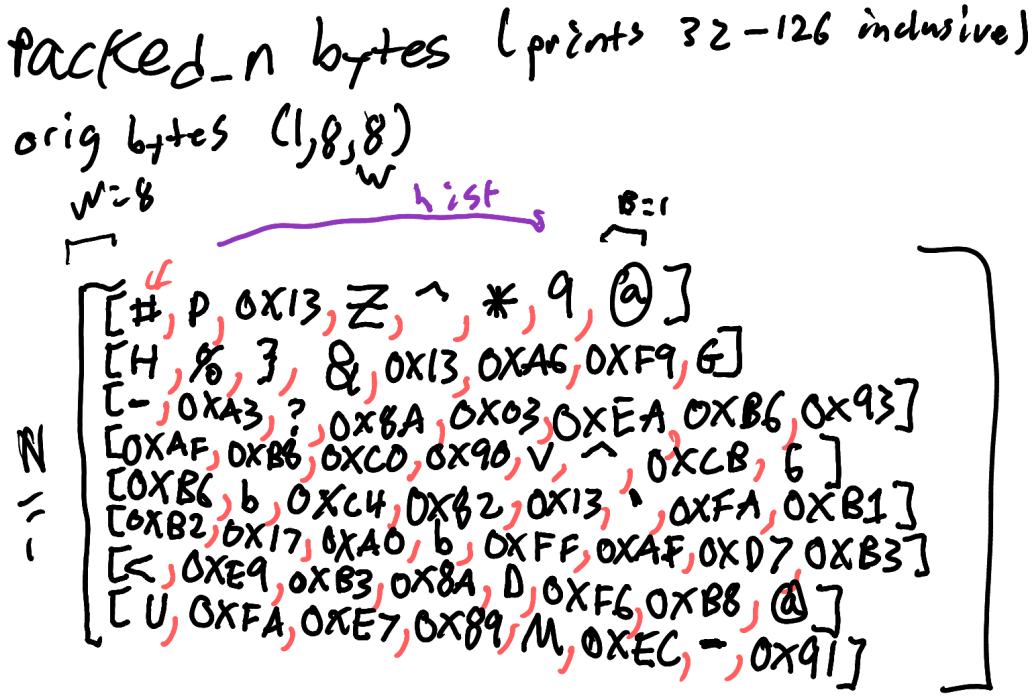
```
        [[188],
         [ 29],
         [127],
         [143],
         [218],
         [223],
         [ 52],
         [ 96]],

        [[162],
         [107],
         [233],
         [130],
         [  9],
         [ 79],
         [ 96],
         [187]]], device='xpu:0',
```

- Conclusion:
    - Technically the transformations and calculations are all correct, although I am not a fan of the format of packed_b, as it is hard to read. But it should be fine
    - As far as I am concerned, none of the code had to be modified to make it work
- To do next time:
    - Test and verify stage 6: LZ4+ZSTD Compression Analysis (Global, Temporal, Spatial)

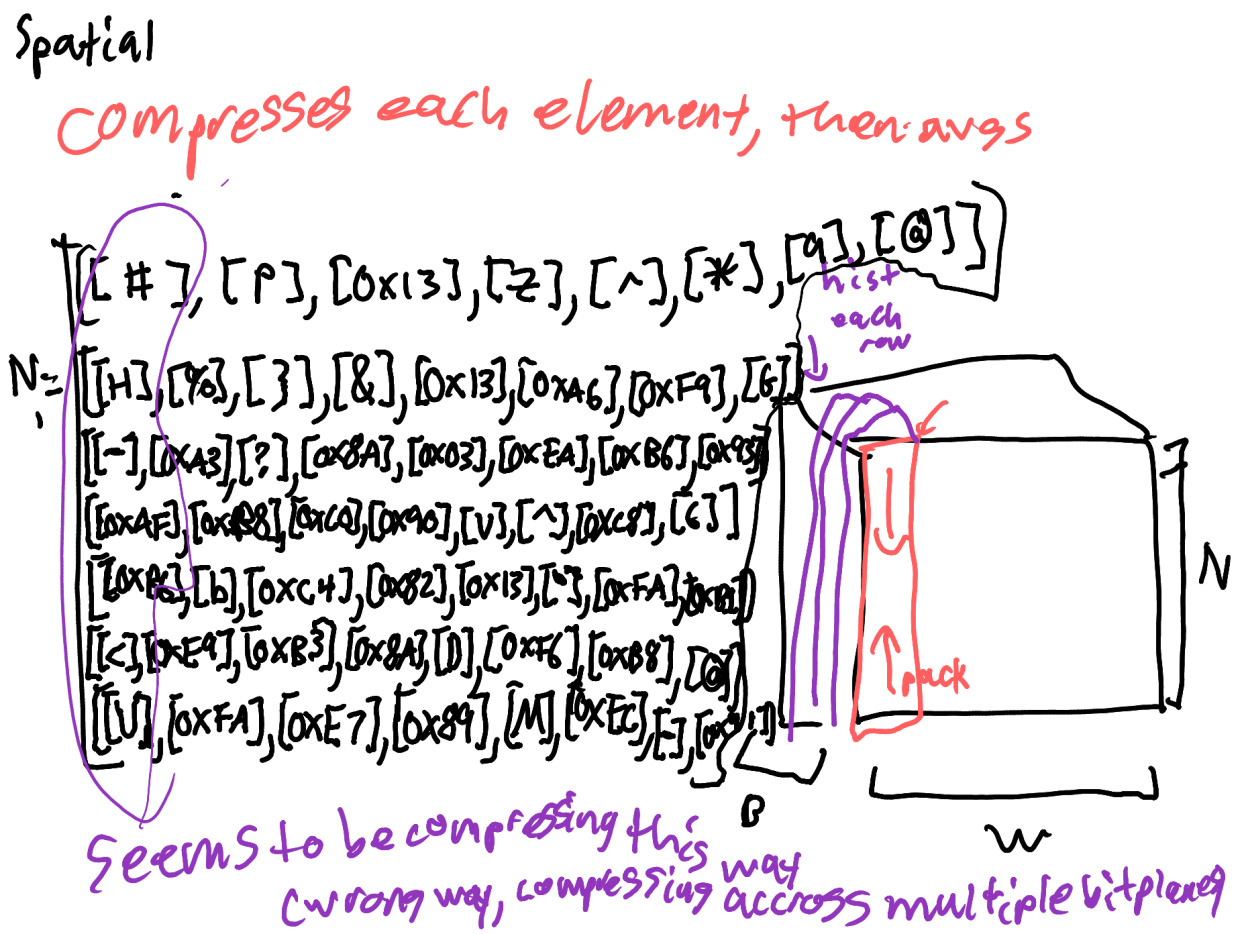
2/26: Debugging Progress Update LZ4+ZSTD Debugging Part 1 / Custom Tensor Dataset: (Tim Update)

- Summary: So far, most of the bit packing seems to work properly, although spatial configuration is having byte direction issues, causing it to mesh together N values across multiple different bitplanes. This causes the calculations to be very wrong. Haven't gotten to verifying the compression ratio calculations yet.
- Tested working:
    - Packed_b across both global and temporal configurations
        - LZ4 and ZSTD byte handling is properly working
- Proved not working properly:
    - Packed_n across spatial configurations
        - Seems to concatenate bytes internally across columns, which would "smear" it across multiple bitplanes. This isn't what we want
- Data Used:
    - Same tensor as previous update (2/25).

- Data comparisons to hand calculations:
  - Packed_n bytes:
    - Original:



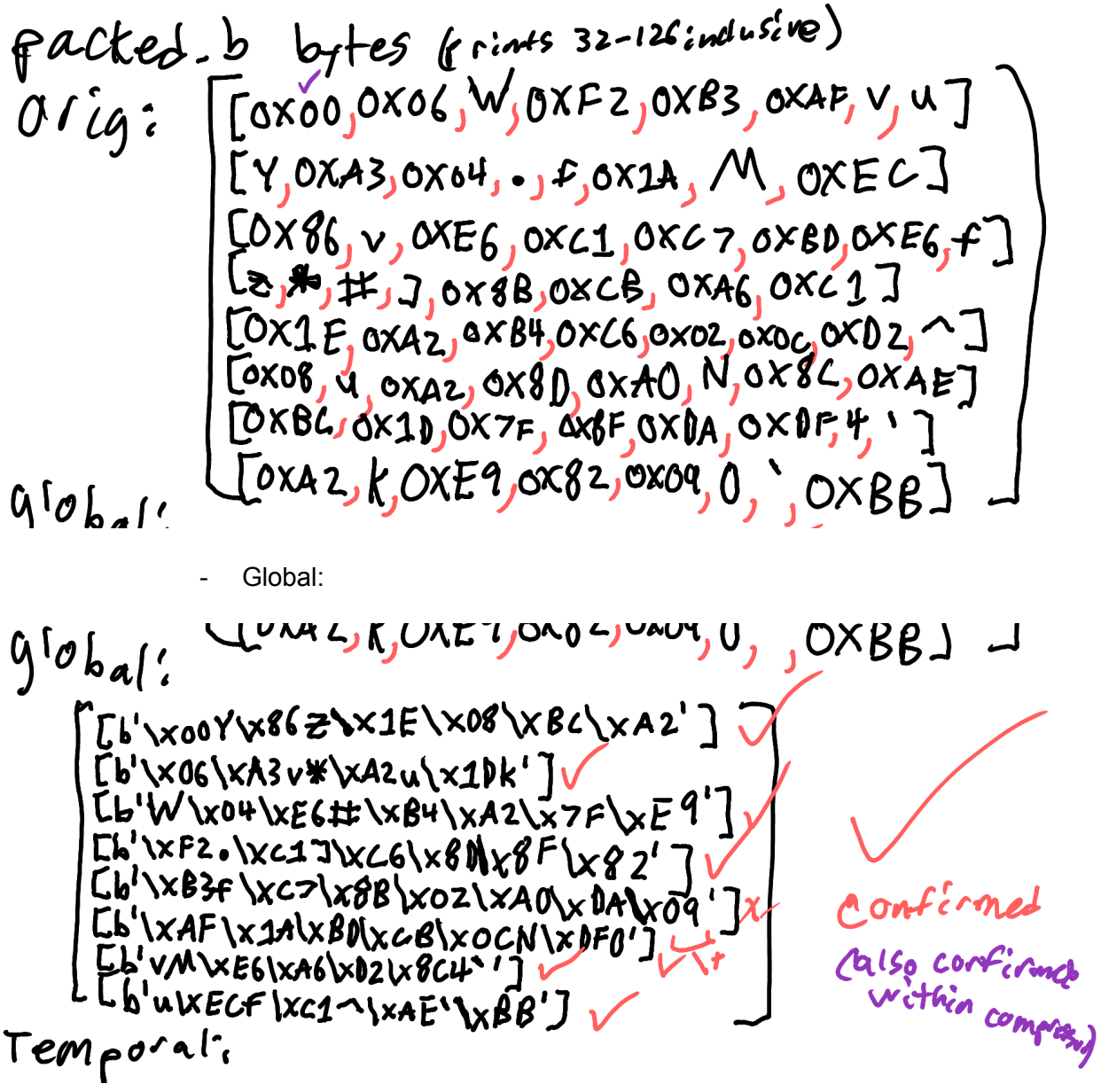
    - Spatial:




```
DEBUG: Spatial bytes: (8, 1):
[[b'#H-\xaf\xb6\xb2<UP%\xa3\xb8b\x17\xe9\xfa\x13}?\xc0\xc4\xa0\xb3\xe7Z&\x8a\x90\x82b\x8a\x89^\x13\x03V\x13\xffDM*\xa6\xea^`\xaf\
xf6\xec9\xf9\xb6\xcb\xfa\xd7\xb8-@G\x936\xb1\xb3@\x91'], [b''], [b''], [b''], [b''], [b''], [b''], [b'']]
```

Notice how it is concatenating across bitplanes, which “smears” it. This kills the accuracy of our calculations. This is wrong. We want it to concatenate across our B dimension, which is labeled “hist” in the picture above, it is row-wise, going across the B dimension.

- Packed_b bytes:
  - Original:



- Global:


```
DEBUG: Global bytes: (8, 1, 8):
[[b'\x00Y\x86z\x1e\x08\xbc\xa2'], [b'\x06\xa3v*\xa2u\x1dk'], [b'W\x04\xe6#\xb4\xa2\x7f\xe9'], [b'\xf2.\xc1]\xc6\x8d\x8f\x82'], [b
'\xb3f\xc7\x8b\x02\xa0\xda\t'], [b'\xaf\x1a\xbd\xcb\x0cN\xdf0'], [b'vM\xe6\xa6\xd2\x8c4`'], [b'u\xecf\xc1^\xae`\xbb']]
```

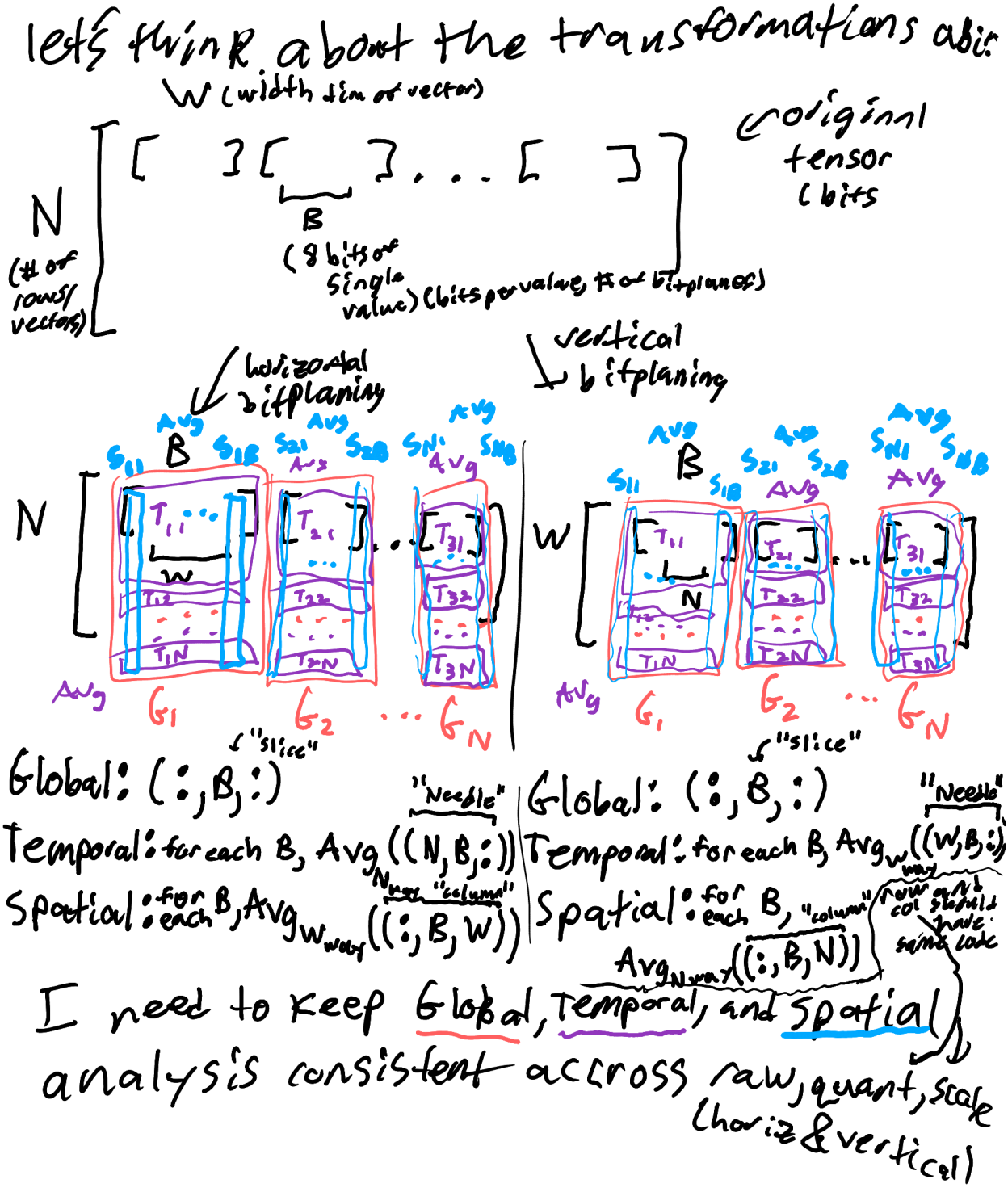
- Temporal:

Temporal:

Temporal is just each value in original,
same data format (but in individual list per element)
↓ within compression is also confirmed
↓ generally makes sense:
Temporal check
↓ would be a list if B≥8
seems for some reason to be a 1D list in ↓ direction, confirmes

```
DEBUG: Temporal bytes: (64, 1):
[[b'\x00'], [b'Y'], [b'\x86'], [b'z'], [b'\x1e'], [b'\x08'], [b'\xbc'], [b'\xa2'], [b'\x06'], [b'\xa3'], [b'v'], [b'*'], [b'\xa2'
], [b'u'], [b'\x1d'], [b'k'], [b'W'], [b'\x04'], [b'\xe6'], [b'#'], [b'\xb4'], [b'\xa2'], [b'\x7f'], [b'\xe9'], [b'\xf2'], [b'.']
, [b'\xc1'], [b']'], [b'\xc6'], [b'\x8d'], [b'\x8f'], [b'\x82'], [b'\xb3'], [b'f'], [b'\xc7'], [b'\x8b'], [b'\x02'], [b'\xa0'], [
b'\xda'], [b'\t'], [b'\xaf'], [b'\x1a'], [b'\xbd'], [b'\xcb'], [b'\x0c'], [b'N'], [b'\xdf'], [b'O'], [b'v'], [b'M'], [b'\xe6'], [
b'\xa6'], [b'\xd2'], [b'\x8c'], [b'4'], [b'`'], [b'u'], [b'\xec'], [b'f'], [b'\xc1'], [b'^'], [b'\xae'], [b'`'], [b'\xbb']]
```

- Some notes:



- To do next:
  - Finish verifying the tensor compression in multiple configs (global, spatial, temporal, packed configs)
  - Finish verifying the math for compression ratios
  - Fix the spatial configuration for byte concatentation

2/27: Mini Update: Custom Tensor LZ4+ZSTD Compression Analysis:

- Summary: as expected, the Global and Temporal results are around 2 and above. The overhead makes the compression ratios worse because the tensor is minimum size, so the compression headers add too much overhead. This won't be too much of a problem when we do high dimensional vector embeddings.
- Data used: Same custom tensor as 2/26 and 2/25
- Results:
    - Horizontal:

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	2.125	3.875	10.0	24.0	
6	6	2.125	3.875	10.0	24.0	
5	5	2.125	3.875	10.0	24.0	
4	4	2.125	3.875	10.0	24.0	
3	3	2.125	3.875	10.0	24.0	
2	2	2.125	3.875	10.0	24.0	
1	1	2.125	3.875	10.0	24.0	
0	0	2.125	3.875	10.0	24.0	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.125	1.375
6	1.125	1.375
5	1.125	1.375
4	1.125	1.375
3	1.125	1.375
2	1.125	1.375
1	1.125	1.375
0	9.125	10.875

    - Vertical:

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	2.125	3.875	10.0	24.0	
6	6	2.125	3.875	10.0	24.0	
5	5	2.125	3.875	10.0	24.0	
4	4	2.125	3.875	10.0	24.0	
3	3	2.125	3.875	10.0	24.0	
2	2	2.125	3.875	10.0	24.0	
1	1	2.125	3.875	10.0	24.0	
0	0	2.125	3.875	10.0	24.0	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.125	1.375
6	1.125	1.375
5	1.125	1.375
4	1.125	1.375
3	1.125	1.375
2	1.125	1.375
1	1.125	1.375
0	9.125	10.875

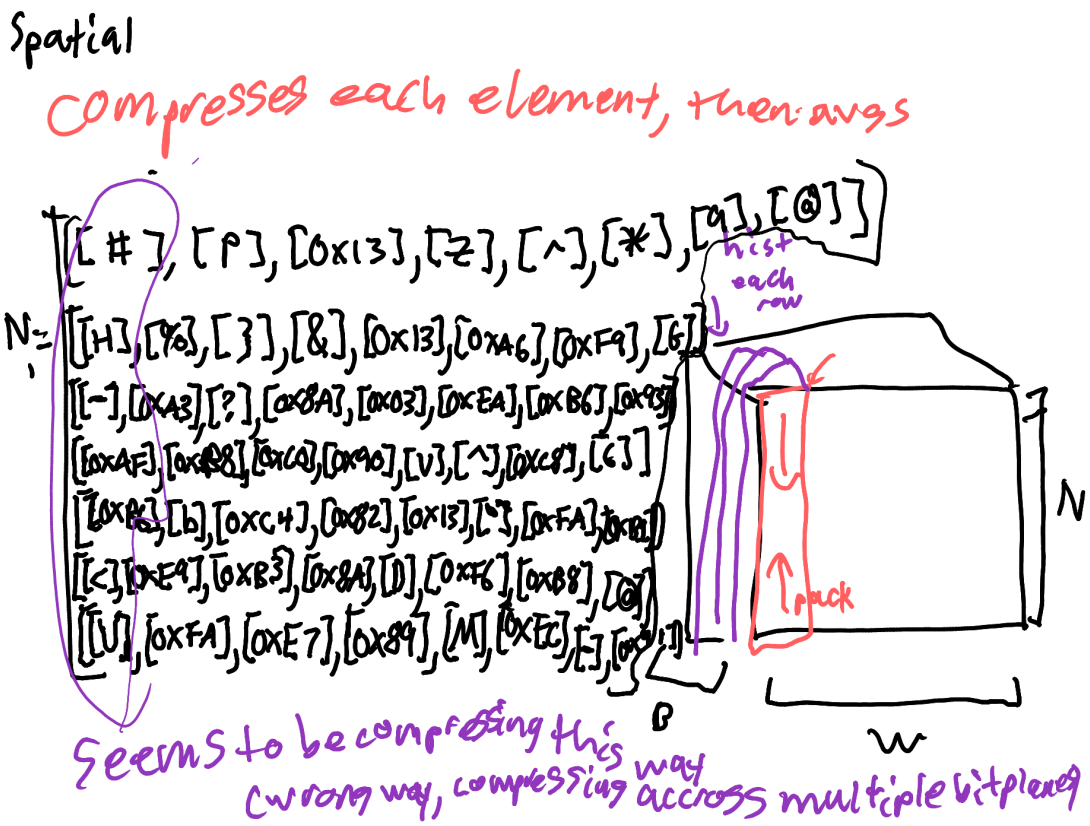
- Some observations:
    - They seem to give the same exact numbers, I may need to look into this a bit more
    - Keep in mind that spatial still isn't quite working yet, hence will give odd numbers
- Progress report:
    - I will finish LZ4+ZSTD compression analysis debugging and verification and continue development of the main compression analysis over break.
- To do next time:
    - Finish debugging and verification of LZ4+ZSTD Compression analysis (also finish spatial debugging, it has some issues)
    - Verify the compression ratios being the same between vertical and horizontal compression

03/05: Debugging Progress Update PT2 (LZ4+ZSTD Compression Analysis) Spatial / Custom Tensor Dataset: (Tim Update)

- Summary: I finally fixed the Spatial analysis for LZ4 + ZSTD. I changed the way the spatial analysis is analyzed. It still does column-granularity compression, but across N-wise and W per bitplane. It still needs to be tested with larger tensors in our real simulation.
- Data used:
    - Same custom tensor as 2/26 update
- Here is what I found was wrong with the original Spatial permutations:
    - It seemingly takes the columns of the packed_n bytes, then concatenates them, then concatenates them again across the rows, then compresses it
    - Here is what it shows in the simulation after packed_n gets permuted for spatial:

```
DEBUG: n_spatial: (8, 8, 1) :
[[[ 35]
  [ 72]
  [ 45]
  [175]
  [182]
  [178]
  [ 60]
  [ 85]]

 [[ 80]
  [ 37]
  [163]
  [184]
  [ 98]
  [ 23]
  [233]
  [250]]

 [[ 19]
  [125]
  [ 63]
  [192]
  [196]
  [160]
  [179]
  [231]]

 [[ 90]
  [ 38]
  [138]
```

- For example, the first 8 values represent the first column from my hand-calculated notes: #H-\xAF\xB6<U
- The next 8 values represent the second column, and so on.



- Then, my spatial tasks processing concatenates these rows together and compresses them. This isnt what I want.

```
for k in range(B_b):
    # Grab the 8 individual bitplanes for this group
    group_bytes = n_spatial[k*8 : (k+1)*8].tobytes()
    spatial_tasks.append([group_bytes])
print("\nDEBUG: Spatial bytes: (" + str(len(spatial_tasks)) + ", " + str(len(spatial_tasks[0])) + "): ")
print(spatial_tasks)
z_ssizes = pool.map(zstd_compress_list, spatial_tasks)
l_ssizes = pool.map(lz4_compress_list, spatial_tasks)
```

```
DEBUG: Spatial bytes: (8, 1):
[[b'#H-\xaf\xb6\xb2<UP%\xa3\xb8b\x17\xe9\xfa\x13}?\xc0\xc4\xa0\xb3\xe7Z&\x8a\x90\x82b\x8a\x89^\x13\x03V\x13\xffDM*\xa6\xea^`\xaf\
xf6\xec9\xf9\xb6\xcb\xfa\xd7\xb8-@G\x936\xb1\xb3@\x91'], [b''], [b''], [b''], [b''], [b''], [b''], [b'']]

DEBUG: ZSTD precompressed bytes: 64 :
b'#H-\xaf\xb6\xb2<UP%\xa3\xb8b\x17\xe9\xfa\x13}?\xc0\xc4\xa0\xb3\xe7Z&\x8a\x90\x82b\x8a\x89^\x13\x03V\x13\xffDM*\xa6\xea^`\xaf\xf
6\xec9\xf9\xb6\xcb\xfa\xd7\xb8-@G\x936\xb1\xb3@\x91'
```

- What I actually need to do will be very CPU intensive. I need to get the packed_n-wise bitplane across each W to be separate tasks for the compressors, then add up the compressed sizes across B dimensions, then get the compression ratio by dividing it by the original size of each slice.

- Actually, it seems that I may be doing this incorrectly. The original tensor permutation of packed_n is correct. It isn't smearing across bitplanes, it is smearing across W. The issue is that it smears bitplanes during the spatial tasks stage. This destroys the compressibility because it meshes everything together, making noise.
- I have a new plan:

    - 1. Take the permuted packed_n tensor, split it into multiple bitplanes that have nested values. So if the packed_n tensor is (N//8, W, B), we split it into B bitplanes, each is (W, N//8)
    - 2. Perform CPU multithreaded LZ4+ZSTD compression analysis on each bitplane (internally concatenates the N dimension to stay consistent)
    - 3. For each bitplane, add up all compressed sizes, then add them up. Then, divide by original bitplane slice size to get compression ratio
    - 4. Print out the results
- Step 1, I got the permuted packed_n tensor, and split it into multiple bitplanes

```
DEBUG: spatial_bitplanes:
[array([[ 35],
       [ 72],
       [ 45],
       [175],
       [182],
       [178],
       [ 60],
       [ 85]], dtype=uint8), array([[ 80],
       [ 37],
       [163],
       [184],
       [ 98],
       [ 23],
       [233],
       [250]], dtype=uint8), array([[ 19],
       [125],
       [ 63],
       [192],
       [196],
       [160],
       [179],
       [231]], dtype=uint8), array([[ 90],
       [ 38],
       [138],
       [144],
```

    - I confirmed that it matches the format I need. There is only 1 element in the innermost dimension due to the N value being exactly 8, so N//8=1.
- Step 2, going through each bitplane, using cpu multithreading to compress each nested vector element per bitplane, then add all of the compressed sizes up.
    - It seems to work correctly. Here is an example of the 0th bitplane (1st column).
    - Ideally, it should compress each of [[#],[H],[-],[\xAF],[\xB2],[\xB6],[<],[U]], but it does it out of order

```
DEBUG: SPATIAL BITPLANE TO COMPRESS:
[[ 35]
 [ 72]
 [ 45]
 [175]
 [182]
 [178]
 [ 60]
 [ 85]]
```

```
DEBUG: ZSTD precompressed bytes: 1 :
b'#'

DEBUG: ZSTD precompressed bytes: 1 :
b'\xb2'

DEBUG: ZSTD precompressed bytes: 1 :
b'U'

DEBUG: ZSTD precompressed bytes: 1 :
b'-'

DEBUG: ZSTD precompressed bytes: 1 :
b'\xaf'

DEBUG: ZSTD precompressed bytes: 1 :
b'H'

DEBUG: ZSTD precompressed bytes: 1 :
b'<'

DEBUG: ZSTD precompressed bytes: 1 :
b'\xb6'
```

```
DEBUG: LZ4 precompressed bytes: 1 :
b'#'

DEBUG: LZ4 precompressed bytes: 1 :
b'H'

DEBUG: LZ4 precompressed bytes: 1 :
b'<'

DEBUG: LZ4 precompressed bytes: 1 :
b'U'

DEBUG: LZ4 precompressed bytes: 1 :
b'-'

DEBUG: LZ4 precompressed bytes: 1 :
b'\xaf'

DEBUG: LZ4 precompressed bytes: 1 :
b'\xb2'

DEBUG: LZ4 precompressed bytes: 1 :
b'\xb6'
```

- It is fine that it does it out of order since we are adding them up anyways
- Here are the results:

```
zstd spatial sizes specific bitplane:
[10, 10, 10, 10, 10, 10, 10, 10]
lz4 spatial sizes specific bitplane:
[24, 24, 24, 24, 24, 24, 24, 24]
```

  - Each element represents the number of compressed bytes per bitplane.
  - Notice how each 1 byte element is compressed to 10 bytes in ZSTD and 24 bytes in LZ4. This is due to the overhead of LZ4 and ZSTD, which are typically around 10-24 bytes. We will need to test much larger tensors to check how the overhead scales with larger tensors.

- Steps 3 + 4: now I need to divide by the original size of each bitplane slice and print out the compression ratio results.
  - I will use the same example/numbers as step 2

```
zstd spatial sizes specific bitplane:
[10, 10, 10, 10, 10, 10, 10, 10]
lz4 spatial sizes specific bitplane:
[24, 24, 24, 24, 24, 24, 24, 24]
z_bp_sizes (all bitplanes):
[80, 80, 80, 80, 80, 80, 80, 80]
l_bp_sizes (all bitplanes):
[192, 192, 192, 192, 192, 192, 192, 192]
   Bitplane  Global(ZSTD):  Global(LZ4):  Temporal(ZSTD):  Temporal(LZ4):  \
7         7          2.125         3.875             10.0             24.0
6         6          2.125         3.875             10.0             24.0
5         5          2.125         3.875             10.0             24.0
4         4          2.125         3.875             10.0             24.0
3         3          2.125         3.875             10.0             24.0
2         2          2.125         3.875             10.0             24.0
1         1          2.125         3.875             10.0             24.0
0         0          2.125         3.875             10.0             24.0

   Spatial(ZSTD):  Spatial(LZ4):
7            10.0           24.0
6            10.0           24.0
5            10.0           24.0
4            10.0           24.0
3            10.0           24.0
2            10.0           24.0
1            10.0           24.0
0            10.0           24.0
```

-
    - Now, as you can see here, the temporal and spatial will yield the same results. This makes sense as they are compressing single bytes, so the overhead is 10-24x higher.
    - This makes sense, as the thing we are dividing it by (orig_size_kb, which is the kiloByte size per bitplane slice) is 0.0078125 kB, which is 8 bytes.
        - ZSTD: 80 bytes per bitplane (compressed) / 8 bytes (original bitplane slice size) = 10.0 compression ratio
        - LZ4: 192 bytes per bitplane (compressed) / 8 bytes (original bitplane slice size) = 24.0 compression ratio
- Conclusion:
    - It seems that the functions have been fixed, verified, and tested.
    - Now, I need to test it on large tensor datasets
- To do next time:
    - Port over the stage 6 code from your debugger to the final simulation
    - Test the final simulation
    - Report results

3/9: Implementation Update: Tensor Transformation oversight / Wikipedia DPR: (Tim Update):

- Summary: I realized I have been getting the wrong results. Spatial was the earliest indicator that something was wrong. I was getting around 0.017-0.07 compression ratios, same value, across all the bitplanes. After doing some math to check, it is wrong. Found that the overhead of ZSTD and LZ4 were causing major issues for all of the bitplane compression techniques.

- Major oversight: Didn't take into account that vertical and horizontal tensor transformations result in tensors that need different packing orientations, or else it causes "smear" and "noise" across bitplanes before compression.
    - I also forgot to take into account that the bitplanes are stored differently across horizontal and vertical bitplane tensors
    - - horizontal: gets the 8-32 bitplanes (1 bitplane per bit position) for each vector embedding (of N) --> number of bitplanes = (8 to 32)*N
        - Importance: measures compression "on the fly" per token
    - - vertical: gets 8-32 bitplanes (1 bitplane per bit position) for each vector dim across all vector embeddings (of N) --> number of bitplanes = (8 to 32)*W
        - Importance: measures total compression across all tokens
- Mathematical Explanation:
    - Lets say our original tensor is in the form (N,W,B), where:
        - N = # of rows of vector embeddings
        - W = dim of each vector embedding (typically 768 or 1024)
        - B = bit representation of each value (typically 8 or 32)
    - Now, we reinterpret those bit tensors into 2 different forms:
        - Horizontal: (N,W,B) → (B,N,W)
        - Vertical: (N,W,B) → (W,B,N)
    - Now, we feed each of those forms into the bitpacking mechanisms:
        - This is what it looked like before:
            - Original tensor: (N2,W2,B2)
            - Packed_n: (N2,W2,B2) → (N2//8,W2,B2)
            - Packed_w: (N2,W2,B2) → (N2,W2//8,B2)
            - Packed_b: (N2,W2,B2) → (N2,W2,B2//8)
            - Where:
                - Horizontal:
                    - N2 = B
                    - W2 = N
                    - B2 = W
                - Vertical:
                    - N2 = W
                    - W2 = B
                    - B2 = N
        - This is how to reinterpret it:
            - Horizontal input: (B,N,W)
                - Packed_n: (B//8,N,W)
                - Packed_w: (B,N,W//8) → good for spatial, packs 8 features (8 of W) together for a specific bitplane and token [redundancy across embedding dimension]
                - Packed_b: (B,N//8,W) → Temporal/Sequence packing, packs 8 tokens (packs 8 rows of N) together per specific bitplane and feature
            - Vertical input: (W,B,N)

        - Packed_n: (W//8,B,N) → packs across features
        - Packed_w: (W,B//8,N) → packs across bitplanes
        - Packed_b: (W,B,N//8) → packs across sequence/time/rows/tokens [Temporal]
    - Now, we feed these tensors into the Global, Temporal, and Spatial benchmarks, this is the part where we need to start changing our implementation:
- Conceptual discoveries:
    - Horizontal bitplane disaggregation: good for per-token compression. Good for on-the-fly compression
    - Vertical bitplane disaggregation: good for KV Cache compression optimization
- What I am currently working on:
    - I need to rewrite my run_phased_benchmark function to account for the horizontal and vertical tensor arrangements to yield more accurate results.
- To do for next time:
    - Finish creating run_phased_benchmark function to account for horizontal and vertical tensor arrangements
    - Record results in this documentation

3/10: Another oversight / Wikipedia DPR: (Tim update)

- Summary: I rewrote the phased benchmark function for LZ4+ZSTD for the 5th time. When porting it over to the main simulation, I was getting odd numbers on the horizontal bitplane disaggregation. I made another oversight
- Major oversight: While vertical interpretations and handling was correct, the horizontal handling was incorrect. I previously thought that the horizontal tensor input is in the form of (B,N,W), but it is actually in the form of (N,B,W). This means I have to rewrite the horizontal half of the function.
- Mathematical explanation:
    - If the horizontal tensor input is (N,B,W), this means that the granularity of a single bitplane is (1,W), and there are BxN of these.
        - In this state, we want to ideally view the redundancy for each vector embedding separately, and get the average. This makes temporal and global the ideal benchmarks for this, although I will still also include spatial benchmarks with this tensor type too.
            - Packed_b: (packs across W) - used for global and temporal benchmarks
            - Packed_n: (packs across N) - used for spatial benchmarks
- Some miscellaneous changes to the main simulation:
    - New global settings letting you choose which stages of the compression analytics to view. This cleans up the output, and also helps improve performance
    - Deleted deprecated code and functions that were crowding the file

3/10: Progress update: LZ4+ZSTD Benchmark Tested Working / Custom Tensor Dataset: (Tim update)

- Summary: I tested the 6th version of the benchmark function, and it shows the symbols as intended.
- To do next:
    - Put real benchmark results in documentation
    - Put updated code in github
    - Explain findings

3/10: Major Findings Update: Data Collected (LZ4+ZSTD) / Wikipedia DPR: (Tim update)

- Summary: The compression results weren't as compressible as I was expecting, as most of the bitplanes were noise.
    - The first few bitplanes are highly compressible.
    - Some results have very weak compression ratios (80 and 192) due to compression header overhead from LZ4 and ZSTD,
        - This is mainly due to them only compressing individual bytes instead of groups of bytes. This suggests that the scales are only compressible in certain directions.
- Data Used:
    - Dataset: Wikipedia DPR
    - Starting Row: 0
    - # of Rows: 512
    - File used: train-00000-of-00157.parquet
- Data collected:
    - Horizontal:
        - Row:
            - Quant:

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.0	1.0	1.139	1.358	
6	6	1.0	1.0	1.139	1.358	
5	5	1.0	1.0	1.139	1.358	
4	4	1.0	1.0	1.139	1.358	
3	3	1.0	1.0	1.117	1.335	
2	2	1.0	1.0	1.046	1.273	
1	1	1.0	1.0	1.046	1.273	
0	0	1.0	1.0	1.046	1.273	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.094	1.24
6	1.094	1.24
5	1.094	1.24
4	1.094	1.24
3	1.094	1.24
2	1.094	1.24
1	1.094	1.24
0	1.094	1.24

            -
            - Scale:

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	2.484	5.375	1.141	1.359	
30	30	2.453	5.125	1.141	1.359	
29	29	2.438	5.234	1.141	1.359	
28	28	2.516	5.328	1.141	1.359	
27	27	2.406	5.047	1.141	1.359	
26	26	2.578	5.250	1.141	1.359	
25	25	2.531	5.016	1.141	1.359	
24	24	2.562	5.234	1.141	1.359	
23	23	2.453	5.281	1.141	1.359	
22	22	2.594	5.094	1.141	1.359	
21	21	2.453	5.453	1.141	1.359	
20	20	2.516	5.234	1.141	1.359	
19	19	2.375	5.078	1.141	1.359	
18	18	2.531	5.172	1.141	1.359	
17	17	2.406	5.156	1.141	1.359	
16	16	2.547	5.203	1.141	1.359	
15	15	2.578	5.297	1.141	1.359	
14	14	2.453	5.297	1.141	1.359	
13	13	2.641	5.297	1.141	1.359	
12	12	2.312	4.656	1.141	1.359	
11	11	1.812	2.859	1.000	1.312	
10	10	0.953	1.500	0.531	0.828	
9	9	0.953	1.500	0.531	0.828	
8	8	0.297	0.547	0.266	0.531	
7	7	0.297	0.547	0.266	0.531	
6	6	0.297	0.547	0.266	0.531	
5	5	0.297	0.547	0.266	0.531	
4	4	0.297	0.547	0.266	0.531	
3	3	0.297	0.547	0.266	0.531	
2	2	0.297	0.547	0.266	0.531	
1	1	0.297	0.547	0.266	0.531	
0	0	0.297	0.547	0.266	0.531	

    -

	Spatial(ZSTD):	Spatial(LZ4):
31	80.0	192.0
30	80.0	192.0
29	80.0	192.0
28	80.0	192.0
27	80.0	192.0
26	80.0	192.0
25	80.0	192.0
24	80.0	192.0
23	80.0	192.0
22	80.0	192.0
21	80.0	192.0
20	80.0	192.0
19	80.0	192.0
18	80.0	192.0
17	80.0	192.0
16	80.0	192.0
15	80.0	192.0
14	80.0	192.0
13	80.0	192.0
12	80.0	192.0
11	80.0	192.0
10	80.0	192.0
9	80.0	192.0
8	80.0	192.0
7	80.0	192.0
6	80.0	192.0
5	80.0	192.0
4	80.0	192.0
3	80.0	192.0
2	80.0	192.0
1	80.0	192.0
0	80.0	192.0

- Col:
    - Quant:
    -

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.0	1.0	1.141	1.359	
6	6	1.0	1.0	1.141	1.359	
5	5	1.0	1.0	1.141	1.359	
4	4	1.0	1.0	1.141	1.359	
3	3	1.0	1.0	1.140	1.359	
2	2	1.0	1.0	1.139	1.358	
1	1	1.0	1.0	1.139	1.358	
0	0	1.0	1.0	1.048	1.275	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.094	1.24
6	1.094	1.24
5	1.094	1.24
4	1.094	1.24
3	1.094	1.24
2	1.094	1.24
1	1.094	1.24
0	1.094	1.24

    - Scale:

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.094	1.240	80.0	192.0	
30	30	1.094	1.240	80.0	192.0	
29	29	1.094	1.240	80.0	192.0	
28	28	1.094	1.240	80.0	192.0	
27	27	1.094	1.240	80.0	192.0	
26	26	1.094	1.240	80.0	192.0	
25	25	1.094	1.240	80.0	192.0	
24	24	1.094	1.240	80.0	192.0	
23	23	1.094	1.240	80.0	192.0	
22	22	1.094	1.240	80.0	192.0	
21	21	1.094	1.240	80.0	192.0	
20	20	1.094	1.240	80.0	192.0	
19	19	1.094	1.240	80.0	192.0	
18	18	1.094	1.240	80.0	192.0	
17	17	1.094	1.240	80.0	192.0	
16	16	1.094	1.240	80.0	192.0	
15	15	1.094	1.240	80.0	192.0	
14	14	1.094	1.240	80.0	192.0	
13	13	1.094	1.240	80.0	192.0	
12	12	1.094	1.240	80.0	192.0	
11	11	1.094	1.240	80.0	192.0	
10	10	1.094	1.240	80.0	192.0	
9	9	1.094	1.240	80.0	192.0	
8	8	1.094	1.240	80.0	192.0	
7	7	1.094	1.240	80.0	192.0	
6	6	1.094	1.240	80.0	192.0	
5	5	1.094	1.240	80.0	192.0	
4	4	0.177	0.354	80.0	192.0	
3	3	0.177	0.354	80.0	192.0	
2	2	0.177	0.354	80.0	192.0	
1	1	0.177	0.354	80.0	192.0	
0	0	0.177	0.354	80.0	192.0	

-

	Spatial(ZSTD):	Spatial(LZ4):
31	1.094	1.240
30	1.094	1.240
29	1.094	1.240
28	1.094	1.240
27	1.094	1.240
26	1.094	1.240
25	1.094	1.240
24	1.094	1.240
23	1.094	1.240
22	1.094	1.240
21	1.094	1.240
20	1.094	1.240
19	1.094	1.240
18	1.094	1.240
17	1.094	1.240
16	1.094	1.240
15	1.094	1.240
14	1.094	1.240
13	1.094	1.240
12	1.094	1.240
11	1.094	1.240
10	1.094	1.240
9	1.094	1.240
8	1.094	1.240
7	1.094	1.240
6	1.094	1.240
5	1.094	1.240
4	0.177	0.354
3	0.177	0.354
2	0.177	0.354
1	0.177	0.354
0	0.177	0.354

-

- Direct:

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.000	1.000	1.141	1.359	
30	30	1.000	1.000	1.141	1.359	
29	29	1.000	1.000	1.141	1.359	
28	28	1.000	1.000	1.141	1.359	
27	27	1.000	1.000	1.141	1.359	
26	26	1.000	1.000	1.141	1.359	
25	25	1.000	1.000	1.141	1.359	
24	24	1.000	1.000	1.141	1.359	
23	23	1.000	1.000	1.141	1.359	
22	22	1.000	1.000	1.141	1.359	
21	21	1.000	1.000	1.141	1.359	
20	20	1.000	1.000	1.141	1.359	
19	19	1.000	1.000	1.141	1.359	
18	18	1.000	1.000	1.141	1.359	
17	17	1.000	1.000	1.141	1.359	
16	16	1.000	1.000	1.141	1.359	
15	15	1.000	1.000	1.141	1.359	
14	14	1.000	1.000	1.141	1.359	
13	13	1.000	1.000	1.141	1.359	
12	12	1.000	1.000	1.141	1.359	
11	11	1.000	1.000	1.140	1.359	
10	10	1.000	1.000	1.140	1.359	
9	9	0.984	1.000	1.140	1.359	
8	8	1.000	1.000	1.139	1.358	
7	7	0.956	1.000	1.137	1.356	
6	6	0.851	1.000	1.104	1.324	
5	5	0.223	0.429	0.634	0.938	
4	4	0.005	0.015	0.269	0.536	
3	3	0.001	0.005	0.266	0.531	
2	2	0.001	0.005	0.266	0.531	
1	1	0.001	0.005	0.266	0.531	
0	0	1.000	1.000	1.048	1.275	

-

	Spatial(ZSTD):	Spatial(LZ4):
31	1.094	1.240
30	1.094	1.240
29	1.094	1.240
28	1.094	1.240
27	1.094	1.240
26	1.094	1.240
25	1.094	1.240
24	1.094	1.240
23	1.094	1.240
22	1.094	1.240
21	1.094	1.240
20	1.094	1.240
19	1.094	1.240
18	1.094	1.240
17	1.094	1.240
16	1.094	1.240
15	1.094	1.240
14	1.094	1.240
13	1.094	1.240
12	1.094	1.240
11	1.094	1.240
10	1.094	1.240
9	1.094	1.240
8	1.094	1.240
7	1.094	1.240
6	1.094	1.240
5	0.598	0.833
4	0.221	0.441
3	0.219	0.438
2	0.219	0.438
1	0.219	0.438
0	1.094	1.240

-
- Vertical:
  - Row:
    - Quant:

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.000	1.000	1.139	1.358	
6	6	1.000	1.000	1.139	1.358	
5	5	1.000	1.000	1.139	1.358	
4	4	0.959	1.000	1.139	1.358	
3	3	0.749	0.909	1.117	1.335	
2	2	0.687	0.833	1.046	1.273	
1	1	0.686	0.833	1.046	1.273	
0	0	0.686	0.833	1.046	1.273	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.094	1.24
6	1.094	1.24
5	1.094	1.24
4	1.094	1.24
3	1.094	1.24
2	1.094	1.24
1	1.094	1.24
0	1.094	1.24

-
    - Scale:

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.141	1.359	1.141	1.359	
30	30	1.141	1.359	1.141	1.359	
29	29	1.141	1.359	1.141	1.359	
28	28	1.141	1.359	1.141	1.359	
27	27	1.141	1.359	1.141	1.359	
26	26	1.141	1.359	1.141	1.359	
25	25	1.141	1.359	1.141	1.359	
24	24	1.141	1.359	1.141	1.359	
23	23	1.141	1.359	1.141	1.359	
22	22	1.141	1.359	1.141	1.359	
21	21	1.141	1.359	1.141	1.359	
20	20	1.141	1.359	1.141	1.359	
19	19	1.141	1.359	1.141	1.359	
18	18	1.141	1.359	1.141	1.359	
17	17	1.141	1.359	1.141	1.359	
16	16	1.141	1.359	1.141	1.359	
15	15	1.141	1.359	1.141	1.359	
14	14	1.141	1.359	1.141	1.359	
13	13	1.141	1.359	1.141	1.359	
12	12	1.141	1.359	1.141	1.359	
11	11	1.000	1.312	1.000	1.312	
10	10	0.531	0.828	0.531	0.828	
9	9	0.531	0.828	0.531	0.828	
8	8	0.266	0.531	0.266	0.531	
7	7	0.266	0.531	0.266	0.531	
6	6	0.266	0.531	0.266	0.531	
5	5	0.266	0.531	0.266	0.531	
4	4	0.266	0.531	0.266	0.531	
3	3	0.266	0.531	0.266	0.531	
2	2	0.266	0.531	0.266	0.531	
1	1	0.266	0.531	0.266	0.531	
0	0	0.266	0.531	0.266	0.531	

-

	Spatial(ZSTD):	Spatial(LZ4):
31	80.0	192.0
30	80.0	192.0
29	80.0	192.0
28	80.0	192.0
27	80.0	192.0
26	80.0	192.0
25	80.0	192.0
24	80.0	192.0
23	80.0	192.0
22	80.0	192.0
21	80.0	192.0
20	80.0	192.0
19	80.0	192.0
18	80.0	192.0
17	80.0	192.0
16	80.0	192.0
15	80.0	192.0
14	80.0	192.0
13	80.0	192.0
12	80.0	192.0
11	80.0	192.0
10	80.0	192.0
9	80.0	192.0
8	80.0	192.0
7	80.0	192.0
6	80.0	192.0
5	80.0	192.0
4	80.0	192.0
3	80.0	192.0
2	80.0	192.0
1	80.0	192.0
0	80.0	192.0

- Col:
  - Quant:

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.000	1.000	1.141	1.359	
6	6	1.000	1.000	1.141	1.359	
5	5	1.000	1.000	1.141	1.359	
4	4	1.000	1.000	1.141	1.359	
3	3	1.000	1.000	1.140	1.359	
2	2	1.000	1.000	1.139	1.358	
1	1	0.919	0.999	1.139	1.358	
0	0	0.688	0.837	1.048	1.275	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.094	1.24
6	1.094	1.24
5	1.094	1.24
4	1.094	1.24
3	1.094	1.24
2	1.094	1.24
1	1.094	1.24
0	1.094	1.24

  - Scale:

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	2.219	5.052	80.0	192.0	
30	30	2.323	5.177	80.0	192.0	
29	29	2.146	5.052	80.0	192.0	
28	28	2.229	5.240	80.0	192.0	
27	27	2.240	5.104	80.0	192.0	
26	26	2.208	5.062	80.0	192.0	
25	25	2.271	5.062	80.0	192.0	
24	24	2.250	5.188	80.0	192.0	
23	23	2.167	5.167	80.0	192.0	
22	22	2.146	5.073	80.0	192.0	
21	21	2.219	5.042	80.0	192.0	
20	20	2.281	5.125	80.0	192.0	
19	19	2.229	5.240	80.0	192.0	
18	18	2.250	5.115	80.0	192.0	
17	17	2.229	5.021	80.0	192.0	
16	16	2.292	5.125	80.0	192.0	
15	15	2.188	4.906	80.0	192.0	
14	14	2.375	5.083	80.0	192.0	
13	13	2.042	5.229	80.0	192.0	
12	12	2.333	5.260	80.0	192.0	
11	11	2.115	5.229	80.0	192.0	
10	10	2.281	4.969	80.0	192.0	
9	9	2.229	5.208	80.0	192.0	
8	8	2.031	4.938	80.0	192.0	
7	7	2.042	5.010	80.0	192.0	
6	6	2.083	4.750	80.0	192.0	
5	5	2.083	4.750	80.0	192.0	
4	4	0.198	0.375	80.0	192.0	
3	3	0.198	0.375	80.0	192.0	
2	2	0.198	0.375	80.0	192.0	
1	1	0.198	0.375	80.0	192.0	
0	0	0.198	0.375	80.0	192.0	

-

	Spatial(ZSTD):	Spatial(LZ4):
31	1.094	1.240
30	1.094	1.240
29	1.094	1.240
28	1.094	1.240
27	1.094	1.240
26	1.094	1.240
25	1.094	1.240
24	1.094	1.240
23	1.094	1.240
22	1.094	1.240
21	1.094	1.240
20	1.094	1.240
19	1.094	1.240
18	1.094	1.240
17	1.094	1.240
16	1.094	1.240
15	1.094	1.240
14	1.094	1.240
13	1.094	1.240
12	1.094	1.240
11	1.094	1.240
10	1.094	1.240
9	1.094	1.240
8	1.094	1.240
7	1.094	1.240
6	1.094	1.240
5	1.094	1.240
4	0.177	0.354
3	0.177	0.354
2	0.177	0.354
1	0.177	0.354
0	0.177	0.354

-
- Direct:

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.000	1.000	1.141	1.359	
30	30	1.000	1.000	1.141	1.359	
29	29	1.000	1.000	1.141	1.359	
28	28	1.000	1.000	1.141	1.359	
27	27	1.000	1.000	1.141	1.359	
26	26	1.000	1.000	1.141	1.359	
25	25	1.000	1.000	1.141	1.359	
24	24	1.000	1.000	1.141	1.359	
23	23	1.000	1.000	1.141	1.359	
22	22	1.000	1.000	1.141	1.359	
21	21	1.000	1.000	1.141	1.359	
20	20	1.000	1.000	1.141	1.359	
19	19	1.000	1.000	1.141	1.359	
18	18	1.000	1.000	1.141	1.359	
17	17	1.000	1.000	1.141	1.359	
16	16	1.000	1.000	1.141	1.359	
15	15	1.000	1.000	1.141	1.359	
14	14	1.000	1.000	1.141	1.359	
13	13	1.000	1.000	1.141	1.359	
12	12	1.000	1.000	1.141	1.359	
11	11	1.000	1.000	1.140	1.359	
10	10	1.000	1.000	1.140	1.359	
9	9	0.983	1.000	1.140	1.359	
8	8	1.000	1.000	1.139	1.358	
7	7	0.931	1.000	1.137	1.356	
6	6	0.799	0.925	1.104	1.324	
5	5	0.208	0.382	0.634	0.938	
4	4	0.004	0.009	0.269	0.536	
3	3	0.001	0.005	0.266	0.531	
2	2	0.001	0.005	0.266	0.531	
1	1	0.001	0.005	0.266	0.531	
0	0	0.688	0.837	1.048	1.275	

-

	Spatial(ZSTD):	Spatial(LZ4):
31	1.094	1.240
30	1.094	1.240
29	1.094	1.240
28	1.094	1.240
27	1.094	1.240
26	1.094	1.240
25	1.094	1.240
24	1.094	1.240
23	1.094	1.240
22	1.094	1.240
21	1.094	1.240
20	1.094	1.240
19	1.094	1.240
18	1.094	1.240
17	1.094	1.240
16	1.094	1.240
15	1.094	1.240
14	1.094	1.240
13	1.094	1.240
12	1.094	1.240
11	1.094	1.240
10	1.094	1.240
9	1.094	1.240
8	1.094	1.240
7	1.094	1.240
6	1.094	1.240
5	0.598	0.833
4	0.221	0.441
3	0.219	0.438
2	0.219	0.438
1	0.219	0.438
0	1.094	1.240

-

- To do next time:
    - Add more analytics (avg compression ratio per global temporal spatial, calculated compression ratio per global temporal spatial, total compression ratio for scalars+quantized)
    - Add explanations for each result output
    - Block compression control for ZSTD (already have it for LZ4)

3/11: Mini debugging update: Checking tensor consistency for bitpacking and bitplaning functions / Custom Tensor set 2: (Tim update)

- Summary: Through hand-calculated analysis and comparison, I have found that the code performs bitpacking in left-justified orientation (where rightmost bits become

leftmost in byte representation). This leads to discrepancies between my hand calculations and simulation results becuase my hand calculations are right-justified.
    - Ex:
        - My calculated bit: 0x03
            - Bit representation: 0b00000011
        - Simulation bit: 96 (0x60)
            - Bit representation: 0b01100000
        - It performs a shift that is more accurate to real hardware, as it also doesn't change the sign of the data
    - This discrepancy isn't concerning apparently, as it doesn't affect the decompression accuracy or compression ratios
- Justification: I was investigating my bitpacking and bitplaning code becuase I suspected that maybe my compression analysis was compressing multiple of the same tensor, that somehow my transformations might have been undone, making them useless.
- Here is the information:
    - Original hand calculations:

Original (N, W, B) ex N=4, W=3, B=4

W=3, B=4

N=4 [[0001, 0111, 0100], [0011, 0010, 0101], [0110, 1000, 1110], [1011, 1111, 1001]]

1	7	4
3	2	5
6	8	14
11	15	9

horizontal (N, B, W)

N=4, B=4, W=3

[[000, 011, 010, 110], [000, 001, 110, 101], [011, 101, 101, 000], [111, 010, 110, 111]]

vertical (W, B, N)

W=3, B=4, N=4

[[0001, 0010, 0111, 1101], [0011, 1001, 1101, 1001], [0011, 1110, 0010, 0101]]

packed_n_caccross_w (W/8, B, N)

W=1, N=4, B=4

[[\x00\x00\x03\x07, \x03\x01\x05\x02, \x02\x06\x05\x06, \x06\x05\x00\x07]]

packed_n_caccross_w (N/8, B, W)

N=1, W=3, B=4

[[\x01\x03\x03, \x02\x09\x0E, \x0D\x09\x05]]

packed_b_caccross_w (N, B, W/8)

N=4, W=1, B=4

[[\x00, \x03, \x02, \x06], [\x08, \x01, \x06, \x05], [\x03, \x05, \x05, \x00], [\x07, \x02, \x06, \x07]]

packed_b_caccross_n (W, B, N/8)

W=3, N=1, B=4

[[\x01, \x02, \x07, \x0D], [\x03, \x09, \x0D, \x09], [\x03, \x0E, \x02, \x05]]

- Code output:
- Original Tensor:

```
Initial Tensor Data: torch.Size([4, 3])
tensor([[ 1,  7,  4],
        [ 3,  2,  5],
        [ 6,  8, 14],
        [11, 15,  9]], dtype=torch.uint8)
```

- Horizontal:

```
tensor([[[0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
         [0, 1, 1],
         [0, 1, 0],
         [1, 1, 0]],

        [[0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
         [0, 0, 1],
         [1, 1, 0],
         [1, 0, 1]],

        [[0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
         [0, 1, 1],
         [1, 0, 1],
         [1, 0, 1],
         [0, 0, 0]],

        [[0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
         [1, 1, 1],
         [0, 1, 0],
         [1, 1, 0],
         [1, 1, 1]]], dtype=torch.uint8)
```

- Packed_n:

```
Horizontal Packed_n: torch.Size([1, 8, 3])
tensor([[[  0,   0,   0],
         [  0,   0,   0],
         [  0,   0,   0],
         [  0,   0,   0],
         [ 16,  48,  48],
         [ 32, 144, 224],
         [112, 208,  32],
         [208, 144,  80]]], device='xpu:0', dtype=torch.uint8)
```

- Packed_b:

```
tensor([[[  0],
         [  0],
         [  0],
         [  0],
         [  0],
         [ 96],
         [ 64],
         [192]],

        [[  0],
         [  0],
         [  0],
         [  0],
         [  0],
         [ 32],
         [192],
         [160]],

        [[  0],
         [  0],
         [  0],
         [  0],
         [ 96],
         [160],
         [160],
         [  0]],

        [[  0],
         [  0],
         [  0],
         [  0],
         [224],
         [ 64],
         [192],
         [224]]], device='xpu:0', dtype=torch.uint8)
```

- Vertical:

```
Vertical Raw:
tensor([[[0, 0, 0, 0],
         [0, 0, 0, 0],
         [0, 0, 0, 0],
         [0, 0, 0, 0],
         [0, 0, 0, 1],
         [0, 0, 1, 0],
         [0, 1, 1, 1],
         [1, 1, 0, 1]],

        [[0, 0, 0, 0],
         [0, 0, 0, 0],
         [0, 0, 0, 0],
         [0, 0, 0, 0],
         [0, 0, 1, 1],
         [1, 0, 0, 1],
         [1, 1, 0, 1],
         [1, 0, 0, 1]],

        [[0, 0, 0, 0],
         [0, 0, 0, 0],
         [0, 0, 0, 0],
         [0, 0, 0, 0],
         [0, 0, 1, 1],
         [1, 1, 1, 0],
         [0, 0, 1, 0],
         [0, 1, 0, 1]]], dtype=torch.uint8)
```

- Packed_n:

```
Vertical Packed_n: torch.Size([1, 8, 4])
tensor([[[  0,   0,   0,   0],
         [  0,   0,   0,   0],
         [  0,   0,   0,   0],
         [  0,   0,   0,   0],
         [  0,   0,  96, 224],
         [ 96,  32, 160,  64],
         [ 64, 192, 160, 192],
         [192, 160,   0, 224]]], device='xpu:0', dtype=torch.uint8)
```

- Packed_b:

```
Vertical Packed_b: torch.Size([3, 8, 1])
tensor([[[  0],
         [  0],
         [  0],
         [  0],
         [ 16],
         [ 32],
         [112],
         [208]],

        [[  0],
         [  0],
         [  0],
         [  0],
         [ 48],
         [144],
         [208],
         [144]],

        [[  0],
         [  0],
         [  0],
         [  0],
         [ 48],
         [224],
         [ 32],
         [ 80]]], device='xpu:0', dtype=torch.uint8)
```

- To do later today:

- Place more results into documentation
- Add more metrics (avg bitplane compression overall, total bitplane compression overall, total bitplane compression overall (quant+scale), etc)
- Optional: add explanations for data
- Clean up your debugger simulation, you added some unneeded code (extra tensors and stage 5 extras)

3/11: Major Data/Results Update: LZ4+ZSTD Compression Across Multiple Sizes / Wikipedia DPR + Fineweb-Edu Embeddings: (Tim Update)

- Summary: Results from both Wikipedia and Fineweb-Edu will be placed here.
- Baseline data information (for consistency):
    - Wikipedia:
        - File used: train-00000-of-00157.parquet
        - Starting row: 0
        - LZ4 block size: 4 KB
        - ZSTD block size: undetermined for now
        - # of rows: 512, 768, 1024, 2048, 4096
    - Fineweb-Edu:
        - File used: new_CC-MAIN-2024-10-train-00019-of-00020.npy
        - Starting row: 0
        - LZ4 block size: 4 KB
        - ZSTD block size: undetermined for now
        - # of rows: 512, 768, 1024, 2048, 4096
- Now, I will show the most promising compression method first: vertical bitplane row-wise quantization

Wikipedia DPR:

Wikipedia DPR: # of Rows:	Quantized Values:	Scalars:

512

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.000	1.000	1.139	1.358	
6	6	1.000	1.000	1.139	1.358	
5	5	1.000	1.000	1.139	1.358	
4	4	0.959	1.000	1.139	1.358	
3	3	0.749	0.909	1.117	1.335	
2	2	0.687	0.833	1.046	1.273	
1	1	0.686	0.833	1.046	1.273	
0	0	0.686	0.833	1.046	1.273	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.094	1.24
6	1.094	1.24
5	1.094	1.24
4	1.094	1.24
3	1.094	1.24
2	1.094	1.24
1	1.094	1.24
0	1.094	1.24

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.141	1.359	1.141	1.359	
30	30	1.141	1.359	1.141	1.359	
29	29	1.141	1.359	1.141	1.359	
28	28	1.141	1.359	1.141	1.359	
27	27	1.141	1.359	1.141	1.359	
26	26	1.141	1.359	1.141	1.359	
25	25	1.141	1.359	1.141	1.359	
24	24	1.141	1.359	1.141	1.359	
23	23	1.141	1.359	1.141	1.359	
22	22	1.141	1.359	1.141	1.359	
21	21	1.141	1.359	1.141	1.359	
20	20	1.141	1.359	1.141	1.359	
19	19	1.141	1.359	1.141	1.359	
18	18	1.141	1.359	1.141	1.359	
17	17	1.141	1.359	1.141	1.359	
16	16	1.141	1.359	1.141	1.359	
15	15	1.141	1.359	1.141	1.359	
14	14	1.141	1.359	1.141	1.359	
13	13	1.141	1.359	1.141	1.359	
12	12	1.141	1.359	1.141	1.359	
11	11	1.000	1.312	1.000	1.312	
10	10	0.531	0.828	0.531	0.828	
9	9	0.531	0.828	0.531	0.828	
8	8	0.266	0.531	0.266	0.531	
7	7	0.266	0.531	0.266	0.531	
6	6	0.266	0.531	0.266	0.531	
5	5	0.266	0.531	0.266	0.531	
4	4	0.266	0.531	0.266	0.531	
3	3	0.266	0.531	0.266	0.531	
2	2	0.266	0.531	0.266	0.531	
1	1	0.266	0.531	0.266	0.531	
0	0	0.266	0.531	0.266	0.531	

	Spatial(ZSTD):	Spatial(LZ4):
31	80.0	192.0
30	80.0	192.0
29	80.0	192.0
28	80.0	192.0
27	80.0	192.0
26	80.0	192.0
25	80.0	192.0
24	80.0	192.0
23	80.0	192.0
22	80.0	192.0
21	80.0	192.0
20	80.0	192.0
19	80.0	192.0
18	80.0	192.0
17	80.0	192.0
16	80.0	192.0
15	80.0	192.0
14	80.0	192.0
13	80.0	192.0
12	80.0	192.0
11	80.0	192.0
10	80.0	192.0
9	80.0	192.0
8	80.0	192.0
7	80.0	192.0
6	80.0	192.0
5	80.0	192.0
4	80.0	192.0
3	80.0	192.0
2	80.0	192.0
1	80.0	192.0
0	80.0	192.0

768

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.000	1.000	1.093	1.238	
6	6	1.000	1.000	1.093	1.238	
5	5	1.000	1.000	1.093	1.238	
4	4	0.961	1.000	1.092	1.238	
3	3	0.751	0.923	1.044	1.210	
2	2	0.689	0.846	0.976	1.144	
1	1	0.688	0.845	0.976	1.143	
0	0	0.688	0.845	0.976	1.143	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.094	1.24
6	1.094	1.24
5	1.094	1.24
4	1.094	1.24
3	1.094	1.24
2	1.094	1.24
1	1.094	1.24
0	1.094	1.24

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.094	1.240	1.094	1.240	
30	30	1.094	1.240	1.094	1.240	
29	29	1.094	1.240	1.094	1.240	
28	28	1.094	1.240	1.094	1.240	
27	27	1.094	1.240	1.094	1.240	
26	26	1.094	1.240	1.094	1.240	
25	25	1.094	1.240	1.094	1.240	
24	24	1.094	1.240	1.094	1.240	
23	23	1.094	1.240	1.094	1.240	
22	22	1.094	1.240	1.094	1.240	
21	21	1.094	1.240	1.094	1.240	
20	20	1.094	1.240	1.094	1.240	
19	19	1.094	1.240	1.094	1.240	
18	18	1.094	1.240	1.094	1.240	
17	17	1.094	1.240	1.094	1.240	
16	16	1.094	1.240	1.094	1.240	
15	15	1.094	1.240	1.094	1.240	
14	14	1.094	1.240	1.094	1.240	
13	13	1.094	1.240	1.094	1.240	
12	12	1.094	1.240	1.094	1.240	
11	11	0.917	1.156	0.917	1.156	
10	10	0.510	0.781	0.510	0.781	
9	9	0.510	0.781	0.510	0.781	
8	8	0.177	0.354	0.177	0.354	
7	7	0.177	0.354	0.177	0.354	
6	6	0.177	0.354	0.177	0.354	
5	5	0.177	0.354	0.177	0.354	
4	4	0.177	0.354	0.177	0.354	
3	3	0.177	0.354	0.177	0.354	
2	2	0.177	0.354	0.177	0.354	
1	1	0.177	0.354	0.177	0.354	
0	0	0.177	0.354	0.177	0.354	

	Spatial(ZSTD):	Spatial(LZ4):
31	80.0	192.0
30	80.0	192.0
29	80.0	192.0
28	80.0	192.0
27	80.0	192.0
26	80.0	192.0
25	80.0	192.0
24	80.0	192.0
23	80.0	192.0
22	80.0	192.0
21	80.0	192.0
20	80.0	192.0
19	80.0	192.0
18	80.0	192.0
17	80.0	192.0
16	80.0	192.0
15	80.0	192.0
14	80.0	192.0
13	80.0	192.0
12	80.0	192.0
11	80.0	192.0
10	80.0	192.0
9	80.0	192.0
8	80.0	192.0
7	80.0	192.0
6	80.0	192.0
5	80.0	192.0
4	80.0	192.0
3	80.0	192.0
2	80.0	192.0
1	80.0	192.0
0	80.0	192.0

1024		

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.000	1.000	1.069	1.178	
6	6	1.000	1.000	1.069	1.178	
5	5	1.000	1.000	1.069	1.178	
4	4	0.962	1.000	1.069	1.178	
3	3	0.753	0.922	1.003	1.147	
2	2	0.688	0.841	0.934	1.078	
1	1	0.688	0.841	0.933	1.077	
0	0	0.688	0.841	0.933	1.077	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.094	1.24
6	1.094	1.24
5	1.094	1.24
4	1.094	1.24
3	1.094	1.24
2	1.094	1.24
1	1.094	1.24
0	1.094	1.24

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.070	1.180	1.070	1.180	
30	30	1.070	1.180	1.070	1.180	
29	29	1.070	1.180	1.070	1.180	
28	28	1.070	1.180	1.070	1.180	
27	27	1.070	1.180	1.070	1.180	
26	26	1.070	1.180	1.070	1.180	
25	25	1.070	1.180	1.070	1.180	
24	24	1.070	1.180	1.070	1.180	
23	23	1.070	1.180	1.070	1.180	
22	22	1.070	1.180	1.070	1.180	
21	21	1.070	1.180	1.070	1.180	
20	20	1.070	1.180	1.070	1.180	
19	19	1.070	1.180	1.070	1.180	
18	18	1.070	1.180	1.070	1.180	
17	17	1.070	1.180	1.070	1.180	
16	16	1.070	1.180	1.070	1.180	
15	15	1.070	1.180	1.070	1.180	
14	14	1.070	1.180	1.070	1.180	
13	13	1.070	1.180	1.070	1.180	
12	12	1.070	1.180	1.070	1.180	
11	11	0.852	1.086	0.852	1.086	
10	10	0.508	0.664	0.508	0.664	
9	9	0.508	0.664	0.508	0.664	
8	8	0.133	0.266	0.133	0.266	
7	7	0.133	0.266	0.133	0.266	
6	6	0.133	0.266	0.133	0.266	
5	5	0.133	0.266	0.133	0.266	
4	4	0.133	0.266	0.133	0.266	
3	3	0.133	0.266	0.133	0.266	
2	2	0.133	0.266	0.133	0.266	
1	1	0.133	0.266	0.133	0.266	
0	0	0.133	0.266	0.133	0.266	

	Spatial(ZSTD):	Spatial(LZ4):
31	80.0	192.0
30	80.0	192.0
29	80.0	192.0
28	80.0	192.0
27	80.0	192.0
26	80.0	192.0
25	80.0	192.0
24	80.0	192.0
23	80.0	192.0
22	80.0	192.0
21	80.0	192.0
20	80.0	192.0
19	80.0	192.0
18	80.0	192.0
17	80.0	192.0
16	80.0	192.0
15	80.0	192.0
14	80.0	192.0
13	80.0	192.0
12	80.0	192.0
11	80.0	192.0
10	80.0	192.0
9	80.0	192.0
8	80.0	192.0
7	80.0	192.0
6	80.0	192.0
5	80.0	192.0
4	80.0	192.0
3	80.0	192.0
2	80.0	192.0
1	80.0	192.0
0	80.0	192.0

2048

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.000	1.000	1.038	1.089	
6	6	1.000	1.000	1.038	1.089	
5	5	1.000	1.000	1.038	1.089	
4	4	0.956	1.000	1.037	1.088	
3	3	0.742	0.901	0.920	1.036	
2	2	0.677	0.820	0.851	0.958	
1	1	0.677	0.820	0.850	0.958	
0	0	0.677	0.820	0.850	0.958	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.094	1.24
6	1.094	1.24
5	1.094	1.24
4	1.094	1.24
3	1.094	1.24
2	1.094	1.24
1	1.094	1.24
0	1.094	1.24

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.039	1.090	1.039	1.090	
30	30	1.039	1.090	1.039	1.090	
29	29	1.039	1.090	1.039	1.090	
28	28	1.039	1.090	1.039	1.090	
27	27	1.039	1.090	1.039	1.090	
26	26	1.039	1.090	1.039	1.090	
25	25	1.039	1.090	1.039	1.090	
24	24	1.039	1.090	1.039	1.090	
23	23	1.039	1.090	1.039	1.090	
22	22	1.039	1.090	1.039	1.090	
21	21	1.039	1.090	1.039	1.090	
20	20	1.039	1.090	1.039	1.090	
19	19	1.039	1.090	1.039	1.090	
18	18	1.039	1.090	1.039	1.090	
17	17	1.039	1.090	1.039	1.090	
16	16	1.039	1.090	1.039	1.090	
15	15	1.039	1.090	1.039	1.090	
14	14	1.039	1.090	1.039	1.090	
13	13	1.039	1.090	1.039	1.090	
12	12	1.039	1.090	1.039	1.090	
11	11	0.711	0.961	0.711	0.961	
10	10	0.414	0.543	0.414	0.543	
9	9	0.414	0.543	0.414	0.543	
8	8	0.074	0.133	0.074	0.133	
7	7	0.074	0.133	0.074	0.133	
6	6	0.074	0.133	0.074	0.133	
5	5	0.074	0.133	0.074	0.133	
4	4	0.074	0.133	0.074	0.133	
3	3	0.074	0.133	0.074	0.133	
2	2	0.074	0.133	0.074	0.133	
1	1	0.074	0.133	0.074	0.133	
0	0	0.074	0.133	0.074	0.133	

	Spatial(ZSTD):	Spatial(LZ4):
31	80.0	192.0
30	80.0	192.0
29	80.0	192.0
28	80.0	192.0
27	80.0	192.0
26	80.0	192.0
25	80.0	192.0
24	80.0	192.0
23	80.0	192.0
22	80.0	192.0
21	80.0	192.0
20	80.0	192.0
19	80.0	192.0
18	80.0	192.0
17	80.0	192.0
16	80.0	192.0
15	80.0	192.0
14	80.0	192.0
13	80.0	192.0
12	80.0	192.0
11	80.0	192.0
10	80.0	192.0
9	80.0	192.0
8	80.0	192.0
7	80.0	192.0
6	80.0	192.0
5	80.0	192.0
4	80.0	192.0
3	80.0	192.0
2	80.0	192.0
1	80.0	192.0
0	80.0	192.0

4096

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.000	1.000	1.018	1.044	
6	6	1.000	1.000	1.018	1.044	
5	5	1.000	1.000	1.018	1.044	
4	4	0.956	1.000	1.018	1.044	
3	3	0.737	0.904	0.873	0.983	
2	2	0.673	0.822	0.805	0.905	
1	1	0.672	0.822	0.805	0.904	
0	0	0.672	0.822	0.805	0.904	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.094	1.24
6	1.094	1.24
5	1.094	1.24
4	1.094	1.24
3	1.094	1.24
2	1.094	1.24
1	1.094	1.24
0	1.094	1.24

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.020	1.045	1.020	1.045	
30	30	1.020	1.045	1.020	1.045	
29	29	1.020	1.045	1.020	1.045	
28	28	1.020	1.045	1.020	1.045	
27	27	1.020	1.045	1.020	1.045	
26	26	1.020	1.045	1.020	1.045	
25	25	1.020	1.045	1.020	1.045	
24	24	1.020	1.045	1.020	1.045	
23	23	1.020	1.045	1.020	1.045	
22	22	1.020	1.045	1.020	1.045	
21	21	1.020	1.045	1.020	1.045	
20	20	1.020	1.045	1.020	1.045	
19	19	1.020	1.045	1.020	1.045	
18	18	1.020	1.045	1.020	1.045	
17	17	1.020	1.045	1.020	1.045	
16	16	1.020	1.045	1.020	1.045	
15	15	1.020	1.045	1.020	1.045	
14	14	1.020	1.045	1.020	1.045	
13	13	1.020	1.045	1.020	1.045	
12	12	1.020	1.045	1.020	1.045	
11	11	0.639	0.850	0.639	0.850	
10	10	0.293	0.410	0.293	0.410	
9	9	0.287	0.410	0.287	0.410	
8	8	0.037	0.068	0.037	0.068	
7	7	0.037	0.068	0.037	0.068	
6	6	0.037	0.068	0.037	0.068	
5	5	0.037	0.068	0.037	0.068	
4	4	0.037	0.068	0.037	0.068	
3	3	0.037	0.068	0.037	0.068	
2	2	0.037	0.068	0.037	0.068	
1	1	0.037	0.068	0.037	0.068	
0	0	0.037	0.068	0.037	0.068	

	Spatial(ZSTD):	Spatial(LZ4):
31	80.0	192.0
30	80.0	192.0
29	80.0	192.0
28	80.0	192.0
27	80.0	192.0
26	80.0	192.0
25	80.0	192.0
24	80.0	192.0
23	80.0	192.0
22	80.0	192.0
21	80.0	192.0
20	80.0	192.0
19	80.0	192.0
18	80.0	192.0
17	80.0	192.0
16	80.0	192.0
15	80.0	192.0
14	80.0	192.0
13	80.0	192.0
12	80.0	192.0
11	80.0	192.0
10	80.0	192.0
9	80.0	192.0
8	80.0	192.0
7	80.0	192.0
6	80.0	192.0
5	80.0	192.0
4	80.0	192.0
3	80.0	192.0
2	80.0	192.0
1	80.0	192.0
0	80.0	192.0

Fineweb-Edu:

Fineweb-Edu: # of Rows:	Quantized Values:	Scalar Values:
512		

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.000	1.000	1.141	1.359	
6	6	1.000	1.000	1.141	1.359	
5	5	1.000	1.000	1.141	1.359	
4	4	1.000	1.000	1.141	1.359	
3	3	1.000	1.000	1.141	1.359	
2	2	1.000	1.000	1.140	1.359	
1	1	0.974	1.000	1.135	1.356	
0	0	0.937	0.983	1.125	1.349	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.07	1.18
6	1.07	1.18
5	1.07	1.18
4	1.07	1.18
3	1.07	1.18
2	1.07	1.18
1	1.07	1.18
0	1.07	1.18

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.141	1.359	1.141	1.359	
30	30	1.141	1.359	1.141	1.359	
29	29	1.141	1.359	1.141	1.359	
28	28	1.141	1.359	1.141	1.359	
27	27	1.141	1.359	1.141	1.359	
26	26	1.141	1.359	1.141	1.359	
25	25	1.141	1.359	1.141	1.359	
24	24	1.141	1.359	1.141	1.359	
23	23	1.141	1.359	1.141	1.359	
22	22	1.141	1.359	1.141	1.359	
21	21	1.141	1.359	1.141	1.359	
20	20	1.141	1.359	1.141	1.359	
19	19	1.141	1.359	1.141	1.359	
18	18	1.141	1.359	1.141	1.359	
17	17	1.141	1.359	1.141	1.359	
16	16	1.141	1.359	1.141	1.359	
15	15	1.141	1.359	1.141	1.359	
14	14	1.141	1.359	1.141	1.359	
13	13	1.141	1.359	1.141	1.359	
12	12	1.141	1.359	1.141	1.359	
11	11	1.141	1.359	1.141	1.359	
10	10	1.141	1.359	1.141	1.359	
9	9	1.141	1.359	1.141	1.359	
8	8	1.141	1.359	1.141	1.359	
7	7	0.266	0.531	0.266	0.531	
6	6	0.266	0.531	0.266	0.531	
5	5	0.266	0.531	0.266	0.531	
4	4	0.266	0.531	0.266	0.531	
3	3	0.266	0.531	0.266	0.531	
2	2	0.266	0.531	0.266	0.531	
1	1	0.266	0.531	0.266	0.531	
0	0	0.266	0.531	0.266	0.531	

768		

	Spatial(ZSTD):	Spatial(LZ4):
31	80.0	192.0
30	80.0	192.0
29	80.0	192.0
28	80.0	192.0
27	80.0	192.0
26	80.0	192.0
25	80.0	192.0
24	80.0	192.0
23	80.0	192.0
22	80.0	192.0
21	80.0	192.0
20	80.0	192.0
19	80.0	192.0
18	80.0	192.0
17	80.0	192.0
16	80.0	192.0
15	80.0	192.0
14	80.0	192.0
13	80.0	192.0
12	80.0	192.0
11	80.0	192.0
10	80.0	192.0
9	80.0	192.0
8	80.0	192.0
7	80.0	192.0
6	80.0	192.0
5	80.0	192.0
4	80.0	192.0
3	80.0	192.0
2	80.0	192.0
1	80.0	192.0
0	80.0	192.0

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.000	1.000	1.094	1.240	
6	6	1.000	1.000	1.094	1.240	
5	5	1.000	1.000	1.094	1.240	
4	4	1.000	1.000	1.094	1.240	
3	3	1.000	1.000	1.094	1.240	
2	2	1.000	1.000	1.093	1.240	
1	1	0.975	1.000	1.081	1.236	
0	0	0.937	0.988	1.059	1.228	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.07	1.18
6	1.07	1.18
5	1.07	1.18
4	1.07	1.18
3	1.07	1.18
2	1.07	1.18
1	1.07	1.18
0	1.07	1.18

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.094	1.240	1.094	1.240	
30	30	1.094	1.240	1.094	1.240	
29	29	1.094	1.240	1.094	1.240	
28	28	1.094	1.240	1.094	1.240	
27	27	1.094	1.240	1.094	1.240	
26	26	1.094	1.240	1.094	1.240	
25	25	1.094	1.240	1.094	1.240	
24	24	1.094	1.240	1.094	1.240	
23	23	1.094	1.240	1.094	1.240	
22	22	1.094	1.240	1.094	1.240	
21	21	1.094	1.240	1.094	1.240	
20	20	1.094	1.240	1.094	1.240	
19	19	1.094	1.240	1.094	1.240	
18	18	1.094	1.240	1.094	1.240	
17	17	1.094	1.240	1.094	1.240	
16	16	1.094	1.240	1.094	1.240	
15	15	1.094	1.240	1.094	1.240	
14	14	1.094	1.240	1.094	1.240	
13	13	1.094	1.240	1.094	1.240	
12	12	1.094	1.240	1.094	1.240	
11	11	1.094	1.240	1.094	1.240	
10	10	1.094	1.240	1.094	1.240	
9	9	1.094	1.240	1.094	1.240	
8	8	1.094	1.240	1.094	1.240	
7	7	0.177	0.354	0.177	0.354	
6	6	0.177	0.354	0.177	0.354	
5	5	0.177	0.354	0.177	0.354	
4	4	0.177	0.354	0.177	0.354	
3	3	0.177	0.354	0.177	0.354	
2	2	0.177	0.354	0.177	0.354	
1	1	0.177	0.354	0.177	0.354	
0	0	0.177	0.354	0.177	0.354	

	Spatial(ZSTD):	Spatial(LZ4):
31	80.0	192.0
30	80.0	192.0
29	80.0	192.0
28	80.0	192.0
27	80.0	192.0
26	80.0	192.0
25	80.0	192.0
24	80.0	192.0
23	80.0	192.0
22	80.0	192.0
21	80.0	192.0
20	80.0	192.0
19	80.0	192.0
18	80.0	192.0
17	80.0	192.0
16	80.0	192.0
15	80.0	192.0
14	80.0	192.0
13	80.0	192.0
12	80.0	192.0
11	80.0	192.0
10	80.0	192.0
9	80.0	192.0
8	80.0	192.0
7	80.0	192.0
6	80.0	192.0
5	80.0	192.0
4	80.0	192.0
3	80.0	192.0
2	80.0	192.0
1	80.0	192.0
0	80.0	192.0

1024

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.000	1.000	1.070	1.180	
6	6	1.000	1.000	1.070	1.180	
5	5	1.000	1.000	1.070	1.180	
4	4	1.000	1.000	1.070	1.180	
3	3	1.000	1.000	1.070	1.180	
2	2	1.000	1.000	1.070	1.180	
1	1	0.975	1.000	1.052	1.176	
0	0	0.936	0.987	1.025	1.167	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.07	1.18
6	1.07	1.18
5	1.07	1.18
4	1.07	1.18
3	1.07	1.18
2	1.07	1.18
1	1.07	1.18
0	1.07	1.18

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.070	1.180	1.070	1.180	
30	30	1.070	1.180	1.070	1.180	
29	29	1.070	1.180	1.070	1.180	
28	28	1.070	1.180	1.070	1.180	
27	27	1.070	1.180	1.070	1.180	
26	26	1.070	1.180	1.070	1.180	
25	25	1.070	1.180	1.070	1.180	
24	24	1.070	1.180	1.070	1.180	
23	23	1.070	1.180	1.070	1.180	
22	22	1.070	1.180	1.070	1.180	
21	21	1.070	1.180	1.070	1.180	
20	20	1.070	1.180	1.070	1.180	
19	19	1.070	1.180	1.070	1.180	
18	18	1.070	1.180	1.070	1.180	
17	17	1.070	1.180	1.070	1.180	
16	16	1.070	1.180	1.070	1.180	
15	15	1.070	1.180	1.070	1.180	
14	14	1.070	1.180	1.070	1.180	
13	13	1.070	1.180	1.070	1.180	
12	12	1.070	1.180	1.070	1.180	
11	11	1.070	1.180	1.070	1.180	
10	10	1.070	1.180	1.070	1.180	
9	9	1.070	1.180	1.070	1.180	
8	8	1.070	1.180	1.070	1.180	
7	7	0.133	0.266	0.133	0.266	
6	6	0.133	0.266	0.133	0.266	
5	5	0.133	0.266	0.133	0.266	
4	4	0.133	0.266	0.133	0.266	
3	3	0.133	0.266	0.133	0.266	
2	2	0.133	0.266	0.133	0.266	
1	1	0.133	0.266	0.133	0.266	
0	0	0.133	0.266	0.133	0.266	

	Spatial(ZSTD):	Spatial(LZ4):
31	80.0	192.0
30	80.0	192.0
29	80.0	192.0
28	80.0	192.0
27	80.0	192.0
26	80.0	192.0
25	80.0	192.0
24	80.0	192.0
23	80.0	192.0
22	80.0	192.0
21	80.0	192.0
20	80.0	192.0
19	80.0	192.0
18	80.0	192.0
17	80.0	192.0
16	80.0	192.0
15	80.0	192.0
14	80.0	192.0
13	80.0	192.0
12	80.0	192.0
11	80.0	192.0
10	80.0	192.0
9	80.0	192.0
8	80.0	192.0
7	80.0	192.0
6	80.0	192.0
5	80.0	192.0
4	80.0	192.0
3	80.0	192.0
2	80.0	192.0
1	80.0	192.0
0	80.0	192.0

2048

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.000	1.000	1.039	1.090	
6	6	1.000	1.000	1.039	1.090	
5	5	1.000	1.000	1.039	1.090	
4	4	1.000	1.000	1.039	1.090	
3	3	1.000	1.000	1.039	1.090	
2	2	1.000	1.000	1.038	1.090	
1	1	0.973	1.000	1.011	1.085	
0	0	0.935	0.985	0.977	1.074	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.07	1.18
6	1.07	1.18
5	1.07	1.18
4	1.07	1.18
3	1.07	1.18
2	1.07	1.18
1	1.07	1.18
0	1.07	1.18

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.039	1.090	1.039	1.090	
30	30	1.039	1.090	1.039	1.090	
29	29	1.039	1.090	1.039	1.090	
28	28	1.039	1.090	1.039	1.090	
27	27	1.039	1.090	1.039	1.090	
26	26	1.039	1.090	1.039	1.090	
25	25	1.039	1.090	1.039	1.090	
24	24	1.039	1.090	1.039	1.090	
23	23	1.039	1.090	1.039	1.090	
22	22	1.039	1.090	1.039	1.090	
21	21	1.039	1.090	1.039	1.090	
20	20	1.039	1.090	1.039	1.090	
19	19	1.039	1.090	1.039	1.090	
18	18	1.039	1.090	1.039	1.090	
17	17	1.039	1.090	1.039	1.090	
16	16	1.039	1.090	1.039	1.090	
15	15	1.039	1.090	1.039	1.090	
14	14	1.039	1.090	1.039	1.090	
13	13	1.039	1.090	1.039	1.090	
12	12	1.039	1.090	1.039	1.090	
11	11	1.039	1.090	1.039	1.090	
10	10	1.039	1.090	1.039	1.090	
9	9	1.039	1.090	1.039	1.090	
8	8	1.039	1.090	1.039	1.090	
7	7	0.074	0.133	0.074	0.133	
6	6	0.074	0.133	0.074	0.133	
5	5	0.074	0.133	0.074	0.133	
4	4	0.074	0.133	0.074	0.133	
3	3	0.074	0.133	0.074	0.133	
2	2	0.074	0.133	0.074	0.133	
1	1	0.074	0.133	0.074	0.133	
0	0	0.074	0.133	0.074	0.133	

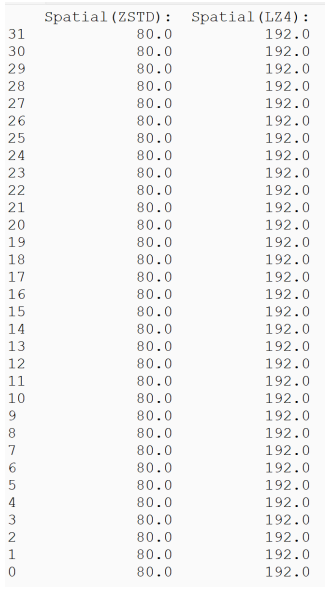
4096		

	Spatial(ZSTD):	Spatial(LZ4):
31	80.0	192.0
30	80.0	192.0
29	80.0	192.0
28	80.0	192.0
27	80.0	192.0
26	80.0	192.0
25	80.0	192.0
24	80.0	192.0
23	80.0	192.0
22	80.0	192.0
21	80.0	192.0
20	80.0	192.0
19	80.0	192.0
18	80.0	192.0
17	80.0	192.0
16	80.0	192.0
15	80.0	192.0
14	80.0	192.0
13	80.0	192.0
12	80.0	192.0
11	80.0	192.0
10	80.0	192.0
9	80.0	192.0
8	80.0	192.0
7	80.0	192.0
6	80.0	192.0
5	80.0	192.0
4	80.0	192.0
3	80.0	192.0
2	80.0	192.0
1	80.0	192.0
0	80.0	192.0

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
7	7	1.000	1.000	1.020	1.045	
6	6	1.000	1.000	1.020	1.045	
5	5	1.000	1.000	1.020	1.045	
4	4	1.000	1.000	1.020	1.045	
3	3	1.000	1.000	1.020	1.045	
2	2	1.000	1.000	1.018	1.045	
1	1	0.976	0.999	0.979	1.039	
0	0	0.935	0.983	0.937	1.027	

	Spatial(ZSTD):	Spatial(LZ4):
7	1.07	1.18
6	1.07	1.18
5	1.07	1.18
4	1.07	1.18
3	1.07	1.18
2	1.07	1.18
1	1.07	1.18
0	1.07	1.18

	Bitplane	Global(ZSTD):	Global(LZ4):	Temporal(ZSTD):	Temporal(LZ4):	\
31	31	1.020	1.045	1.020	1.045	
30	30	1.020	1.045	1.020	1.045	
29	29	1.020	1.045	1.020	1.045	
28	28	1.020	1.045	1.020	1.045	
27	27	1.020	1.045	1.020	1.045	
26	26	1.020	1.045	1.020	1.045	
25	25	1.020	1.045	1.020	1.045	
24	24	1.020	1.045	1.020	1.045	
23	23	1.020	1.045	1.020	1.045	
22	22	1.020	1.045	1.020	1.045	
21	21	1.020	1.045	1.020	1.045	
20	20	1.020	1.045	1.020	1.045	
19	19	1.020	1.045	1.020	1.045	
18	18	1.020	1.045	1.020	1.045	
17	17	1.020	1.045	1.020	1.045	
16	16	1.020	1.045	1.020	1.045	
15	15	1.020	1.045	1.020	1.045	
14	14	1.020	1.045	1.020	1.045	
13	13	1.020	1.045	1.020	1.045	
12	12	1.020	1.045	1.020	1.045	
11	11	1.020	1.045	1.020	1.045	
10	10	1.020	1.045	1.020	1.045	
9	9	1.020	1.045	1.020	1.045	
8	8	1.020	1.045	1.020	1.045	
7	7	0.037	0.068	0.037	0.068	
6	6	0.037	0.068	0.037	0.068	
5	5	0.037	0.068	0.037	0.068	
4	4	0.037	0.068	0.037	0.068	
3	3	0.037	0.068	0.037	0.068	
2	2	0.037	0.068	0.037	0.068	
1	1	0.037	0.068	0.037	0.068	
0	0	0.037	0.068	0.037	0.068	

	Spatial(ZSTD):	Spatial(LZ4):
31	80.0	192.0
30	80.0	192.0
29	80.0	192.0
28	80.0	192.0
27	80.0	192.0
26	80.0	192.0
25	80.0	192.0
24	80.0	192.0
23	80.0	192.0
22	80.0	192.0
21	80.0	192.0
20	80.0	192.0
19	80.0	192.0
18	80.0	192.0
17	80.0	192.0
16	80.0	192.0
15	80.0	192.0
14	80.0	192.0
13	80.0	192.0
12	80.0	192.0
11	80.0	192.0
10	80.0	192.0
9	80.0	192.0
8	80.0	192.0
7	80.0	192.0
6	80.0	192.0
5	80.0	192.0
4	80.0	192.0
3	80.0	192.0
2	80.0	192.0
1	80.0	192.0
0	80.0	192.0

Some general observations/takeaways:

- Wikipedia DPR is notably more compressible than Fineweb-Edu
- The best compression process is:
    - [Stage 1] Quantization: row-wise (1 scalar per vector embedding/row)
    - [Stage 2] Bitplaning: vertical (across all tokens/rows)
    - [Stage 3] Bitpacking method: n-wise (across rows/vector embeddings)
    - [Stage 4] Granularity: Global (each bitplane is individually compressed)
    - [Stage 5] Compression Algorithm: ZSTD (Highest Compression Ratios Overall)

Some observations:

- The compression ratios are remarkably consistent across different numbers of rows. The compression ratio only slightly worsens as the number of rows keeps doubling. This means the method is highly scalable to large data applications.
- Spatial Granularity compression is not feasible for vector embeddings. The header tax overhead from LZ4 and ZSTD is too high for this highly granular technique. (Compression ratios of 80 and 192)
- Spatial Granularity compression is generally terrible
- Global granularity compression is generally the best one, which is not surprising as it has the most data for LZ4 and ZSTD to find patterns, along with having less overhead than the other granular methods.
- Temporal is the middle-child, as it performs worse than Global, but performs a lot better than Spatial. Temporal has more overhead due to having higher granularity, but it scales better as the number of rows increases.

To Note:

- In real-world scenarios, negative compression is just “noise”, so the compressor could be smarter and just not compress it if it notices negative compression ratios. So we can just compress the actually compressible parts to maximize our overall compression ratios.
    - I may work on a setting that allows for this in the future.

To do next:

- Add more metrics: (direct file compression, avg compression Global Temporal Spatial, exact compression Global Temporal Spatial, exact compression for Quant + Scalar for Global Temporal Spatial)
- Add block sizes for ZSTD

3/13: Major Update: Space Saving Metrics / Wikipedia DPR: (Tim Update)

- Summary: There are new space saving metrics. I am worried that the % reduction in memory is too small. The quantization does most of the memory reduction, but the bitplaning ZSTD+LZ4 seems to do very little. I will try much larger sizes for this data collection.
- Data used:
    - Dataset: Wikipedia DPR
    - File: train-00000-of-00157.parquet
    - Starting row: 0
    - # of rows: 4096, 8192, 16384, 32768
    - Note: I will only show the most promising results, which is Global vertical, row-wise quantization+scaling
    - Note: I will be using the new No Negatives option, which replaces negative compression ratios with 1.0s (no compression), which improves the compression ratios
- Data Collected:

Overall:

# of Rows:	Image:	% Saved From Only Quant +Scale	% Overall Savings from Compression: [ZSTD/LZ4]
4096		74.87	3.693/2.009

8192		74.87	3.698/1.962
16384		74.87	3.747/1.992
32768		74.87	3.829/1.982
65536 (out of curiousity)		74.87	3.829/2.000
131072 (out of curiosity)	<Ran out of VRAM, hit the 16GB limit><Upped VRAM limit to 25.2GB> <Still fails with 25.2GB of VRAM, would need beefier machine for this>		
Max row count for file ()	<Ran out of VRAM, hit the 16GB limit><Upped VRAM limit to 25.2GB> <Still fails with 25.2GB of VRAM, would need a beefier machine for this>		

```
                                Global(ZSTD):  Global(LZ4):
vs Original Data Size:                78.568        76.832
vs Quantized+Scalar:                  14.718         7.808
```

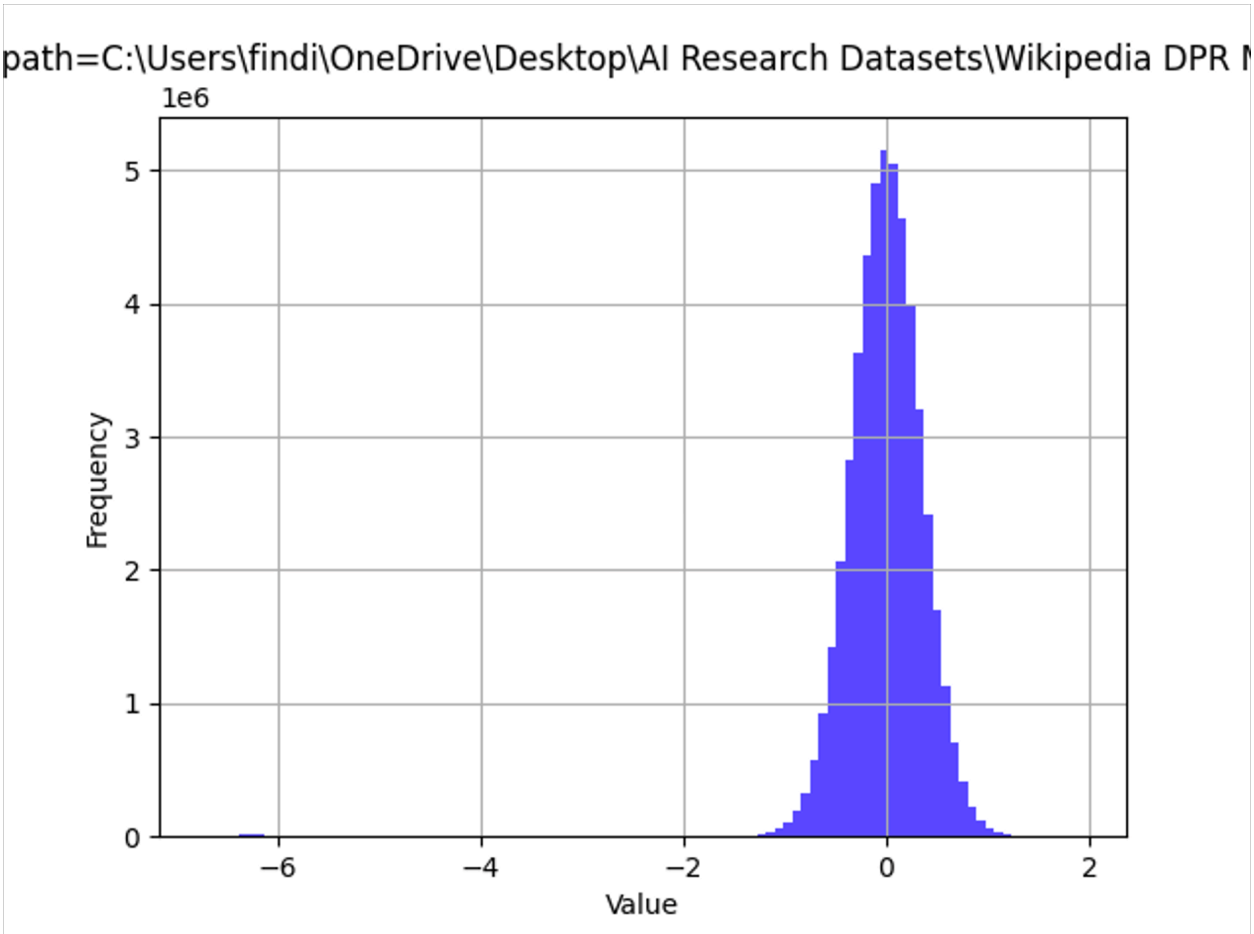
```
                                Global(ZSTD):  Global(LZ4):
vs Original Data Size:                78.617        76.862
vs Quantized+Scalar:                  14.910         7.929
```

```
                                Global(ZSTD):  Global(LZ4):
vs Original Data Size:                78.699        76.852
vs Quantized+Scalar:                  15.236         7.887
```

```
                                Global(ZSTD):  Global(LZ4):
vs Original Data Size:                78.699        76.870
vs Quantized+Scalar:                  15.236         7.958
```

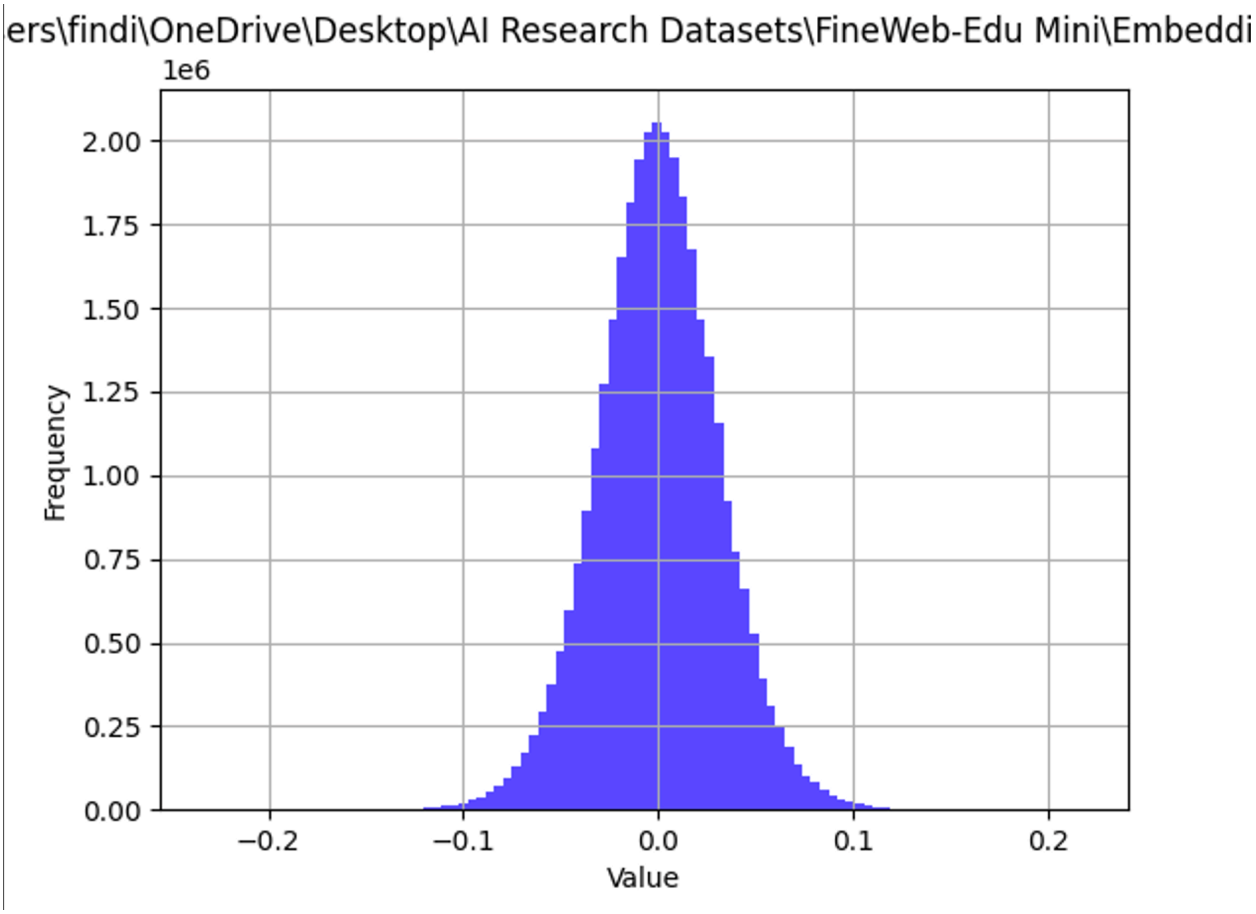
- Some Observations:
  - The space saved actually scales pretty well with increasing # of rows. The amount saved only slightly increases with # of rows for ZSTD, and stays around the same for LZ4.
  - ZSTD saves nearly double the % compared to LZ4
  - The quantization is the reason for the vast majority of % of space saved, while the bitplane compression only weakly improves the compressibility.
- Performance note: Larger tensors beyond 65536 rows may need more than 16GB of VRAM. Due to fragmentation, the memory requirements may exponentially increase. This is even after multiple memory optimizations in the code.

Also, here is a histogram of the data for Wikipedia DPR: (65536 rows)



- Observation: Range is between -1 and 1

Here is the histogram for FineWeb-Edu (.npy):



- Observation: the range is in between -0.1 and 0.1

- Summary of Progress: New generalized metrics can now be collected, showing the % memory saved via compression. Here are the currently working ones:
    - Metric: Bitplane Compressed Quant+Scale vs Original Prequantized Uncompressed
    - Metric: Bitplane Compressed Quant+Scale vs Uncompressed Bitplane Quant+Scale
    - Extra Metric: Uncompressed Quantized+Scalar vs Uncompressed Original Prequantized
- What I still need to implement:
    - Metric: Bitplane Compressed Quant+Scale vs Direct Compressed Original (No quantization)
    - Metric: Bitplane Compressed Quant+Scale vs Compressed Non-Bitplane Quant+Scale
- Accomplished in late beta B0.8.2:
    - Negative Ratio Toggle Option: allows one to see results with negative compressions replaced with 1.0x

        - Custom block sizes for ZSTD (results in worse compression ratios)
        - New Metrics: Space saving metrics
        - New Histogram: Range of Vector Embedding Input data
    - To do for late beta B0.9.0:
        - Implement the 2 metrics:
            - Metric: Bitplane Compressed Quant+Scale vs Direct Compressed Original (No quantization)
            - Metric: Bitplane Compressed Quant+Scale vs Compressed Non-Bitplane Quant+Scale
        - Implement Direct Compression on the block file
        - Test on more datasets
        - Test on much larger row counts: 4096+ (multiples of 4096)

3/15: Major Update: New Metrics Findings + Direct Compression / Wikipedia DPR: (Tim Update)

- Summary: Through preliminary results with new metrics, it seems that quantization drastically reduces the compression effectiveness of bitplane disaggregation to the point where it performs worse than compression of quantization without bitplaning. If quantization isn't involved, then bitplaning+compression notably improves space savings compared to direct compression (no bitplaning).
- Data used:
    - Dataset: Wikipedia DPR
    - File: train-00000-of-00157.parquet
    - Starting row: 0
    - Number of rows: 4096, 8192, 16384, 32768
    - Block size: 4096 (4kb)
- Data Collection:

Number of rows:	Data:
4096	Space Saving Metrics: Vertical_Row (Quant+Scale): Global(ZSTD): Global(LZ4): vs Original Data Size: 78.563 76.879 vs Quantized+Scalar: 14.696 7.994 vs Direct Compression: 77.129 76.879 vs Quant+Scale Compressed (No BP): -36.194 7.995

8192	Space Saving Metrics: Vertical_Row (Quant+Scale): Global(ZSTD): Global(LZ4): vs Original Data Size: 78.568 76.832 vs Quantized+Scalar: 14.718 7.808 vs Direct Compression: 77.135 76.832 vs Quant+Scale Compressed (No BP): -36.017 7.808
16384	Space Saving Metrics: Vertical_Row (Quant+Scale): Global(ZSTD): Global(LZ4): vs Original Data Size: 78.617 76.862 vs Quantized+Scalar: 14.910 7.929 vs Direct Compression: 77.186 76.862 vs Quant+Scale Compressed (No BP): -35.762 7.930
32768	<Ran out of vram, removing the direct compression comparison> Space Saving Metrics: Vertical_Row (Quant+Scale): Global(ZSTD): Global(LZ4): vs Original Data Size: 78.699 76.852 vs Quantized+Scalar: 15.236 7.887 vs Direct Compression: 0.000 0.000 vs Quant+Scale Compressed (No BP): -35.076 7.887

Some observations:

- When comparing quant+scale compression, bitplaning performs worse than no bitplaning (~35-36% higher memory footprint compared to quantized compression without bitplaning). This is something that disproves our initial hypothesis.
    - But when comparing non-quantized data for compression, bitplaning performs noticeably better than without bitplaning. (~10% more space saved)
        - This suggests that bitplaning is more effective without quantization because quantization removes redundancy from the bitplanes.
- Bitplaning+quantization+compression performs pretty well vs bitplaning+quantization precompressed, with ~15% space savings on average
- ZSTD performs about 2x as well in compressibility compared to LZ4

Now for direct compression results:

Number of	Data:

rows:	
4096	Space Saving Metric: Direct Raw Compression (No bitplaning) vs Original Raw: Direct Compression(ZSTD) Ratio & % Memory Saving: 0.937 , 6.27% Direct Compression(LZ4) Ratio & % Memory Saving: 1.0 , -0.0%
8192	Space Saving Metric: Direct Raw Compression (No bitplaning) vs Original Raw: Direct Compression(ZSTD) Ratio & % Memory Saving: 0.937 , 6.269% Direct Compression(LZ4) Ratio & % Memory Saving: 1.0 , -0.0%
16384	Space Saving Metric: Direct Raw Compression (No bitplaning) vs Original Raw: Direct Compression(ZSTD) Ratio & % Memory Saving: 0.937 , 6.27% Direct Compression(LZ4) Ratio & % Memory Saving: 1.0 , -0.0%
32768	<Ran out of VRAM>

Some Observations:

- ZSTD Direct compression consistently saves ~6.3% memory compared to original data.
- LZ4 Direct compression cannot compress the tensor directly at all (~0% space savings)

Now for bitplane compression vs original and vs direct compression:

Number of Rows:	Data:
4096	Space Saving Metrics: Vertical (Raw): Global(ZSTD): Global(LZ4): Temporal(ZSTD): Temporal(LZ4): Spatial(ZSTD): Spatial(LZ4): vs Original Data Size: 16.514 15.28 14.84 13.583 10.338 7.53 Global(ZSTD): Global(LZ4): Temporal(ZSTD): Temporal(LZ4): Spatial(ZSTD): Spatial(LZ4): vs Direct [ZSTD,LZ4]: 10.929 15.281 9.144 13.583 4.34 7.53 Space Saving Metrics: Horizontal (Raw): Global(ZSTD): Global(LZ4): Temporal(ZSTD): Temporal(LZ4): Spatial(ZSTD): Spatial(LZ4): vs Original Data Size: 15.261 14.345 14.84 13.583 10.338 7.53 Global(ZSTD): Global(LZ4): Temporal(ZSTD): Temporal(LZ4): Spatial(ZSTD): Spatial(LZ4): vs Direct [ZSTD,LZ4]: 9.593 14.345 9.144 13.583 4.34 7.53

8192	Space Saving Metrics: Vertical (Raw): Global(ZSTD): Global(LZ4): Temporal(ZSTD): Temporal(LZ4): Spatial(ZSTD): Spatial(LZ4): vs Original Data Size: 16.523 15.272 15.677 14.291 10.342 7.535 Global(ZSTD): Global(LZ4): Temporal(ZSTD): Temporal(LZ4): Spatial(ZSTD): Spatial(LZ4): vs Direct [ZSTD,LZ4]: 10.94 15.272 10.037 14.291 4.345 7.535 Space Saving Metrics: Horizontal (Raw): Global(ZSTD): Global(LZ4): Temporal(ZSTD): Temporal(LZ4): Spatial(ZSTD): Spatial(LZ4): vs Original Data Size: 15.268 14.352 15.677 14.291 10.342 7.535 Global(ZSTD): Global(LZ4): Temporal(ZSTD): Temporal(LZ4): Spatial(ZSTD): Spatial(LZ4): vs Direct [ZSTD,LZ4]: 9.601 14.352 10.037 14.291 4.345 7.535
16384	Space Saving Metrics: Vertical (Raw): Global(ZSTD): Global(LZ4): Temporal(ZSTD): Temporal(LZ4): Spatial(ZSTD): Spatial(LZ4): vs Original Data Size: 16.542 15.285 16.237 14.686 10.345 7.54 Global(ZSTD): Global(LZ4): Temporal(ZSTD): Temporal(LZ4): Spatial(ZSTD): Spatial(LZ4): vs Direct [ZSTD,LZ4]: 10.959 15.285 10.633 14.686 4.348 7.54 Space Saving Metrics: Horizontal (Raw): Global(ZSTD): Global(LZ4): Temporal(ZSTD): Temporal(LZ4): Spatial(ZSTD): Spatial(LZ4): vs Original Data Size: 15.269 14.355 16.237 14.686 10.345 7.54 Global(ZSTD): Global(LZ4): Temporal(ZSTD): Temporal(LZ4): Spatial(ZSTD): Spatial(LZ4): vs Direct [ZSTD,LZ4]: 9.601 14.355 10.633 14.686 4.348 7.54
32768	<Ran out of VRAM>

Some observations:

- Bitplaning vs direct compression: including bitplaning in the compression process yields ~10-15% improvement compared to direct compression (without bitplaning). (ZSTD: ~10%, LZ4: ~15%)
- Temporal granularity shows similar levels of compression improvements
- Spatial granularity has notably worse improvements compared to Global and Temporal. Most likely due to the header overhead that LZ4 and ZSTD incurs.

Conclusion:

- Based on the data gathered today, it seems bitplaning works better on tensors that aren't quantized, as they have higher bit redundancy.

Changes in B0.9.0:

- Added new display settings: PRINT_EXTRA_SPACE_SAVING_METRICS - Controls whether to show optional metrics, can improve performance when disabled
- Added 2 new metrics for Quant+Scale:
    - (% space saving) vs direct compression
    - (% space saving) vs direct compression quant+scale (no bitplaning)
- Added 2 metrics for Raw (bitplane+compression, no quantization):
    - (% space saving) vs original size
    - (% space saving) vs direct compression
- Added Direct Compression (+2 metrics: compression ratio, % savings (for each lz4 and zstd))
- Added Direct Quantization Compression (optional toggle in settings)

To do next: (B0.9.8)

- Add block row randomization support (random starting row #)(+settings)
- Test on more types of files (+new datasets support)(add more file support than just .parquet and .npy)

Future roadmap:

- (B0.9.9)
- Test for Nvidia CUDA support
- Test for AMD ROCm support
- Optionally test CPU fallback support
- (R1.0.0 - Full Release)
- Clean up code, comments, and documentation
- Take new documentation in new document. Make as clean and understandable as possible

3/16: Simulation Rewrite: Slimming down and focusing on Horizontal Bitplaning / Custom Dataset (Minecraft Ending Poem): (Tim Update)

- Summary of progress: I quickly drafted up a rewrite using modified versions of my code from a previous simulation. It now has these new changes:
    - Automatic dataset normalization
    - Added .txt support (with built-in vector embedding computational model)
    - Added entropy space savings
- Summary of results: Generally, very weak compressibility. Based on all the datasets I have available, I have gotten 0% space savings. Based on my custom dataset, I was getting around 0.02-2.76% space savings with theoretical Entropy. With that same dataset, I was getting around 3-21% space savings with RLE, but it would have 5% space savings on average.
- Custom dataset:
    - Source: minecraft ending poem (text)
    - Dim: 4096
    - Starting row: 0
    - Number of rows: 1024
    - Block size: 512 (due to the bitpacking, doing 4096 adds padding and overhead)
- Results:
    - RLE Compression:

```
DEBUG: Quant-Horizontal Overall results (RLE): torch.Size([1024, 8])
tensor([[0.7939, 0.7939, 0.7910,  ..., 0.7949, 0.8555, 0.9814],
        [0.9521, 0.9531, 0.9531,  ..., 0.9858, 1.0000, 1.0000],
        [0.9453, 0.9453, 0.9473,  ..., 0.9570, 0.9912, 1.0000],
        ...,
        [0.9224, 0.9233, 0.9204,  ..., 0.9263, 0.9756, 1.0000],
        [0.9058, 0.9058, 0.9087,  ..., 0.9136, 0.9780, 1.0000],
        [0.9419, 0.9419, 0.9419,  ..., 0.9565, 0.9932, 0.9722]],
       device='xpu:0')

Horizontal Per Bitplane:
tensor([0.9587, 0.9585, 0.9586, 0.9585, 0.9604, 0.9719, 0.9928, 0.9939],
       device='xpu:0')

Horizontal Per Vector Embedding:
tensor([0.8246, 0.9693, 0.9611,  ..., 0.9391, 0.9276, 0.9537], device='xpu:0')


DEBUG: Overall Average RLE Compression Ratio: tensor(0.9691, device='xpu:0')
```

- Per bitplane compression (row-wise, LZ4+ZSTD):

	Bitplane	Spatial(ZSTD):	Spatial(LZ4):
7	7	1.0	1.0
6	6	1.0	1.0
5	5	1.0	1.0
4	4	1.0	1.0
3	3	1.0	1.0
2	2	1.0	1.0
1	1	1.0	1.0
0	0	1.0	1.0

- Per row compression (LZ4+ZSTD):

	Vector Embedding	Bit Plane Ratios (ZSTD):	Bit Plane Ratios (LZ4):
1023	1023	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1022	1022	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1021	1021	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1020	1020	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1019	1019	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
...	...	...	...
4	4	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
3	3	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
2	2	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1	1	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
0	0	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]

- Entropy:

```
--- BITPLANE ENTROPY CHECK (Shape: 1024x8x4096) ---
Plane      | Mean (Bias)     | Status
-----------------------------------------
Bitplane 0  | 0.4113          | Partial Structure
Bitplane 1  | 0.4115          | Partial Structure
Bitplane 2  | 0.4120          | Partial Structure
Bitplane 3  | 0.4140          | Partial Structure
Bitplane 4  | 0.4185          | Partial Structure
Bitplane 5  | 0.4374          | Partial Structure
Bitplane 6  | 0.4872          | Random (Incompressible)
Bitplane 7  | 0.4996          | Random (Incompressible)
```

- Savings Entropy:

```
Bitplane 0 | Avg Entropy: 0.9724 | Max Potential Savings: 2.76%
Bitplane 1 | Avg Entropy: 0.9724 | Max Potential Savings: 2.76%
Bitplane 2 | Avg Entropy: 0.9725 | Max Potential Savings: 2.75%
Bitplane 3 | Avg Entropy: 0.9727 | Max Potential Savings: 2.73%
Bitplane 4 | Avg Entropy: 0.9741 | Max Potential Savings: 2.59%
Bitplane 5 | Avg Entropy: 0.9818 | Max Potential Savings: 1.82%
Bitplane 6 | Avg Entropy: 0.9983 | Max Potential Savings: 0.17%
Bitplane 7 | Avg Entropy: 0.9998 | Max Potential Savings: 0.02%
```

To do next time:

- I need to make it closer to the TRACE architecture
  - Use Delta Transformation on the Quantized values to improve redundancy (add the delta exponent size to the scalar size or quant size)
  - Chunk in blocks after packing, concatenate those chunks into BLOCK_SIZE (not doing the block sizes within the compressor itself)
  - Maybe only analyze MSBs (only do sign bit and exponent bits for floating point)

3/18: Major Update: Bitplane Concatenation + Compression Results / Custom Qwen3 Tokenized Dataset (Minecraft ending poem) + Wikipedia DPR + Fineweb-edu: (Tim Update)

- Summary: For each row, concatenating the packed bitplanes, then compressing it by block size led to massive improvements in compressibility. % Savings increased from 0% to 30-40% on average. My Qwen3 custom dataset led to a 40-43% overall memory reduction, while wikipedia dpr led to a 30% overall memory reduction. Fineweb-Edu seems much harder to compress than the others, with a 1-2% overall memory reduction. Delta transformations didn't help, at best, it led to a 1% improvement in compressibility.
- Changes made:

        - For each row, bitplanes are concatenated together, then chunked by block size and compressed. This gives the compressor much larger context windows, which allows it to compress further
        - Delta transformation was added
- Data information:
    - Qwen 3 Embedding Model Custom Dataset: Source: Minecraft Ending Poem
        - File used: MinecraftEndPoem.txt (converted into vector embedding data through embedding model)
        - Embedding dimensions: 4096
        - Characteristics: Sparse (very low entropy)
        - Block size: 4kB
    - Wikipedia DPR:
        - File used: train-00000-of-00157.parquet
        - Embedding dimensions: 768
        - Characteristics: Relatively Sparse (low entropy)
        - Block size: 768B
    - Fineweb-edu:
        - File used: new_CC-MAIN-2024-10-train-00019-of-00020.npy
        - Embedding dimensions: 1024
        - Characteristics: Very Dense (very high entropy)
        - Block size: 1KB
    - Overall information:
        - Starting row: 0
        - Number of rows: 1024
        - Quantization: INT8
        - Note: I have different block sizes for each dataset so I don't skew the results. Having a block size that is larger than the row size itself adds padding, which inflates compression results.
        - Negative compression ratios are clamped to 0
        - Note: these metrics are the compressed quantization vs uncompressed quantization, so not against baseline prequantized values
- Data Collected:

Qwen3 Minecraft Ending Poem:

```
     Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
1023              1023                          54.053                         49.194
1022              1022                          56.323                         52.075
1021              1021                          52.637                         48.804
1020              1020                          55.737                         51.880
1019              1019                          54.956                         50.659
...                ...                             ...                            ...
4                    4                          41.309                         38.281
3                    3                          45.874                         43.286
2                    2                          49.976                         46.143
1                    1                          48.340                         44.751
0                    0                          59.937                         55.103

[1024 rows x 3 columns]
Concat % Saving Overall (ZSTD): 43.253, Average: 43.253203125000056
Concat % Saving Overall (LZ4): 39.728, Average: 39.72783593749996
4096.0
     Vector Embedding                  Bit Plane Ratios (ZSTD):                   Bit Plane Ratios (LZ4):
1023              1023  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1022              1022  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1021              1021  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1020              1020  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1019              1019  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
...                ...                                       ...                                       ...
4                    4  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
3                    3  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
2                    2  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1                    1  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
0                    0  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

- It seems that ZSTD only yields ~3% improvement over LZ4, and since LZ4 is faster, I would recommend that one to strike a good balance between performance and compression efficiency.

Wikipedia DPR:

```
     Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
1023              1023                          30.729                         27.734
1022              1022                          27.474                         24.089
1021              1021                          30.339                         27.083
1020              1020                          28.646                         25.781
1019              1019                          27.995                         25.000
...                ...                             ...                            ...
4                    4                          31.120                         27.865
3                    3                          31.901                         29.167
2                    2                          31.380                         28.385
1                    1                          30.339                         27.344
0                    0                          30.859                         27.604

[1024 rows x 3 columns]
Concat % Saving Overall (ZSTD): 29.411, Average: 29.410815429687496
Concat % Saving Overall (LZ4): 26.274, Average: 26.27396972656253
768.0
     Vector Embedding                  Bit Plane Ratios (ZSTD):                   Bit Plane Ratios (LZ4):
1023              1023  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1022              1022  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1021              1021  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1020              1020  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1019              1019  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
...                ...                                       ...                                       ...
4                    4  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
3                    3  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
2                    2  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1                    1  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
0                    0  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

- Same pattern here. ZSTD has only a ~3% lead over LZ4, so I would prioritize LZ4 over ZSTD to strike that balance between performance and compression efficiency.

Fineweb-edu:

```
      Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
1023              1023                           1.660                          0.000
1022              1022                           1.172                          0.000
1021              1021                           3.516                          1.172
1020              1020                           3.027                          0.781
1019              1019                           7.617                          5.273
...                ...                             ...                            ...
4                    4                           0.000                          0.000
3                    3                           3.613                          1.172
2                    2                           5.859                          3.223
1                    1                           1.172                          0.000
0                    0                           2.930                          0.000

[1024 rows x 3 columns]
Concat % Saving Overall (ZSTD): 2.184, Average: 2.4578085937499967
Concat % Saving Overall (LZ4): 0.189, Average: 0.9418603515624998
1024.0
      Vector Embedding                  Bit Plane Ratios (ZSTD):                   Bit Plane Ratios (LZ4):
1023              1023  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1022              1022  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1021              1021  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1020              1020  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1019              1019  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
...                ...                                       ...                                       ...
4                    4  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
3                    3  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
2                    2  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
1                    1  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
0                    0  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

- For this dataset, LZ4 and ZSTD are not worth using. It is too dense with too little compression benefit to justify the computational overhead it induces.

- Some General Observations:
    - Qwen3: Very compressible at ~40% reduction
        - This suggests that 4096 dimensions per token is excessive space for context, as there are only around 1800 tokens in the poem. I should test this embedding model with text that represents more than 4096 tokens, so we can get the full context.
    - Wikipedia DPR: Reasonably compressible, ~25-30% reduction
        - This uses a smaller dim, and represents a typical online article database. It doesn't have the excessive context space that Qwen3 utilizes, so it has less redundancy, but it still has good overall compressibility.
    - Fineweb-edu: Weak compression, ~1-2% reduction
        - This suggests that fineweb-edu is packed in a way that is already highly efficient with minimal redundancy. This is good for making efficiently packed data, but it also means that it has almost no redundancy to exploit.
    - Since all of these datasets have <=512 byte context windows per bitplane when bit-packed, they don't compress well, so each bitplane has ~0% reduction after compression because the block size would be too small for LZ4 and ZSTD to exploit.
- To do next:
    - Add some new metrics:
        - (% savings) compressed quantized vs baseline (uncompressed non-quantized)

- Try this compression on the prequantized data too
- Try an option to only take the most significant bits
- Test on more datasets
- Add .safetensor file support

3/20: Coding Update: Some new metrics + Preliminary results / Custom Qwen3 data (Minecraft End Poem) + Wikipedia DPR + Fineweb Edu: (Tim update)

- Summary: I added a few more metrics that may be important. The most important new metric is quantized bitplane compressed vs baseline (no quantization or compression). I was able to achieve up to 92% reduction in memory.
- Data information:
  - Custom data:
    - Models used: Qwen3 8B FP16, E5 Mistral 7B FP16, E5 Large V2, BAAI BGE-M3, Nomic Embed V1.5 FP32
    - Block Sizes used: 4096, 4096, 1024, 1024, 768
    - File used: MinecraftEndPoem.txt
    - Base Datatype: FP64
  - Wikipedia DPR:
    - File used: train-00000-of-00157.parquet
    - Block Size: 768B
    - Base Datatype: FP32
  - Fineweb Edu:
    - File used: new_CC-MAIN-2024-10-train-00019-of-00020.npy
    - Block Size: 1024B (1KB)
    - Base Datatype: FP16
  - General Information:
    - Number of rows: 512
    - Starting row: 0
    - Quantized + Unquantized results
    - Quantization Datatype: INT8
- Data Collected:

Custom Data (Minecraft Ending Poem):

Qwen3:

```
Direct Compression(ZSTD) Ratio & % Memory Saving: 0.541 , 45.925%

Direct Compression(LZ4) Ratio & % Memory Saving: 0.896 , 10.351%
                                  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
vs Original Data Size:                                    92.679                         92.244
vs Quantized+Scalar:                                      41.492                         38.013
vs Direct Compression:                                    92.426                         92.244
vs Quant Compressed+Scale (No BP):                        -8.108                         30.864

 Extra: Uncompressed Bitplaned Quant+Scalar vs Uncompressed Bitplaned Original Tensor: 87.488
```

Qwen3 (without quantization):

```
     Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
511                511                          10.419                         10.123
510                510                          10.471                         10.199
509                509                          10.449                         10.165
508                508                          10.458                         10.217
507                507                          10.446                         10.184
..                 ...                             ...                            ...
4                    4                          10.526                         10.278
3                    3                          10.446                         10.202
2                    2                          10.513                         10.266
1                    1                          10.480                         10.205
0                    0                          10.767                         10.342

[512 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 10.497, Average: 10.49728710937498
Concat % Saving Overall (LZ4) vs Quantized: 10.235, Average: 10.23536132812499
```

E5 Mistral 7B FP16:

```
Direct Compression(ZSTD) Ratio & % Memory Saving: 0.587 , 41.265%

Direct Compression(LZ4) Ratio & % Memory Saving: 0.993 , 0.738%
                                 Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
vs Original Data Size:                                   91.861                         91.428
vs Quantized+Scalar:                                     34.950                         31.493
vs Direct Compression:                                   91.579                         91.428
vs Quant Compressed+Scale (No BP):                      -10.675                         30.985

 Extra: Uncompressed Bitplaned Quant+Scalar vs Uncompressed Bitplaned Original Tensor: 87.488
```

E5 Mistral 7B FP16 (without quantization):

```
     Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
511                511                          10.519                         10.281
510                510                          10.547                         10.300
509                509                          10.529                         10.284
508                508                          10.522                         10.297
507                507                          10.565                         10.333
..                 ...                             ...                            ...
4                    4                          10.461                         10.233
3                    3                          10.461                         10.178
2                    2                          10.507                         10.245
1                    1                          10.477                         10.208
0                    0                          10.580                         10.358

[512 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 10.527, Average: 10.527261718749997
Concat % Saving Overall (LZ4) vs Quantized: 10.273, Average: 10.273249999999988
```

E5 Large V2:

```
Direct Compression(ZSTD) Ratio & % Memory Saving: 0.957 , 4.259%

Direct Compression(LZ4) Ratio & % Memory Saving: 1.0 , 0.0%
                                  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
vs Original Data Size:                                    75.780                         75.361
vs Quantized+Scalar:                                       3.496                          1.828
vs Direct Compression:                                    75.443                         75.361
vs Quant Compressed+Scale (No BP):                        -0.779                          1.832

 Extra: Uncompressed Bitplaned Quant+Scalar vs Uncompressed Bitplaned Original Tensor: 74.902
```

E5 Large V2 (without quantization):

```
     Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
511               511                          10.840                         10.742
510               510                          11.475                         11.377
509               509                          11.255                         11.304
508               508                          11.011                         10.913
507               507                          11.230                         11.279
..                ...                             ...                            ...
4                   4                          11.206                         11.279
3                   3                          11.206                         11.255
2                   2                          11.206                         11.255
1                   1                          11.084                         11.084
0                   0                          11.035                         10.938

[512 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 11.178, Average: 11.177871093750012
Concat % Saving Overall (LZ4) vs Quantized: 11.197, Average: 11.196869140625044
```

BAAI BGE-M3:

```
Direct Compression(ZSTD) Ratio & % Memory Saving: 0.691 , 30.854%

Direct Compression(LZ4) Ratio & % Memory Saving: 0.529 , 47.05%
                                  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
vs Original Data Size:                                    81.559                         80.906
vs Quantized+Scalar:                                      26.521                         23.920
vs Direct Compression:                                    81.374                         80.906
vs Quant Compressed+Scale (No BP):                        -6.082                        -43.188

 Extra: Uncompressed Bitplaned Quant+Scalar vs Uncompressed Bitplaned Original Tensor: 74.902
```

BAAI BGE-M3 (without quantization):

```
     Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
511               511                          11.108                         11.133
510               510                          11.206                         11.255
509               509                          11.035                         10.913
508               508                          11.206                         11.255
507               507                          11.108                         11.133
..                ...                             ...                            ...
4                   4                          11.133                         11.133
3                   3                          11.206                         11.255
2                   2                          10.986                         10.913
1                   1                          11.133                         11.133
0                   0                          11.206                         11.255

[512 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 11.131, Average: 11.130443359374997
Concat % Saving Overall (LZ4) vs Quantized: 11.141, Average: 11.141109374999996
```

Nomic Embed V1.5 FP32:

```
Direct Compression(ZSTD) Ratio & % Memory Saving: 0.975 , 2.538%

Direct Compression(LZ4) Ratio & % Memory Saving: 1.0 , 0.0%
                                  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
vs Original Data Size:                                   87.777                         87.500
vs Quantized+Scalar:                                      2.726                          0.515
vs Direct Compression:                                   87.934                         87.500
vs Quant Compressed+Scale (No BP):                        0.207                          0.521

 Extra: Uncompressed Bitplaned Quant+Scalar vs Uncompressed Bitplaned Original Tensor: 87.435
```

Nomic Embed V1.5 FP32 (without quantization):

```
     Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
511               511                           9.375                          9.896
510               510                           9.424                          9.977
509               509                           9.424                          9.977
508               508                           9.424                         10.010
507               507                           9.424                          9.961
..                ...                             ...                            ...
4                   4                           9.375                          9.912
3                   3                           9.424                          9.993
2                   2                           9.375                          9.880
1                   1                           9.375                          9.896
0                   0                           9.424                          9.977

[512 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 9.394, Average: 9.394009765624991
Concat % Saving Overall (LZ4) vs Quantized: 9.932, Average: 9.931623046874986
```

Wikipedia DPR:

```
Direct Compression(ZSTD) Ratio & % Memory Saving: 0.684 , 31.581%

Direct Compression(LZ4) Ratio & % Memory Saving: 1.0 , 0.0%
                                  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
vs Original Data Size:                                   82.230                         81.440
vs Quantized+Scalar:                                     29.290                         26.145
vs Direct Compression:                                   82.459                         81.440
vs Quant Compressed+Scale (No BP):                       -3.101                         26.149

 Extra: Uncompressed Bitplaned Quant+Scalar vs Uncompressed Bitplaned Original Tensor: 74.87
```

(without quantization)

```
     Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
511               511                          10.775                         10.938
510               510                          10.677                         10.807
509               509                          10.677                         10.807
508               508                          10.677                         10.807
507               507                          10.645                         10.840
..                ...                             ...                            ...
4                   4                          10.775                         10.938
3                   3                          10.677                         10.807
2                   2                          10.677                         10.807
1                   1                          10.775                         10.938
0                   0                          10.677                         10.807

[512 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 10.718, Average: 10.71771289062496
Concat % Saving Overall (LZ4) vs Quantized: 10.855, Average: 10.85547460937501
```

Fineweb-Edu:

```
Direct Compression(ZSTD) Ratio & % Memory Saving: 0.961 , 3.907%

Direct Compression(LZ4) Ratio & % Memory Saving: 1.0 , 0.0%
                                 Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
vs Original Data Size:                                   50.904                         49.903
vs Quantized+Scalar:                                      2.189                          0.195
vs Direct Compression:                                   50.589                         49.904
vs Quant Compressed+Scale (No BP):                       -1.772                          0.199

 Extra: Uncompressed Bitplaned Quant+Scalar vs Uncompressed Bitplaned Original Tensor: 49.805
```

(without quantization)

```
     Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
511               511                           4.834                          4.395
510               510                           4.834                          4.395
509               509                           4.834                          4.395
508               508                           4.834                          4.395
507               507                           4.834                          4.395
..                ...                             ...                            ...
4                   4                           4.834                          4.395
3                   3                           4.834                          4.395
2                   2                           4.834                          4.395
1                   1                           4.834                          4.395
0                   0                           4.834                          4.395

[512 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 4.839, Average: 4.8394374999999865
Concat % Saving Overall (LZ4) vs Quantized: 4.396, Average: 4.396224609374984
```

- Some General Observations:
    - Higher dimensionality embedding models seem to be more compressible: 4096 dim Qwen3 and E5 Mistral have very similar compressibility of around 40% reduction
    - Lower dimensionality embedding models struggle with compression: 1024 and 768 dim E5 large V2, and Nomic Embed V1.5 FP32. around 0-2% reduction

    - Outlier: BAAI has surprisingly good compressibility considering it has low dimensionality (1024 dim). Around 25% reduction through compression alone
    - The results without quantization are so highly consistent that it may be a cause for concern. I will look into it further.
- Conclusion: The embedding model makes a considerable difference in the resulting output. I wonder if the high dimensionality embedding models introduce lots of padding and redundancy? I should try comparing the base and compressed size of each embedding model against eachother to see which one benefits the most from the compression. The without quantization results are a little too consistent, I have been getting odd outputs, so I will look into those.

Simulation Progress Report: B0.8.0

- Summary: This is mostly a progress update on what major changes have been added to the simulation. I will also gather more datasets today so I can put a wider variety of data on this document on Monday 3/23/26.

Added features:

- added 64 bit data input support (float64, int64)
- added bfloat16 support
- New Direct Compression support + comparisons
- Raw Tensor Bitplane Compression Analysis (enabled through a setting)
- New Metrics:

    --> (% memory savings) vs original data size

    --> (% memory savings) vs quantized+scalar

    --> (% memory savings) vs Direct compression (ZSTD & LZ4)

    --> (% memory savings) vs Quantized compressed (No bitplanes) + scalar uncompressed

    --> (% memory savings) extra: Uncompressed quant+scalar vs uncompressed original data

- New filetypes support: .arrow, .feather, .csv, .safetensor (don't recommend using these, since robust support for .parquet & .npy is enough for 95% of cases)

- New Embedding models added:

    --> E5 Mistral 7B FP16

    --> E5 Large V2

    --> BAAI BGE-M3

    --> Nomic Embed V1.5 FP32

- New Proprietary Embedding Models added (use only if you know what to do, and can pay for it):

    --> OpenAI V3 Large

    --> NV-Embed V2

    --> Llama Nemotron Embedding 1B

    --> Gemini Embedding 2

- To do for next time (B0.8.5) 3/23+:

    - Add an option to only do compression analytics on the first few MSBs (controllable with settings)

    - Test on more datasets

    - (maybe?) add some more metrics:

        - bandwidth savings

        - compression timing/latency

3/23: More Datasets / many datasets: (Tim Update)

Planning:

TRACE Paper:

- Booksum
- Wikitext

Original:

- Wikipedia DPR
- TREC RAG 2024
- Fineweb Edu

Planned:

- Wikimedia [not precomputed]
- ~~Amazon_vector_database [seems precomputed]~~
- Word2vec (google news) [seems precomputed]
- MS MARCO (researcher version with precomputed vector embeddings)
- ~~Cohere “Wikipedia-22” (multilingual edition)~~
- BEIR Benchmark (+GTE & BGE) [maybe, has text, not computed vector weights]
- Cohere msmarco v2.1 embed english v3 [seems precomputed, uses TREC RAG 2024]

Planned 2:

- FAISS (Deep-1B & GIST-1M) [maybe? Should ignore for now]
- Segment Anything Model (SAM) Embeddings [maybe? Should ignore for now]
- ~~Google Satellite Embeddings (64-2048 dim)~~
- Google Natural Questions (NQ) dataset (there are community precomputed ones) [couldn't find the precomputed ones, may have to manually compute the text]
- Amazon QA embeddings [maybe? Uses words instead of floats]
- ~~ESC-50 or audio set (Amazon audio set)~~
- ~~Nvidia Nemotron Cascade 2 SFT Data (nvidia/Nemotron-Cascade-2-SFT-Data)~~
- ~~Numtabdata2vec~~

Okay, I'll be trying to add the most viable precomputed vector embedding sources first:

- ~~Amazon_vector_database [seems precomputed](broken, isn't precomputed)~~
- Word2vec (google news) [seems precomputed](too small, only 300 dims)
- Cohere msmarco v2.1 embed english v3 [seems precomputed, uses TREC RAG 2024]

3/23: New Datasets tested / Cohere MS MARCO V2.1 Embed English V3 (TREC RAG 2024): (Tim Update)

- Summary: The new dataset MS MARCO seems to have very low compressibility, ~2-3% reduction in space. This makes sense since it is high entropy, and high density. This high entropy is partially due to how semantically random bing searches are (the data that makes up this dataset). Another issue is that its dimensions per vector are on the lower end (~1024 dims), so the LZ4 and ZSTD Compressors are struggling to find patterns within this small context.
- Data Information:
    - Dataset: MS MARCO V2.1 Embed English V3 (Cohere Embedding)(data source: TREC RAG 2024)
    - File Used: msmarco_v2.1_doc_segmented_00.parquet, msmarco_v2.1_doc_segmented_29.parquet, msmarco_v2.1_doc_segmented_59.parquet
    - Row start: 0
    - Block Size: 1KB
    - Number of rows: 1024
    - Dimensions: 1024
    - Quantized Datatype: INT8
- Data results:

File 00:

```
--- BITPLANE ENTROPY CHECK (Shape: 1024x8x1024) ---
Plane      | Mean (Bias)     | Status
--------------------------------------------
Bitplane 0  | 0.4997          | Random (Incompressible)
Bitplane 1  | 0.5006          | Random (Incompressible)
Bitplane 2  | 0.5009          | Random (Incompressible)
Bitplane 3  | 0.5002          | Random (Incompressible)
Bitplane 4  | 0.5000          | Random (Incompressible)
Bitplane 5  | 0.5003          | Random (Incompressible)
Bitplane 6  | 0.5009          | Random (Incompressible)
Bitplane 7  | 0.4999          | Random (Incompressible)
      Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
1023              1023                            1.660                          0.000
1022              1022                            1.758                          0.000
1021              1021                            3.809                          1.172
1020              1020                            2.051                          0.098
1019              1019                            5.664                          3.320
...                ...                              ...                            ...
4                    4                            1.758                          0.000
3                    3                            4.004                          1.270
2                    2                            2.734                          0.391
1                    1                            2.930                          0.391
0                    0                            5.176                          2.832

[1024 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 2.487, Average: 2.726260742187495
Concat % Saving Overall (LZ4) vs Quantized: 0.441, Average: 1.0918681640625003
```

```
DEBUG: DIRECT SIZE RIGHT BEFORE COMPRESSION:
DIRECT BYTES: 4194304

Direct Compression(ZSTD) Ratio & % Memory Saving: 0.716 , 28.354%

Direct Compression(LZ4) Ratio & % Memory Saving: 0.961 , 3.906%
DEBUG: DIRECT QUANTIZED SIZE RIGHT BEFORE COMPRESSION:
DIRECT BYTES: 1048576

Direct Compression(ZSTD) Ratio & % Memory Saving: 0.957 , 4.296%

Direct Compression(LZ4) Ratio & % Memory Saving: 1.0 , 0.0%
                              Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
vs Original Data Size:                            75.524                         75.013
vs Quantized+Scalar:                               2.477                          0.440
vs Direct Compression:                            65.838                         73.997
vs Quant Compressed+Scale (No BP):                -1.883                          0.442

 Extra: Uncompressed Bitplaned Quant+Scalar vs Uncompressed Bitplaned Original Tensor: 74.902
Bitplane 0 | Avg Entropy: 0.9995 | Max Potential Savings: 0.05%
Bitplane 1 | Avg Entropy: 0.9994 | Max Potential Savings: 0.06%
Bitplane 2 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 3 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 4 | Avg Entropy: 0.9994 | Max Potential Savings: 0.06%
Bitplane 5 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 6 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 7 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
```

File 29:

```
--- BITPLANE ENTROPY CHECK (Shape: 1024x8x1024) ---
Plane      | Mean (Bias)     | Status
-------------------------------------------
Bitplane 0  | 0.4960          | Random (Incompressible)
Bitplane 1  | 0.4986          | Random (Incompressible)
Bitplane 2  | 0.4995          | Random (Incompressible)
Bitplane 3  | 0.4996          | Random (Incompressible)
Bitplane 4  | 0.5001          | Random (Incompressible)
Bitplane 5  | 0.4999          | Random (Incompressible)
Bitplane 6  | 0.5007          | Random (Incompressible)
Bitplane 7  | 0.5002          | Random (Incompressible)
      Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
1023              1023                             1.367                           0.000
1022              1022                             4.590                           2.148
1021              1021                             7.715                           5.371
1020              1020                             4.590                           1.367
1019              1019                             1.270                           0.000
...                ...                               ...                             ...
4                    4                             5.664                           3.418
3                    3                             5.762                           3.027
2                    2                             5.762                           3.027
1                    1                             1.465                           0.000
0                    0                             1.855                           0.000

[1024 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 2.89, Average: 3.055774414062503
Concat % Saving Overall (LZ4) vs Quantized: 0.737, Average: 1.2311933593749989
```

```
DEBUG: DIRECT SIZE RIGHT BEFORE COMPRESSION:
DIRECT BYTES: 4194304

Direct Compression(ZSTD) Ratio & % Memory Saving: 0.716 , 28.357%

Direct Compression(LZ4) Ratio & % Memory Saving: 0.961 , 3.919%
DEBUG: DIRECT QUANTIZED SIZE RIGHT BEFORE COMPRESSION:
DIRECT BYTES: 1048576

Direct Compression(ZSTD) Ratio & % Memory Saving: 0.955 , 4.505%

Direct Compression(LZ4) Ratio & % Memory Saving: 1.0 , 0.0%
                                 Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
vs Original Data Size:                                   75.625                          75.087
vs Quantized+Scalar:                                      2.879                           0.734
vs Direct Compression:                                   65.977                          74.070
vs Quant Compressed+Scale (No BP):                       -1.684                           0.736

 Extra: Uncompressed Bitplaned Quant+Scalar vs Uncompressed Bitplaned Original Tensor: 74.902
Bitplane 0 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 1 | Avg Entropy: 0.9994 | Max Potential Savings: 0.06%
Bitplane 2 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 3 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 4 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 5 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 6 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 7 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
```

File 59:

```
--- BITPLANE ENTROPY CHECK (Shape: 1024x8x1024) ---
Plane      | Mean (Bias)     | Status
--------------------------------------------
Bitplane 0  | 0.5042          | Random (Incompressible)
Bitplane 1  | 0.5052          | Random (Incompressible)
Bitplane 2  | 0.5042          | Random (Incompressible)
Bitplane 3  | 0.4994          | Random (Incompressible)
Bitplane 4  | 0.5000          | Random (Incompressible)
Bitplane 5  | 0.4997          | Random (Incompressible)
Bitplane 6  | 0.5003          | Random (Incompressible)
Bitplane 7  | 0.5000          | Random (Incompressible)
      Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
1023              1023                            2.930                          0.293
1022              1022                            2.637                          0.000
1021              1021                            0.000                          0.000
1020              1020                            0.000                          0.000
1019              1019                            0.000                          0.000
...                ...                              ...                            ...
4                    4                            4.199                          1.855
3                    3                            2.344                          0.000
2                    2                            2.734                          0.586
1                    1                            0.000                          0.000
0                    0                            0.000                          0.000

[1024 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 2.011, Average: 2.264975585937501
Concat % Saving Overall (LZ4) vs Quantized: 0.0, Average: 0.6997187500000002
```

```
DEBUG: DIRECT SIZE RIGHT BEFORE COMPRESSION:
DIRECT BYTES: 4194304

Direct Compression(ZSTD) Ratio & % Memory Saving: 0.716 , 28.358%

Direct Compression(LZ4) Ratio & % Memory Saving: 0.96 , 3.995%
DEBUG: DIRECT QUANTIZED SIZE RIGHT BEFORE COMPRESSION:
DIRECT BYTES: 1048576

Direct Compression(ZSTD) Ratio & % Memory Saving: 0.962 , 3.788%

Direct Compression(LZ4) Ratio & % Memory Saving: 1.0 , 0.0%
                              Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
vs Original Data Size:                                75.405                         74.902
vs Quantized+Scalar:                                   2.003                          0.000
vs Direct Compression:                                65.670                         73.858
vs Quant Compressed+Scale (No BP):                    -1.839                          0.002

 Extra: Uncompressed Bitplaned Quant+Scalar vs Uncompressed Bitplaned Original Tensor: 74.902
Bitplane 0 | Avg Entropy: 0.9995 | Max Potential Savings: 0.05%
Bitplane 1 | Avg Entropy: 0.9994 | Max Potential Savings: 0.06%
Bitplane 2 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 3 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 4 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 5 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 6 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
Bitplane 7 | Avg Entropy: 0.9993 | Max Potential Savings: 0.07%
```

Some Observations:

- The entropy on this dataset is very high, with predicted entropy floor savings of 0.07%. The bias is ~50% on all the files tested, which means it has nearly the same number of 1s and 0s per bitplane, making it harder to compress.

- Overall space savings range from 2-3%
- Directly compressing the quantized data yields slightly better results than using the bitplane. So to save on latency, it would probably be best to not use bitplane disaggregation.
- Direct compression (no bitplaning) of the non-quantized data led to ~30% reduction

Out of curiosity, I tried Bitplane Disaggregation Compression analysis on the non-quantized data too. I got much better compression results, typically ~50% with both ZSTD and LZ4:

File 00:

```
--- BITPLANE ENTROPY CHECK (Shape: 1024x32x1024) ---
Plane      | Mean (Bias)     | Status
-------------------------------------------
Bitplane 0  | 0.5064          | Random (Incompressible)
Bitplane 1  | 0.0000          | Highly Structured
Bitplane 2  | 1.0000          | Highly Structured
Bitplane 3  | 1.0000          | Highly Structured
...
Bitplane 16 | 0.4994          | Random (Incompressible)
Bitplane 28 | 0.0000          | Highly Structured
Bitplane 29 | 0.0000          | Highly Structured
Bitplane 30 | 0.0000          | Highly Structured
Bitplane 31 | 0.0000          | Highly Structured
      Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
1023              1023                           51.367                          51.514
1022              1022                           51.367                          51.611
1021              1021                           51.343                          51.636
1020              1020                           51.392                          51.611
1019              1019                           51.245                          51.514
...                ...                              ...                             ...
4                    4                           51.343                          51.611
3                    3                           51.270                          51.489
2                    2                           51.270                          51.489
1                    1                           51.245                          51.489
0                    0                           51.172                          51.318

[1024 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 51.276, Average: 51.27617480468749
Concat % Saving Overall (LZ4) vs Quantized: 51.494, Average: 51.4934912109374
```

File 29:

```
--- BITPLANE ENTROPY CHECK (Shape: 1024x32x1024) ---
Plane      | Mean (Bias)     | Status
-------------------------------------------
Bitplane 0  | 0.5030          | Random (Incompressible)
Bitplane 1  | 0.0000          | Highly Structured
Bitplane 2  | 1.0000          | Highly Structured
Bitplane 3  | 1.0000          | Highly Structured
...
Bitplane 16 | 0.4993          | Random (Incompressible)
Bitplane 28 | 0.0000          | Highly Structured
Bitplane 29 | 0.0000          | Highly Structured
Bitplane 30 | 0.0000          | Highly Structured
Bitplane 31 | 0.0000          | Highly Structured
      Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
1023              1023                           51.343                          51.611
1022              1022                           51.270                          51.489
1021              1021                           51.270                          51.416
1020              1020                           51.343                          51.611
1019              1019                           51.099                          51.221
...                ...                              ...                             ...
4                    4                           51.294                          51.489
3                    3                           51.196                          51.343
2                    2                           51.147                          51.245
1                    1                           51.343                          51.611
0                    0                           51.245                          51.489

[1024 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 51.275, Average: 51.27502343750002
Concat % Saving Overall (LZ4) vs Quantized: 51.488, Average: 51.487830078124844
```

File 59:

```
--- BITPLANE ENTROPY CHECK (Shape: 1024x32x1024) ---
Plane      | Mean (Bias)     | Status
-------------------------------------------
Bitplane 0  | 0.5108          | Random (Incompressible)
Bitplane 1  | 0.0000          | Highly Structured
Bitplane 2  | 1.0000          | Highly Structured
Bitplane 3  | 1.0000          | Highly Structured
...
Bitplane 16 | 0.4991          | Random (Incompressible)
Bitplane 28 | 0.0000          | Highly Structured
Bitplane 29 | 0.0000          | Highly Structured
Bitplane 30 | 0.0000          | Highly Structured
Bitplane 31 | 0.0000          | Highly Structured
      Vector Embedding  Concatenated % Savings (ZSTD):  Concatenated % Savings (LZ4):
1023              1023                           51.172                          51.343
1022              1022                           51.245                          51.514
1021              1021                           51.392                          51.611
1020              1020                           51.343                          51.611
1019              1019                           51.099                          51.343
...                ...                              ...                             ...
4                    4                           51.172                          51.196
3                    3                           51.172                          51.318
2                    2                           51.172                          51.392
1                    1                           51.245                          51.514
0                    0                           51.172                          51.367

[1024 rows x 3 columns]
Concat % Saving Overall (ZSTD) vs Quantized: 51.276, Average: 51.275904296875105
Concat % Saving Overall (LZ4) vs Quantized: 51.494, Average: 51.49417871093746
```

Individual Bitplane compression File 59 (to show each compressed bitplane):

Vector Embedding		Bit Plane Ratios (ZSTD):	Bit Plane Ratios (LZ4):
1023	1023	[1.0, 0.148, 0.172, 0.172, 0.219, 1.0, 1.0, 1....	[1.0, 0.266, 0.266, 0.266, 0.352, 1.0, 1.0, 1....
1022	1022	[1.0, 0.148, 0.172, 0.172, 0.195, 1.0, 1.0, 1....	[1.0, 0.266, 0.266, 0.266, 0.32, 1.0, 1.0, 1.0...
1021	1021	[1.0, 0.148, 0.172, 0.172, 0.172, 1.0, 1.0, 1....	[1.0, 0.266, 0.266, 0.266, 0.266, 1.0, 1.0, 1....
1020	1020	[1.0, 0.148, 0.172, 0.172, 0.172, 1.0, 1.0, 1....	[1.0, 0.266, 0.266, 0.266, 0.266, 1.0, 1.0, 1....
1019	1019	[1.0, 0.148, 0.172, 0.172, 0.258, 1.0, 1.0, 1....	[1.0, 0.266, 0.266, 0.266, 0.406, 1.0, 1.0, 1....
...	...	...	...
4	4	[1.0, 0.148, 0.172, 0.172, 0.203, 1.0, 1.0, 1....	[1.0, 0.266, 0.266, 0.266, 0.352, 1.0, 1.0, 1....
3	3	[1.0, 0.148, 0.172, 0.172, 0.227, 1.0, 1.0, 1....	[1.0, 0.266, 0.266, 0.266, 0.359, 1.0, 1.0, 1....
2	2	[1.0, 0.148, 0.172, 0.172, 0.227, 1.0, 1.0, 1....	[1.0, 0.266, 0.266, 0.266, 0.359, 1.0, 1.0, 1....
1	1	[1.0, 0.148, 0.172, 0.172, 0.203, 1.0, 1.0, 1....	[1.0, 0.266, 0.266, 0.266, 0.32, 1.0, 1.0, 1.0...
0	0	[1.0, 0.148, 0.172, 0.172, 0.227, 1.0, 1.0, 1....	[1.0, 0.266, 0.266, 0.266, 0.367, 1.0, 1.0, 1....

Observations:

- Compressibiility is much better on non-quantized values. I get ~51% reduction in memory footprint. This suggests that the quantization greatly increases entropy.
- Individual Bitplane Compressibility: The MSB Bit 0 is highly random, hence isn't very compressible. MSB bits 1-4 are highly compressible. It seems like the rest of the bits aren't very compressible.
- What concerns me a bit is the entropy of each bitplane, it is not as expected, where the first few MSBs have the most bias, while the rest have almost no bias

- To do for next time (B0.8.5) 3/24+:

    - Add an option to only do compression analytics on the first few MSBs (controllable with settings)

    - Test on more datasets

    - Important: Add support for more datasets by allowing embedding models on many file types (.parquet, .npy, .jsonl, etc)

    - (maybe?) add some more metrics:

        - bandwidth savings

        - compression timing/latency

- To do next for research documentation:

- maintain a large table of quantized and unquantized results for each dataset and embedding model

Datasets to test for today (add support for it, manual vector embedding models on these) 3/24/26:

- Booksum (maybe)
- Fineweb Edu (should)
- Wikitext (maybe)
- Amazon_vector_database (definitely)
- BEIR Benchmark (+GTE & BGE) [has text, definitely]
- FAISS (Deep-1B & GIST-1M) [definitely]

4/2: Progress Report update: Embedding model overhaul / Fineweb-edu: (Tim Update)

- Summary: Over the past few days, have been ironing out bugs, compatibility issues (hardware and software), recompilation of libraries, etc
- Pulled from text:

I finally made some large progress over the past few days.

- finally got GPU acceleration working on the embedding models
- added default casting native to each embedding model
- removed all of the proprietary models (openai, Gemini, Nvidia)
- added 2 new models:

--> Gemma 2 Embed 9B FP32 (open source version of Gemini 2) --> Snowflake Arctic Embedding L V2.0 FP32 (extremely common in large scale RAG in the AI industry)

- unified embedding model API to llama (a lot more automation and standardization)
- better wide range GPU support
- fixes in place for Intel ARC-specific issues

Overall, we now have 7 models to use:

1. Qwen3 embed 8B FP16
2. e5 Mistral 7B FP16
3. Nomic Embed text v1.5 FP32 (common in industry for performance)
4. E5 Large v2 FP32 (has NaN issues on Intel ARC GPUs, won't collect data for it yet)(not commonly used for LLMs due to legacy status. Superceded by e5 Mistral)
5. BAAI BGE-M3 FP16
6. Gemma 2 Embed 9B FP32

7. Snowflake Arctic embed v2.0 FP32 (extremely common in RAG industry)

- I'll finish testing today and verifying data integrity, and will try to get some results in soon
- To do next time 4/3+:
    - Fix the embedding model truncation and padding characteristics, follow Rui's rules
    - Extensively test Gemma 2 and Snowflake
    - Import the data into this documentation
    - Download the rest of the datasets
    - Maybe add the precursor prompting to (Nomic Embed (optional, search_query), E5 large v2 (query:), E5 Mistral (query:), Gemma 2 (Retrieval-query:))

4/7: Embedding Chunking Implementation Large Update: Consistent RAG Embedding Model Chunking / Fineweb-edu: (Tim update)

- Summary of progress: I finally implemented a robust chunking method for Intel XPU, and plan to implement it for CUDA and ROCm by the end of the day.
- Here is a summary of this pipeline: (This is for line-based semantic chunking with wrapping only for overflowing lines)
    - 1. Get text data line-by-line
    - 2. For each line:
        - A. Tokenize it
        - B. Split into equally sized chunks of tokens (only if too long)
        - C. Detokenize each chunk
        - D. Feed each text chunk into the embedding model
        - E. Format output of embedding model so a single vector embedding comes out per chunk (semantic line)
    - 3. Stack all the vector embeddings vertically in order of the source (vertical stack numpy ndarray)
    - 4. Feed into benchmark simulation

4/16: Mini update: Updated Embedding model roster + new raw metrics + BFloat16 support / multiple datasets: (Tim update)

- Summary of progress: Nearly done collecting data.
    - Added BFloat16 support
    - Replaced Gemma 2 Embed 9B FP32 with EmbeddingGemma 300M FP16
    - Replaced Qwen3 Embed 8B with Qwen3 Embed 4B
    - Fixed NaN issues on E5 Large V2
    - Added Harrier OSS V1 0.6B Embedding BFloat16
    - Added new Raw compression metrics:
        - % space savings direct compression vs baseline
        - % space savings bitplane compression vs baseline

        - % space savings bitplane compression vs direct compression (no bitplane)
    - Optimization: Reduces the amount of embedding computation to the required amount (no change in results, but computation is faster, especially on long-context sources)

4/24: Progress update: Collected data, Found integrity issues, fixed / all datasets: (Tim Update)

- Summary: Byte Block Compression was performed incorrectly on LZ4 due to block_size parameter not working as intended. Replaced with manual blocking to ensure robust results.
- Note: Thank you Rui
- What went wrong:
    - Crazy outlier data: ex: -10577% loss compared to no bitplaned LZ4, meant 99.507% savings without bitplaning, which is theoretically impossible
    - Broken data for 3 datasets (gemma-embedding, harrier oss, e5 large v2)
    - For pure data integrity, after the fix, data will be collected running on CPU-only. It will take longer to run the simulations, but ensures more accurate, reliable results
- To do next: requires recollecting all the data again. Should take another 3-4 days

4/29: Progress update: Finished recollecting data / all datasets: (Tim Update)

- Summary: Data looks as intended. No more high negative % outliers. Bitplaning generally outperforms without bitplaning.
- To do next: will implement into slideshow

4/30: Progress update: Finished Slideshow / All datasets: (Tim Update)

- Summary: Slideshow is finished with summarized results.
- To do next:
    - Work on experimental features:
        - Multimodal model support
        - Nvidia CUDA support
        - Elastic retrieval?
    - Add back in gemma 2 embed and qwen3 8B
    - Prepare for the presentation
- Known issues:
    - E5 large V2 is still broken on intel arc SYCL backend
    - E5 large V2 is also broken on AMD ROCm backend

Major tables:

Configurations:

Configurations: Precomputed + Computed	Wikipedia DPR: (Precomputed)	Fineweb-Edu: (Precomputed)	Cohere MS MARCO V2.1 English V3 (TREC RAG 2024): (Precomputed)	Google News (Words2Vec): (Precomputed)	Booksum: (Computed)	Fineweb Edu emb: (Computed)	Wikitext (103): (Computed)	Amazon_vector_database: (Computed)	BEIR nfcorpus: (Computed)	BEIR fiqa: (Computed)	BEIR touch-2020: (Computed)	BEIR dbpedia-entity: (Computed)	BEIR fever: (Computed)
File Used:	train-00000-of-00157.parquet	new_CC-MAIN-2024-10-train-00019-of-00020.npy	msmarco_v2.1_doc_segmented_00.parquet	word2vec-google-news-300.model.vectors.npy	train.csv	000_00000.parquet	train-00000-of-00002.parquet	embeddings.parquet	corpus-00000-of-00001.parquet	corpus-00000-of-00001.parquet	corpus-00000-of-00001.parquet	corpus-00000-of-00001.parquet	corpus-00000-of-00001.parquet
Rows Sampled/Rows Analyzed:	1024	1024	1024	1024	1024	1024	1024	1024	1024	1024	1024	1024	1024
Dimension:	768	1024	1024	300	-	-	-	-	-	-	-	-	-
Block Size (Compression):	0.768 KB	1KB	1KB	300B	-	-	-	-	-	-	-	-	-
Base Datatype:	FP32	FP16	FP32	FP32									
Quantized Datatype:	INT8	INT8	INT8	INT8									

New graph order:

EmbeddingGemma 300M (FP16)(768 DIM)	BAAI BGE-M3 (FP16)(1024 DIM)	Qwen3 4B (FP16)(2560 DIM)	Harrier OSS V1 0.6B (BF16)(1024 DIM)	Nomic Embed V1.5 (FP32)(768 DIM)	Arctic Snowflake Embed v2.0 (FP32)(1024 DIM)	E5 Large V2 (FP32)(1024 DIM)

Fineweb Edu (FP16)(1024 DIM)

Wikipedia DPR (FP32)(768 DIM)

Cohere MS Marco (FP32)(1024 DIM)

Embedding Model Computed Vector Embeddings:

Configurations: Embedding Models	Qwen3 4B FP16:	E5 Mistral 7B FP16: (not used)	E5 Large V2 FP32:	BAAI BGE-M3 FP16:	Nomic Embed V1.5 FP32:	Gemma-2 Embed 9B FP16: (not used)	Arctic Snowflake Embed v2.0 FP32:	EmbeddingGemma 300M FP16: (based on Gemma 3)	Harrier OSS V1 0.6B BF16:
Pooling Method:	CLS	CLS	Mean	CLS	Mean	Mean	CLS	Mean	CLS
Rows Sampled/ Rows Analyzed:	1024	1024	1024	1024	1024	1024	1024	1024	1024
Token Context:	32,768	4096	512	8192	2048	8192	8192	2048	32,768
Dimensions:	2560 (8B does 4096)	4096	1024	1024	768	3584	1024	768	1024
Block Size (Compression):	2.56KiB (8B does 4KiB)	4KiB	1KiB	1KiB	0.768KiB	3.584KiB	1KiB	0.768KiB	1KiB
Base	FP16	FP16	FP32	FP16	FP32	FP16	FP32	FP16	BF16

Datatype:									
Quantized Datatype:	INT8	INT8	INT8	INT8	INT8	INT8	INT8	INT8	INT8

Precomputed Vector Embeddings:

Precomputed Vector Embeddings:	Quantized: Overall % Space Savings: (ZSTD/LZ4)	Quantized: % Space Savings vs Quantization: (ZSTD/LZ4)	Quantized: % Space Savings vs Direct Compression: (ZSTD/LZ4)	Quantized: % Space Savings vs (no bitplane): (ZSTD/LZ4)	Not Quantized: Overall % Space Savings: (ZSTD/LZ4)	Not Quantized: % Space Savings vs Direct Compression (ZSTD/LZ4)
Wikipedia DPR (FP32):	82.222 / 81.438	29.258 / 26.138	82.451 / 81.438	-3.197 / 26.140	10.718 / 10.857	11.866 / 10.858
Fineweb-Edu (FP16):	50.897 / 49.899	2.176 / 0.188	50.587 / 49.900	-1.748 / 0.190	4.841 / 4.396	4.240 / 4.397
Cohere MS MARCO V2.1 English V3 (TREC RAG 2024) (FP32):	75.524 / 75.013	2.477 / 0.440	65.838 / 73.997	-1.883 / 0.442	51.276 / 51.494	31.993 / 49.522

Note: Google news crashes the simulation becuase it's dim is too small for the compressor (dim = 300)

Embedding Model Computed Vector Embeddings: (Overall % Space Savings)

How to interpret general table:

- (1, 2) / (3, 4)
- 1: (ZSTD) bitplane compressed (quant compressed + scalar uncompressed) vs uncompressed unquantized
- 2: (LZ4) bitplane compressed (quant compressed + scalar uncompressed) vs uncompressed unquantized
- 3: (ZSTD) bitplane compressed (unquantized) vs uncompressed unquantized
- 4: (LZ4) bitplane compressed (unquantized) vs uncompressed unquantized

Embedding	Qwen3 4B	Harrier OSS	E5 Large V2	BAAI	Nomic	Embedding	Arctic

Model Computed Vector Embeddings:	FP16: (Quant/Unquant)	V1 0.6B BF16: (Quant/Unquant)	FP32: (Quant/Unquant)	BGE-M3 FP16: (Quant/Unquant)	Embed V1.5 FP32: (Quant/Unquant)	Gemma 300M FP16: (Quant/Unquant)	Snowflake Embed v2.0 FP32: (Quant/Unquant)
Booksum	(58.057 / 57.181) / (5.645 / 5.247)	(71.133 / 69.923) / (22.659 / 22.061)	(76.953 / 76.416) / (11.133 / 11.133)	(57.598 / 56.198) / (4.84 / 4.396)	(76.278 / 75.658) / (10.758 / 10.902)	(68.621 / 66.534) / (4.375 / 3.846)	(75.988 / 75.466) / (11.146 / 11.165)
Fineweb Edu emb	(60.101 / 59.199) / (5.649 / 5.238)	(71.401 / 70.258) / (22.739 / 22.112)	(76.953 / 76.416) / (11.133 / 11.133)	(56.533 / 55.146) / (4.839 / 4.396)	(76.622 / 76.017) / (10.756 / 10.898)	(68.417 / 66.433) / (4.37 / 3.843)	(75.748 / 75.238) / (11.148 / 11.167)
Wikitext (103)	(60.050 / 59.191) / (5.646 / 5.235)	(65.696 / 64.648) / (22.75 / 22.084)	(76.953 / 76.416) / (11.133 / 11.133)	(58.371 / 56.866) / (4.838 / 4.395)	(77.191 / 76.578) / (10.726 / 10.86)	(65.948 / 63.904) / (4.363 / 3.841)	(76.195 / 75.626) / (11.18 / 11.212)
Amazon_vector_database	(56.758 / 55.820) / (5.645 / 5.234)	(71.240 / 70.068) / (22.803 / 22.266)	(76.953 / 76.416) / (11.133 / 11.133)	(56.707 / 55.280) / (4.839 / 4.396)	(76.767 / 76.150) / (10.758 / 10.896)	(67.391 / 65.565) / (4.365 / 3.842)	(75.697 / 75.198) / (11.151 / 11.171)
BEIR nfcorpus	(60.137 / 59.252) / (5.654 / 5.24)	(71.705 / 70.606) / (22.724 / 22.085)	(76.953 / 76.416) / (11.133 / 11.133)	(56.851 / 55.439) / (4.838 / 4.396)	(76.499 / 75.870) / (10.756 / 10.894)	(66.814 / 64.711) / (4.366 / 3.843)	(75.758 / 75.257) / (11.15 / 11.17)
BEIR fiqa	(59.007 / 58.108) / (5.649 / 5.239)	(71.151 / 70.042) / (22.7 / 22.039)	(76.953 / 76.416) / (11.133 / 11.133)	(56.592 / 55.179) / (4.838 / 4.396)	(76.699 / 76.082) / (10.754 / 10.895)	(66.315 / 64.236) / (4.364 / 3.842)	(75.783 / 75.279) / (11.152 / 11.173)
BEIR touche 2020	(58.865 / 57.931) / (5.648 / 5.238)	(71.846 / 70.816) / (22.723 / 22.078)	(76.953 / 76.416) / (11.133 / 11.133)	(56.791 / 55.363) / (4.837 / 4.395)	(75.897 / 75.300) / (10.759 / 10.896)	(67.240 / 65.142) / (4.368 / 3.843)	(76.129 / 75.609) / (11.148 / 11.164)
BEIR dbpedia-entity	(59.750 / 58.845) / (5.649 / 5.235)	(71.230 / 70.074) / (22.786 / 22.23)	(76.953 / 76.416) / (11.133 / 11.133)	(56.090 / 54.756) / (4.839 / 4.396)	(76.818 / 76.222) / (10.754 / 10.894)	(65.351 / 63.275) / (4.363 / 3.842)	(75.735 / 75.242) / (11.156 / 11.177)
BEIR fever	(63.283 / 62.299) / (5.645 / 5.236)	(70.158 / 68.707) / (22.696 / 22.086)	(76.953 / 76.416) / (11.133 / 11.133)	(55.738 / 54.494) / (4.838 / 4.396)	(77.042 / 76.468) / (10.745 / 10.885)	(65.283 / 63.221) / (4.363 / 3.842)	(75.810 / 75.312) / (11.15 / 11.171)

Average: 9	(59.556 / 58.647) / (5.647 / 5.238)	(70.617 / 69.460) / (20.508 / 22.115)	(76.953 / 76.416) / (9.610 / -18013.325)	(56.807 / 55.413) / (4.838 / 4.395)	(76.645 / 76.149) / (10.751 / 10.891)	(66.82 / 64.780) / (4.366 / 3.842)	(75.871 / 75.358) / (11.153 / 11.187)

Embedding Model Computed Vector Embeddings (Detailed):

How to interpret detailed table:

- Row1: % memory savings (BP) vs Original Data Size: (ZSTD) / (LZ4)
- Row2: % memory savings (BP) vs Quantized+Scalar: (ZSTD) / (LZ4)
- Row3: % memory savings (BP) vs Compressed Raw: (ZSTD) / (LZ4)
- Row4: % memory savings (BP) vs Quant Compressed + Scale (No BP): (ZSTD) / (LZ4)
- Unquant: Anything below this is for unquantized compression
- Row5: % memory savings (BP) vs Original Size: (ZSTD) / (LZ4)
- Row6: % memory savings (BP) vs Compressed No BP: (ZSTD) / (LZ4) [still adding for this row]

Embedding Model Computed Vector Embeddings:	Qwen3 4B FP16: (Quant/Unquant)	Harrier OSS V1 0.6B BF16: (Quant/Unquant)	E5 Large V2 FP32: (Quant/Unquant)	BAAI BGE-M3 FP16: (Quant/Unquant)	Nomic Embed V1.5 FP32: (Quant/Unquant)	Embedding Gemma 300M FP16: (Quant/Unquant)	Arctic Snowflake Embed v2.0 FP32: (Quant/Unquant)
Booksum	58.057 / 57.181 16.246 / 14.496 55.752 / -668.127 (overflow?) -5.747 / -2576.948 unquant 5.645 / 5.247 0.459 / -1599.766 (overflow?)	71.133 / 69.923 42.491 / 40.079 65.731 / -955.184 (overflow?) -2.877 / -2729.825 (overflow?) Unquant 22.659 / 22.061 8.185 / -2634.295 (overflow?)	76.953 / 76.416 8.171 / 6.031 76.558 / -4706.998 (overflow?) 2.781 / -10577.720 (overflow?) Unquant 11.133 / 11.133 9.610 / -18013.325 (overflow?)	57.598 / 56.198 15.526 / 12.737 57.105 / 55.104 -3.536 / -81.484 Unquant 4.84 / 4.396 3.734 / 2.008	76.278 / 76.658 5.605 / 3.138 76.583 / 75.659 0.796 / -1.491 Unquant 10.758 / 10.902 11.905 / 10.902	68.621 / 66.534 37.567 / 33.415 69.024 / 66.535 -6.712 / 29.521 Unquant 4.375 / 3.846 5.604 / 3.847	75.988 / 75.466 4.327 / 2.246 75.711 / 75.466 -0.646 / -103.300 Unquant 11.146 / 11.279 10.122 / 11.165
Fineweb Edu emb	60.101 / 59.199 20.326 / 18.525 57.877 /	71.401 / 70.258 43.024 / 40.748 65.979 /	76.953 / 76.416 8.171 / 6.031 76.558 /	56.533 / 55.146 13.404 / 10.642 56.035 /	76.622 / 76.017 6.974 / 4.565 76.923 /	68.417 / 66.433 37.161 / 33.214 68.822 /	75.748 / 75.238 3.371 / 1.338 75.467 /

	54.427 -6.238 / 1.770 unquant 5.649 / 5.238 0.391 / -5.844	63.359 -3.295 / -46.921 Unquant 22.739 / 22.112 8.094 / 4.045	-4706.998 (overflow?) 2.781 / -10577.720 (overflow?) Unquant 11.133 / 11.133 9.610 / -18013.325 (overflow?)	55.147 -3.669 / 10.644 Unquant 4.839 / 4.396 3.748 / 4.397	76.017 1.119 / 4.568 Unquant 10.756 / 10.898 11.903 / 10.899	66.433 -6.604 / 32.947 Unquant 4.37 / 3.843 5.598 / 3.845	75.238 -0.499 / 1.340 Unquant 11.148 / 11.167 10.118 / 11.167
Wikitext (103)	60.050 / 59.191 20.225 / 18.510 57.855 / -1.824 -6.519 / -156.492 unquant 5.646 / 5.235 0.461 / -136.454	65.696 / 64.648 31.659 / 29.572 59.242 / -17.525 -4.684 / -244.768 Unquant 22.75 / 22.084 8.217 / -159.028	76.953 / 76.416 8.171 / 6.031 76.558 / -4706.998 (overflow?) 2.781 / -10577.720 (overflow?) Unquant 11.133 / 11.133 9.610 / -18013.325 (overflow?)	58.371 / 56.866 17.067 / 14.067 57.843 / 35.665 -4.823 / -29.144 Unquant 4.838 / 4.395 3.630 / -42.594	77.191 / 76.578 9.238 / 6.798 77.485 / 65.258 0.273 / -40.939 Unquant 10.726 / 10.86 11.873 / -32.224	65.948 / 63.904 32.248 / 28.182 66.385 / 45.937 -6.842 / -7.918 Unquant 4.363 / 3.841 5.593 / -44.020	76.195 / 75.626 5.149 / 2.882 75.953 / 64.711 -1.360 / -46.806 Unquant 11.18 / 11.212 10.280 / -28.546
Amazon_vector _database	56.758 / 55.820 13.651 / 11.778 54.404 / -1237.449 (overflow?) -6.252 / -12481.000 (overflow?) unquant 5.645 / 5.234 0.509 / -2768.834 (overflow?)	71.240 / 70.068 42.704 / 40.370 65.796 / -2632.509 (overflow?) -2.435 / -6675.820 (overflow?) Unquant 22.803 / 22.266 8.189 / -6996.500 (overflow?)	76.953 / 76.416 8.171 / 6.031 76.558 / -4706.998 (overflow?) 2.781 / -10577.720 (overflow?) Unquant 11.133 / 11.133 9.610 / -18013.325 (overflow?)	56.707 / 55.280 13.750 / 10.908 56.209 / 55.281 -3.789 / 10.910 Unquant 4.839 / 4.396 3.746 / 4.397	76.767 / 76.150 7.548 / 5.093 77.065 / 76.150 1.220 / 5.096 Unquant 10.758 / 10.896 11.905 / 10.896	67.391 / 65.565 35.120 / 31.488 67.810 / 65.566 -7.270 / 31.490 Unquant 4.365 / 3.842 5.593 / 3.844	75.697 / 75.198 3.168 / 1.179 75.418 / 75.198 -0.659 / 1.181 Unquant 11.151 / 11.171 10.129 / 11.171
BEIR nfcorpus	60.137 / 59.252 20.398 / 18.631	71.705 / 70.606 43.631 / 41.441	76.953 / 76.416 8.171 / 6.031	56.851 / 55.439 14.038 / 11.226	76.499 / 75.870 6.483 / 3.981	66.814 / 64.711 33.972 / 29.788	75.758 / 75.257 3.410 / 1.411

	57.946 / 41.147 -6.062 / -45.295 unquant 5.654 / 5.24 0.469 / -36.862	66.346 / 44.509 -2.902 / -139.562 Unquant 22.724 / 22.085 8.087 / -47.091	76.558 / -4706.998 (overflow?) 2.781 / -10577.720 (overflow?) Unquant 11.133 / 11.133 9.610 / -18013.325 (overflow?)	56.372 / 55.440 -3.735 / 11.228 Unquant 4.838 / 4.396 3.780 / 4.397	76.801 / 75.870 1.516 / 3.984 Unquant 10.756 / 10.894 11.903 / 10.895	67.240 / 64.712 -7.263 / 29.790 Unquant 4.366 / 3.843 5.594 / 3.844	75.484 / 75.257 -0.602 / 1.414 Unquant 11.15 / 11.17 10.145 / 11.171
BEIR fiqa	59.007 / 58.108 18.143 / 16.347 56.743 / 50.993 -6.145 / -14.387 unquant 5.649 / 5.239 0.437 / -10.855	71.151 / 70.042 42.527 / 40.317 65.680 / 57.184 -3.588 / -69.729 Unquant 22.7 / 22.039 8.038 / -11.421	76.953 / 76.416 8.171 / 6.031 76.558 / -4706.998 (overflow?) 2.781 / -10577.720 (overflow?) Unquant 11.133 / 11.133 9.610 / -18013.325 (overflow?)	56.592 / 55.179 13.522 / 10.706 56.114 / 55.179 -3.857 / 10.708 Unquant 4.838 / 4.396 3.789 / 4.397	76.699 / 76.082 7.279 / 4.825 76.999 / 76.082 1.209 / 4.828 Unquant 10.754 / 10.895 11.901 / 10.896	66.315 / 64.236 32.980 / 28.842 66.748 / 64.236 -7.093 / 28.844 Unquant 4.364 / 3.842 5.593 / 3.844	75.783 / 75.279 3.511 / 1.501 75.503 / 75.279 -0.623 / 1.503 Unquant 11.152 / 11.173 10.121 / 11.173
BEIR touche-2020	58.865 / 57.931 17.858 / 15.994 56.584 / 43.822 -6.050 / -26.888 unquant 5.648 / 5.238 0.417 / -26.545	71.846 / 70.816 43.910 / 41.859 66.497 / 53.945 -3.263 / -83.762 Unquant 22.723 / 22.078 8.042 / -22.969	76.953 / 76.416 8.171 / 6.031 76.558 / -4706.998 (overflow?) 2.781 / -10577.720 (overflow?) Unquant 11.133 / 11.133 9.610 / -18013.325 (overflow?)	56.791 / 55.363 13.918 / 11.073 56.329 / 55.363 -3.848 / 11.075 Unquant 4.837 / 4.395 3.818 / 4.397	75.897 / 75.300 4.087 / 1.712 76.207 / 75.300 1.047 / 1.715 Unquant 10.759 / 10.896 11.906 / 10.897	67.240 / 65.142 34.819 / 30.646 67.660 / 65.143 -6.934 / 30.126 Unquant 4.368 / 3.843 5.595 / 3.845	76.129 / 75.609 4.887 / 2.815 75.857 / 75.609 -0.818 / 2.817 Unquant 11.148 / 11.164 10.134 / 11.164
BEIR dbpedia-entity	59.750 / 58.845 19.625 /	71.230 / 70.074 42.684 /	76.953 / 76.416 8.171 /	56.090 / 54.756 12.521 /	76.818 / 76.222 7.754 /	65.351 / 63.275 31.062 /	75.735 / 75.242 3.317 /

	17.818 57.490 / 29.880 -6.467 / -68.515 unquant 5.649 / 5.235 0.353 / -61.461	40.382 65.773 / 32.843 -3.354 / -192.330 Unquant 22.786 / 22.23 8.140 / -74.525	6.031 76.558 / -4706.998 (overflow?) 2.781 / -10577.720 (overflow?) Unquant 11.133 / 11.133 9.610 / -18013.325 (overflow?)	9.864 55.571 / 54.757 -3.548 / 9.866 Unquant 4.839 / 4.396 3.714 / 4.397	5.379 77.116 / 76.222 1.272 / 5.382 Unquant 10.754 / 10.894 11.901 / 10.895	26.930 65.797 / 63.275 -6.225 / 26.932 Unquant 4.363 / 3.842 5.592 / 3.843	1.352 75.455 / 75.242 -0.596 / 1.355 Unquant 11.156 / 11.177 10.131 / 11.177
BEIR fever	63.283 / 62.299 26.680 / 24.715 61.253 / -24.589 -6.400 / -236.274 unquant 5.645 / 5.236 0.430 / -213.160	70.158 / 68.707 40.549 / 37.658 64.394 / -50.285 -3.517 / -401.735 Unquant 22.696 / 22.086 7.763 / -274.186	76.953 / 76.416 8.171 / 6.031 76.558 / -4706.998 (overflow?) 2.781 / -10577.720 (overflow?) Unquant 11.133 / 11.133 9.610 / -18013.325 (overflow?)	55.738 / 54.494 11.821 / 9.343 55.219 / 54.093 -3.164 / 7.919 Unquant 4.838 / 4.396 3.723 / 3.553	77.042 / 76.468 8.644 / 6.358 77.337 / 76.353 0.777 / 4.329 Unquant 10.745 / 10.885 11.893 / 10.450	65.283 / 63.221 30.926 / 26.824 65.729 / 62.751 -6.387 / 24.991 Unquant 4.363 / 3.842 5.591 / 2.612	75.810 / 75.312 3.617 / 1.634 75.541 / 75.313 -0.853 / -0.604 Unquant 11.15 / 11.171 10.161 / 11.172
Average: 9	59.556 / 58.647 19.239 / 17.423 57.322 / -190.191 -6.208 / -1733.781 Unquant 5.647 / 5.238 0.436 / -539.975	70.617 / 69.460 41.464 / 39.158 65.048 / -378.184 -3.323 / -1176.050 Unquant 20.508 / 22.115 8.083 / -1135.107	76.953 / 76.416 8.171 / 6.031 76.558 / -4706.998 2.781 / -10577.720 Unquant 11.133 / 11.133 9.610 / -18013.325	56.807 / 55.413 13.951 / 11.174 56.310 / 52.892 -3.774 / -4.253 Unquant 4.838 / 4.395 3.742 / -1.183	76.645 / 76.149 7.068 / 4.649 76.946 / 74.767 1.025 / -1.392 Unquant 10.751 / 10.891 11.898 / 6.056	66.82 / 64.780 33.983 / 29.925 67.246 / 62.732 -6.814 / 25.191 Unquant 4.366 / 3.842 5.594 / -1.610	75.871 / 75.358 3.861 / 1.817 75.598 / 74.145 -0.739 / -15.677 Unquant 11.153 / 11.187 10.149 / 6.757

Datset file download links:

- Wikipedia DPR:

- Fineweb-Edu: https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu/tree/main/data/CC-MAIN-2024-10
- Cohere MS MARCO V2.1 English V3 (TREC RAG 2024): https://huggingface.co/datasets/Cohere/msmarco-v2.1-embed-english-v3/tree/main/passages_parquet
- Google News (Words2Vec): https://huggingface.co/fse/word2vec-google-news-300/tree/main
- Booksum: https://huggingface.co/datasets/kmfoda/booksum/tree/main
- Fineweb Edu emb:
- Wikitext: https://huggingface.co/datasets/Salesforce/wikitext/tree/main
- Amazon_vector_data_base: https://huggingface.co/datasets/chen196473/amazon_vector_database/tree/main
- BEIR GTE:
- BEIR BGE:
- FAISS Deep-1B:
- FAISS GIST-1M:
- TREC RAG 2024 Index: https://huggingface.co/datasets/Cohere/trec-rag-2024-index/tree/main/corpus/00

Not used:

- BEIR (datasets): https://huggingface.co/BeIR/datasets?p=1
- BEIR nfcorpus: https://huggingface.co/datasets/BeIR/nfcorpus
- (don't use) BEIR quora: https://huggingface.co/datasets/BeIR/quora/tree/main/corpus
- BEIR fever: https://huggingface.co/datasets/BeIR/fever/tree/main/corpus
- BEIR touche-2020: https://huggingface.co/datasets/BeIR/webis-touche2020/tree/main/corpus
- BEIR dbpedia-entity: https://huggingface.co/datasets/BeIR/dbpedia-entity/tree/main/corpus
- BEIR fiqa: https://huggingface.co/datasets/BeIR/fiqa/tree/main/corpus
- BEIR beir-corpus: https://huggingface.co/datasets/BeIR/beir-corpus
- Amazon QA: https://huggingface.co/datasets/embedding-data/Amazon-QA/blob/main/README.md#dataset-structure
- Google Natural Questions: https://huggingface.co/datasets/sentence-transformers/natural-questions/blob/main/README.md

Embedding model downloads:

- Qwen3 8B FP16:
- E5 Mistral 7B FP16:
- E5 Large V2 FP32: https://huggingface.co/ChristianAzinn/e5-large-v2-gguf/tree/main

- BAAI BGE-M3 (FP16): https://huggingface.co/gpustack/bge-m3-GGUF/tree/main
- Nomic Embed V1.5 FP32:
- Gemma 2 9B (FP16): https://huggingface.co/alamios/gemma-2-9b-it-GGUF-f16/tree/main
- Gemma 2 9B (FP32): (unused due to hardware constraints)
- Arctic Snowflake Embed L V2.0 (FP32): https://huggingface.co/Casual-Autopsy/snowflake-arctic-embed-l-v2.0-gguf/tree/main

Lower parameter versions of models:

- (replaces e5 mistral 7B) Harrier OSS V1 0.6B: https://huggingface.co/SuperPauly/harrier-oss-v1-0.6b-gguf/tree/main
- (Replaces Gemma 2 Embed 9B) EmbeddingGemma 300M: https://huggingface.co/second-state/embeddinggemma-300m-GGUF/tree/main
- (Replaces Qwen3 embed 8B) Qwen3 embed 4B: https://huggingface.co/Qwen/Qwen3-Embedding-4B-GGUF/tree/main

Proprietary embedding model downloads:

- OpenAI V3 Large:
- NV-Embed V2:
- Llama Nemotron Embedding 1B:
- Gemini Embedding 2: https://huggingface.co/google/embeddinggemma-300m/blob/main/README.md

Not used:

- Meta llama 8.1B: https://huggingface.co/meta-llama/Llama-3.1-8B/tree/main

5/1: Mini update: Reintroduces 2 large embedding models / Gemma 2 Embed 9B (FP16) & Qwen3 Embed 8B (FP16):

- Summary: I re-added support for these 2 models:
  - Gemma 2 Embed 9B (FP16)
  - Qwen3 Embed 8B (FP16)
- Note: these 2 models are very heavy, you will need to run them on beefier machines with more RAM/VRAM (16+GB VRAM for Qwen3, 19+GB VRAM for Gemma 2)
- To do next:
  - Implement experimental features:
    - Multimodal embedding support
    - Nvidia CUDA support

5/3: Large Update: Experimental Support for Multimodal Embeddings / 4 new embedding models: (Tim Update) [B0.9.8]

- Summary:

- 4 new multimodal embedding models were added:
    - Laion.ai CLAP
    - ImageBind by Meta
    - CLIP ViT-L/14 by OpenAI
    - Qwen3-VL by Alibaba
- New supported file formats: (only in not-precomputed mode)
    - Images: .jpg, .png
    - Video: .mp4, .mov
    - Audio: .mp3, .wav
- Here is what each model supports:
-

Medium:	CLAP	ImageBind	CLIP ViT-L/14	Qwen3-VL
Images:	No	Yes	Yes	Yes
Videos:	No	Yes	Yes*	Yes
Audio:	Yes	Yes	No	No

- *Not natively supported, I added experimental support for it
- Each model handles data differently:
-

Model:	Images:	Videos:	Audio:
CLAP	N/A	N/A	Chunks by 10 second segments. 1 vector embedding per chunk
ImageBind	1 vector embedding per file	1 vector embedding per file	1 vector embedding per file
CLIP ViT-L/14	Does chunk patterning. Downsamples to 224x224, splits into chunks of 16x16 pixels, each chunk gets its own vector embedding. So, 257 vector embeddings in total. First vector embedding is the semantics of the entire image. The next 256 vector	Each sampled frame is treated as an image, which is then converted into a vector embedding (1 vector embedding per frame)	N/A

	embeddings represent each 16x16 pattern chunk.		
Qwen3-VL	Images are split into 512 chunks, most likely horizontal lines. It does this for semantic feature recognition. Each chunk gets a vector embedding row. (512 rows of vector embeddings per image)	1 vector per sampled frame of video. Each sampled frame is treated like an image.	N/A

- Here is some more information about each embedding model:
-

Metric:	CLAP	ImageBind	CLIP ViT-L/14	Qwen3-VL
Dimensions:	512	1024	768	2560
Max Token Context:	Not documented	Not documented	77*	32768
Datatype:	FP16*	FP16*	FP16*	FP16*
Special Characteristic:	Max audio context: 10 seconds	N/A	N/A	N/A

- *Not confirmed
- Notes on experimental nature + data integrity:
    - Methods weren't checked for data integrity, and these features are still in very early stages
    - I wouldn't trust these for experimental data collection yet, more rigorous testing is needed
    - A decision must be made on the granularity of data to be used in the simulation for multimodal embeddings
- Notes on compatibility:
    - ImageBind is highly unstable, and may need multiple library prerequisites and even modifications. (the modifications are less likely if you aren't using the Intel SYCL backend)
    - Multimodal embeddings in general are way less standardized than the text embeddings

- To do next:
    - [B0.9.9] Implement/Test NVIDIA CUDA support
    - [R1.0.0] Clean up code, UI, settings/parameters, and comments. Also update the github homepage and add a full release on github
    - [R1.x.x?] Add elastic precision retrieval

5/5-5/6/26: Large Update: CUDA and CPU support validated + Multimodal Embedding Model Refinements + NUMA Optimization for text embeddings / Custom Datasets + All datasets: (Tim Update) [B0.9.9]

- Progress report:
    - CUDA support works on text and multimodal embeddings
    - CPU works for both text and multimodal embeddings
    - New setting lets one toggle file-level RAG retrieval vs Advanced RAG retrieval (defaults to industry standard file-level RAG retrieval)(setting is named FILE_RETRIEVAL)
    - Removed ImageBind model, too deprecated, too unstable
    - Changed Qwen3-VL from Instruct 4B model to Embed 2B model
    - Changed Qwen3-VL behavior to default to 1 vector embedding per file
    - Changed default multimodal behavior to output 1 vector embedding per file
    - Added NUMA optimizations to text embedding models
    - Removed unused settings/parameters from top, slightly updated settings comments (in preparation for R1.0.0 full release)
- Known issues:
    - Intel ARC SYCL backend doesn't play nicely with multimodal embedding libraries
    - Certain models (E5 large, harrier oss, embeddinggemma) give garbage output on Intel ARC backends, use CPU for best reliability
- To do next 5/7+/26: [R1.0.0]
    - Clean up code comments
    - Clean up code UI
    - Remove commented out and unnecessary code
    - Documentation side:
        - Update Github home page
        - Add library requirements + minimum versions
        - Add page of current known issues
        - Create usage notes (how to use, compatibility, system requirements, etc)
        - Slightly update license to require your name to be mentioned when forked
        - Create Full-release R1.0.0 + Channelog
        - Update to-do next (R1.x.x+)